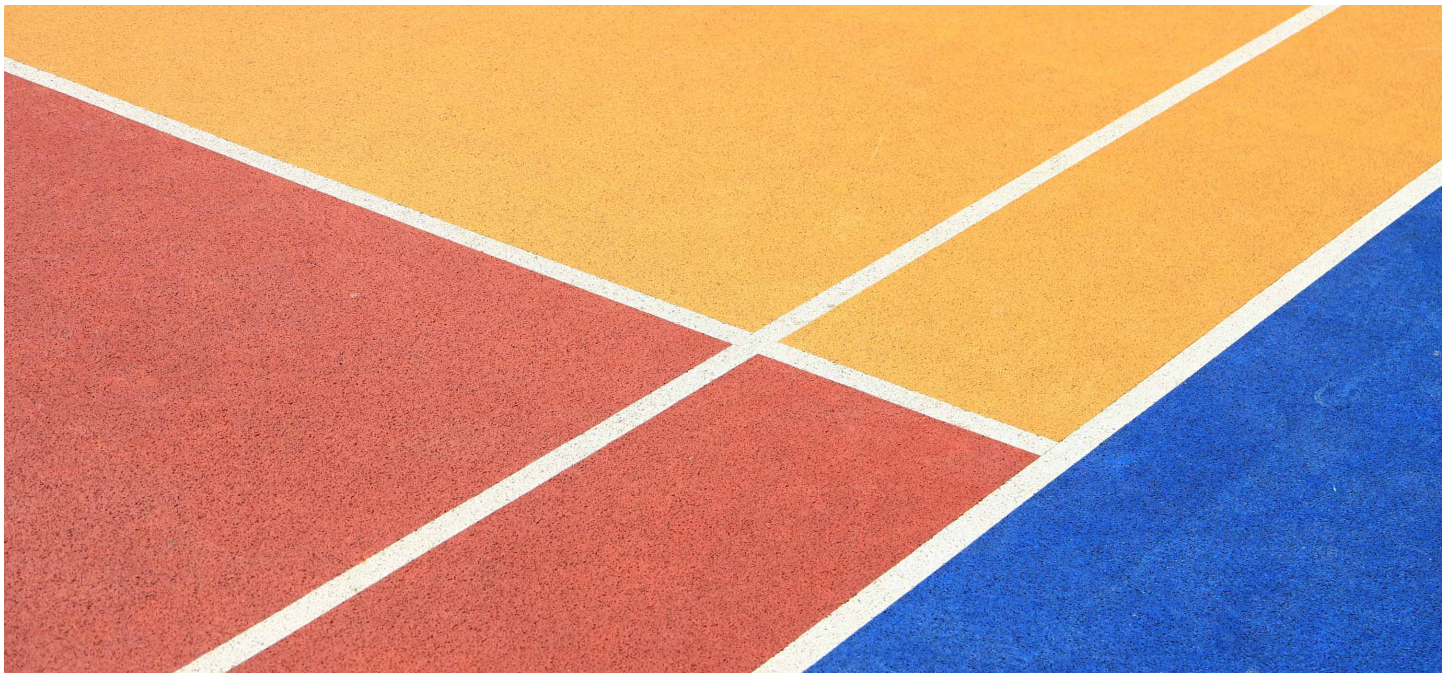




01_SparkCore

1. Spark简介

1.1 什么是Spark

Spark是一种快速、通用、可扩展的**大数据分析引擎**，2009年诞生于加州大学伯克利分校AMPLab，2010年开源，2013年6月成为Apache孵化项目，2014年2月成为Apache的顶级项目，2014年5月发布spark1.0，2016年7月发布spark2.0，2020年6月18日发布spark3.0.0



[Download](#) [Libraries ▾](#) [Documentation ▾](#) [Examples](#) [Community ▾](#) [Developers ▾](#)

Apache Spark™ is a unified analytics engine for large-scale data processing.

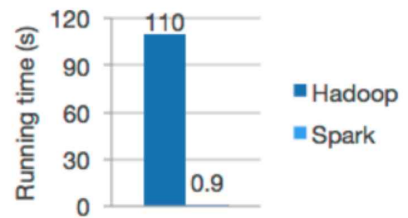
1.2 Spark的特点

- **Speed：快速高效**

Speed

Run workloads 100x faster.

Apache Spark achieves high performance for both batch and streaming data, using a state-of-the-art DAG scheduler, a query optimizer, and a physical execution engine.



Logistic regression in Hadoop and Spark

Hadoop的MapReduce作为第一代分布式大数据计算引擎，在设计之初，受当时计算机硬件条件所限（内存、磁盘、cpu等），为了能够计算海量数据，需要将中间结果保存到HDFS中，那么就要频繁读写HDFS从而使得网络IO和磁盘IO成为性能瓶颈。Spark可以将中间结果写到本地磁盘或将中间cache到内存中，节省了大量的网络IO和磁盘IO开销。并且Spark使用更先进的DAG任务调度思想，可以将多个计算逻辑构建成一个有向无环图，并且还会将DAG先进行优化后再生成物理执行计划，同时Spark也支持数据缓存在内存中的计算。性能比Hadoop MapReduce快100倍。即便是不将数据cache到内存中，其速度也是MapReduce10 倍以上。

- **Ease of Use：简洁易用**

Ease of Use

Write applications quickly in Java, Scala, Python, R, and SQL.

Spark offers over 80 high-level operators that make it easy to build parallel apps. And you can use it *interactively* from the Scala, Python, R, and SQL shells.

```
df =  
spark.read.json("logs.json")  
df.where("age > 21")  
  .select("name.first").show()
```

Spark's Python DataFrame API
Read JSON files with automatic schema
inference

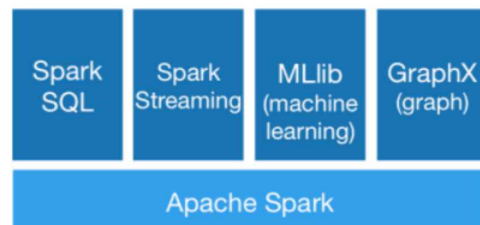
Spark支持 Java、Scala、Python和R等编程语言编写应用程序，大大降低了使用者的门槛。自带了80多个高等级操作算子，并且允许在Scala，Python，R 的使用命令进行交互式运行，可以非常方便的在Spark Shell中地编写spark程序。

- **Generality：通用、全栈式数据处理**

Generality

Combine SQL, streaming, and complex analytics.

Spark powers a stack of libraries including [SQL and DataFrames](#), [MLlib](#) for machine learning, [GraphX](#), and [Spark Streaming](#). You can combine these libraries seamlessly in the same application.



Spark提供了统一的大数据处理解决方案，非常具有吸引力，毕竟任何公司都想用统一的平台去处理遇到的问题，减少开发和维护的人力成本和部署平台的物力成本。同时Spark还支持SQL，大大降低了大数据开发者的使用门槛，同时提供了SparkStream和Structured Streaming可以处理实时流数据；MLlib机器学习库，提供机器学习相关的统计、分类、回归等领域的多种算法实现。其高度封装的API接口大大降低了用户的学习成本；Spark GraghX提供分布式图计算处理能力；PySpark支持Python编写Spark程序；SparkR支持R语言编写Spark程序。

- **Runs Everywhere：可以运行在各种资源调度框架和读写多种数据源**

Runs Everywhere

Spark runs on Hadoop, Apache Mesos, Kubernetes, standalone, or in the cloud. It can access diverse data sources.

You can run Spark using its [standalone cluster mode](#), on [EC2](#), on [Hadoop YARN](#), on [Mesos](#), or on [Kubernetes](#). Access data in [HDFS](#), [Alluxio](#), [Apache Cassandra](#), [Apache HBase](#), [Apache Hive](#), and hundreds of other data sources.



Spark支持的多种部署方案：Standalone是Spark自带的资源调度模式；Spark可以运行在Hadoop的YARN上面；Spark 可以运行在Mesos上（Mesos是一个类似于YARN的资源调度框架）；Spark还可以Kubernetes实现容器化的资源调度

丰富的数据源支持。Spark除了可以访问操作系统自身的本地文件系统和HDFS之外，还可以访问Cassandra、HBase、Hive、Alluxio（Tachyon）以及任何Hadoop兼容的数据源。这极大地方便了已经的大数据系统进行顺利迁移到Spark。

1.3 Spark与MapReduce的对比

框架	优点	缺点
MapReduce	历史悠久、稳定	编程API不灵活、速度慢、只能做离线计算
Spark	通用、编程API简洁、快	跟MapReduce比暂无缺点

面试题：MapReduce和Spark的本质区别：

- 1. MR只能做离线计算，如果实现复杂计算逻辑，一个MR搞不定，就需要将多个MR按照先后顺序连成一串，一个MR计算完成后会将计算结果写入到HDFS中，下一个MR将上一个MR的输出作为输入，这样就要频繁读写HDFS，网络IO和磁盘IO会成为性能瓶颈。从而导致效率低下。
- 2. 既可以做离线计算，又可以做实时计算，提供了抽象的数据集（RDD、Dataset、DataFrame、DStream）

有高度封装的API，算子丰富，并且使用了更先进的DAG有向无环图调度思想，可以对执行计划优化后在执行，并且可以数据可以cache到内存中进行复用。

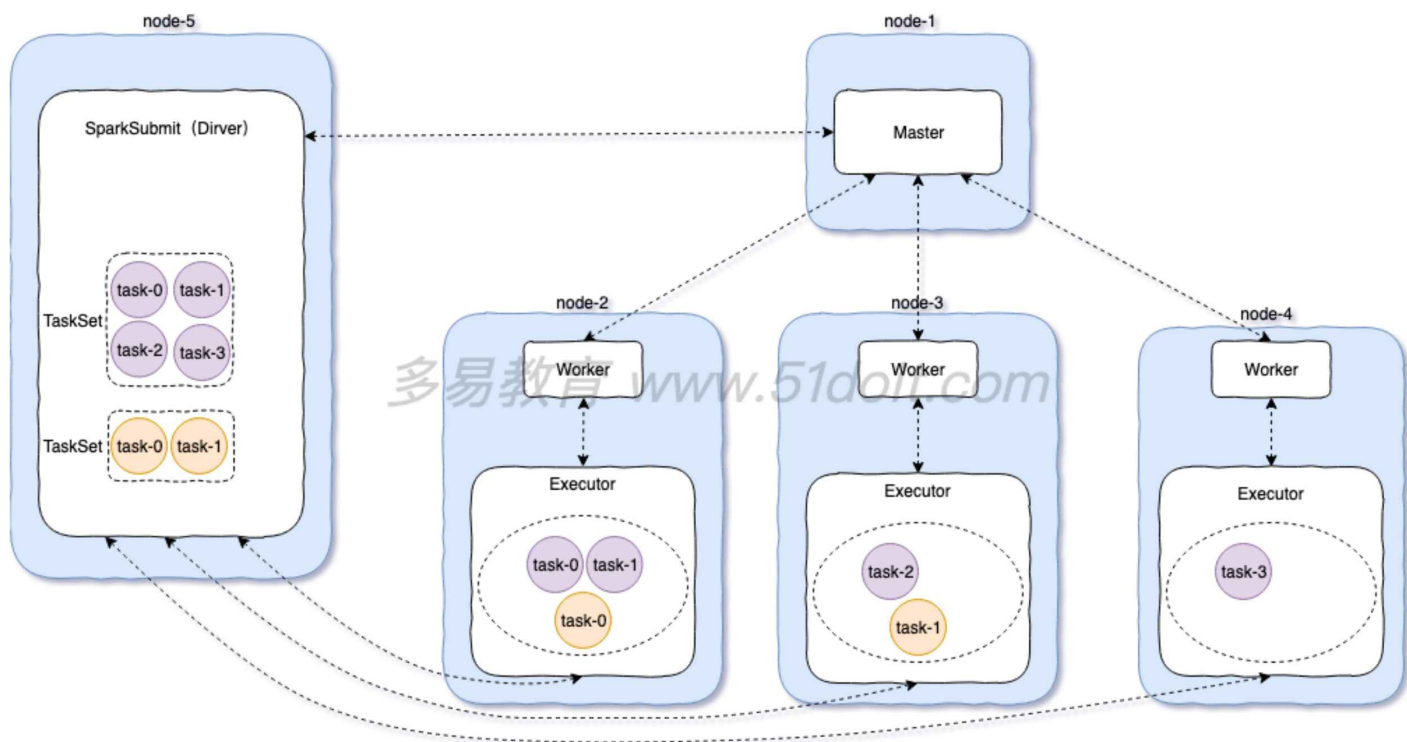
注意：MR和Spark在Shuffle时数据都落本地磁盘

2. Spark架构体系

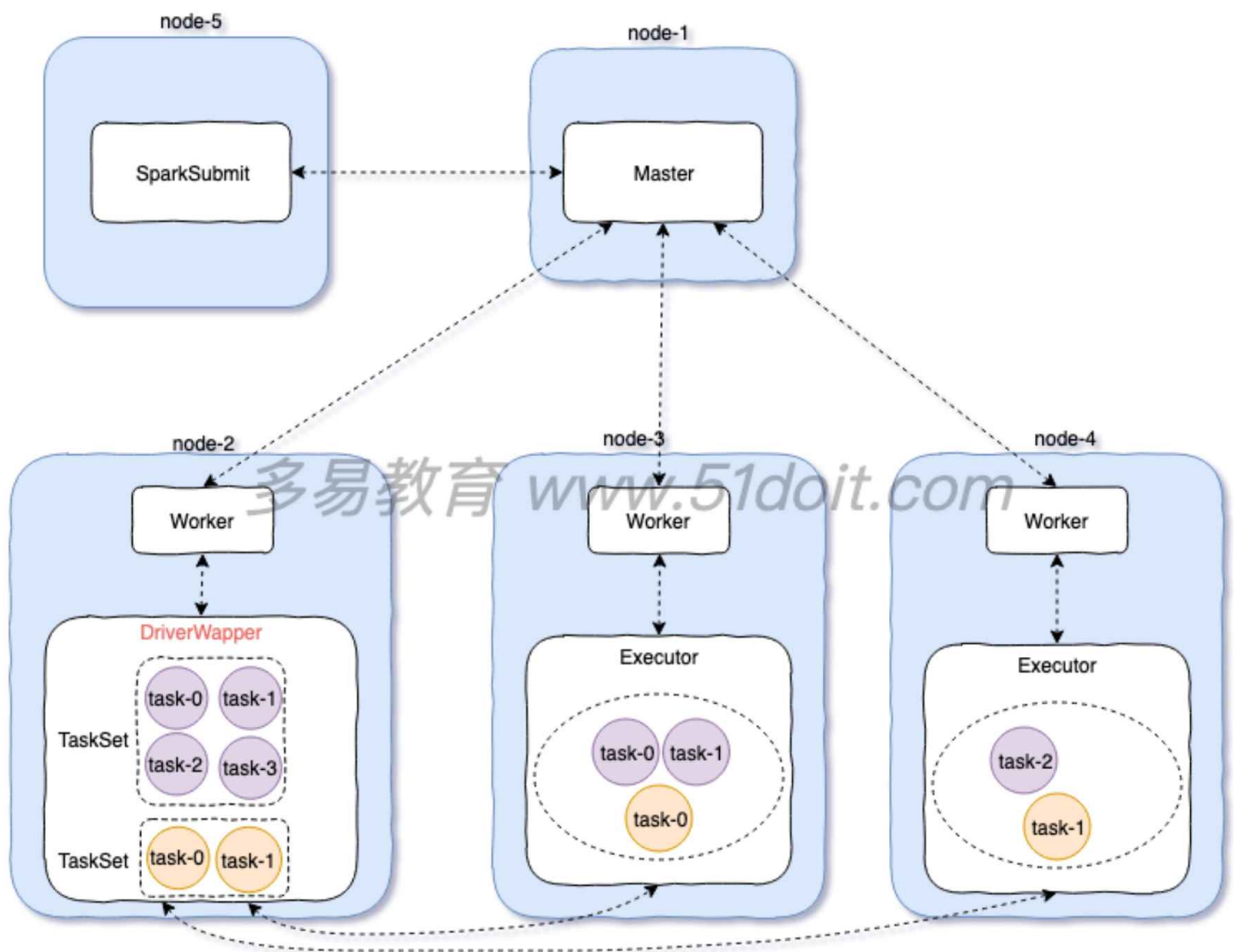
StandAlone模式是spark自带的集群运行模式，不依赖其他的资源调度框架，部署起来简单。

StandAlone模式又分为client模式和cluster模式，本质区别是Driver运行在哪里，如果Driver运行在SparkSubmit进程中就是Client模式，如果Driver运行在集群中就是Cluster模式

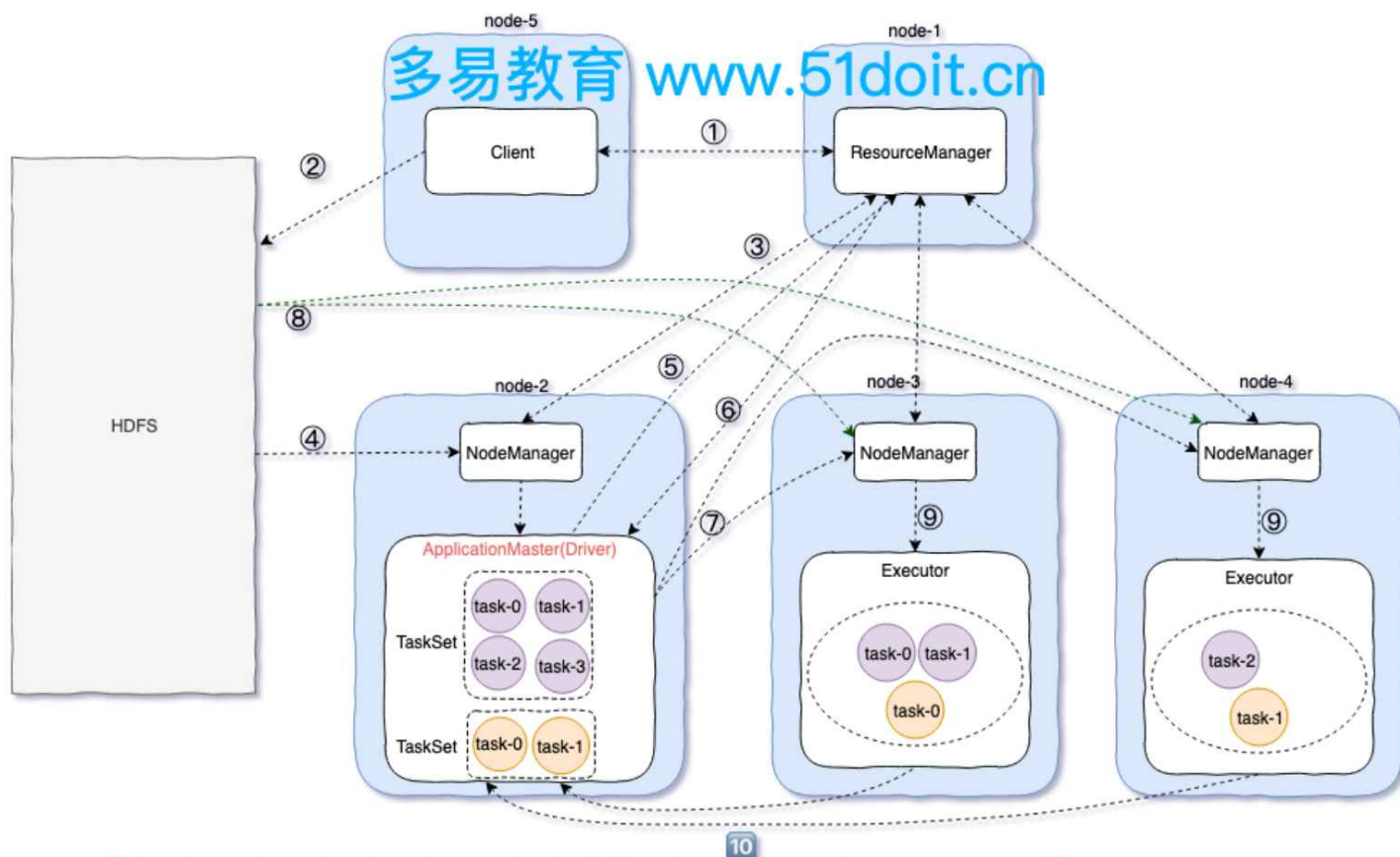
2.1 standalone client模式



2.2 standalone cluster模式



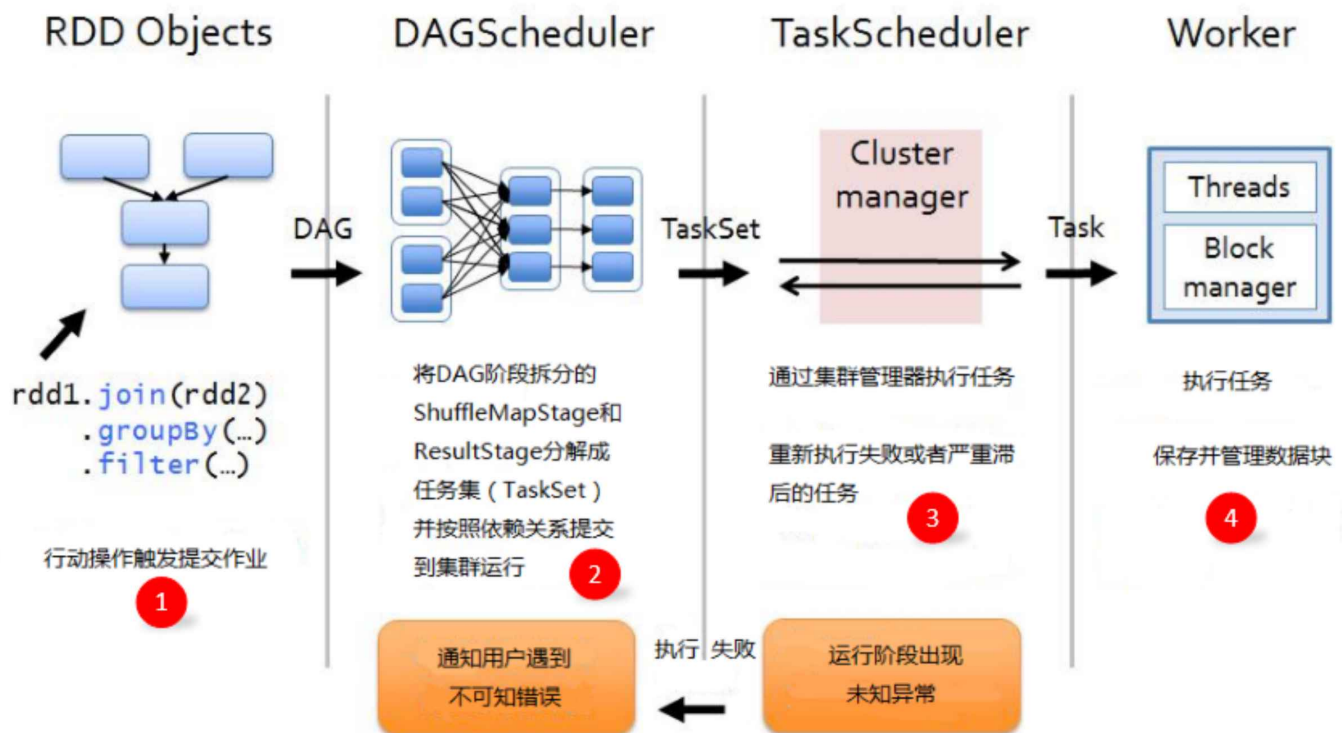
2.3 Spark On YARN cluster模式



Spark On Yarn Cluster模式执行流程

- ① client向ResourceManager申请资源，返回一个application ID
- ② client上传spark jars 下面的jar包、自己写的jar和配置
- ③ ResourceManager随机找一个资源充足的NodeManager
- ④ 然后通过RPC让NodeManager从HDFS上下载jar包和配置，启动ApplicationMaster
- ⑤ ApplicationMaster向ResourceManager申请资源
- ⑥ ResourceManager 中的 ResourceScheduler找到符合条件的NodeManager，将NodeManager的信息返回给ApplicationMaster
- ⑦ ApplicationMaster跟返回的NodeManager进行通信
- ⑧ NodeManager从HDFS下载下载依赖
- ⑨ NodeManager启动Executor
- ⑩ Executor启动之后反向向ApplicationMaster【Driver】注册

2.4 Spark执行流程简介



- Job: RDD每一个行动操作都会生成一个或者多个调度阶段 调度阶段 (Stage) : 每个Job都会根据依赖关系, 以Shuffle过程作为划分, 分为Shuffle Map Stage和Result Stage。每个Stage对应一个TaskSet, 一个Task中包含多Task, TaskSet的数量与该阶段最后一个RDD的分区数相同。
- Task: 分发到Executor上的工作任务, 是Spark的最小执行单元
- DAGScheduler: DAGScheduler是将DAG根据宽依赖将切分Stage, 负责划分调度阶段并Stage转成TaskSet提交给TaskScheduler
- TaskScheduler: TaskScheduler是将Task调度到Worker下的Exexcutor进程, 然后丢入到Executor的线程池的中进行执行

2.5 Spark中重要角色

- **Master** : 是一个Java进程, 接收Worker的注册信息和心跳、移除异常超时的Worker、接收客户端提交的任务、负责资源调度、命令Worker启动Executor。
- **Worker** : 是一个Java进程, 负责管理当前节点的资源管理, 向Master注册并定期发送心跳, 负责启动Executor、并监控Executor的状态。
- **SparkSubmit** : 是一个Java进程, 负责向Master提交任务。

- **Driver**：是很多类的统称，可以认为SparkContext就是Driver，client模式Driver运行在SparkSubmit进程中，cluster模式单独运行在一个进程中，负责将用户编写的代码转成Tasks，然后调度到Executor中执行，并监控Task的状态和执行进度。
- **Executor**：是一个Java进程，负责执行Driver端生成的Task，将Task放入线程中运行。

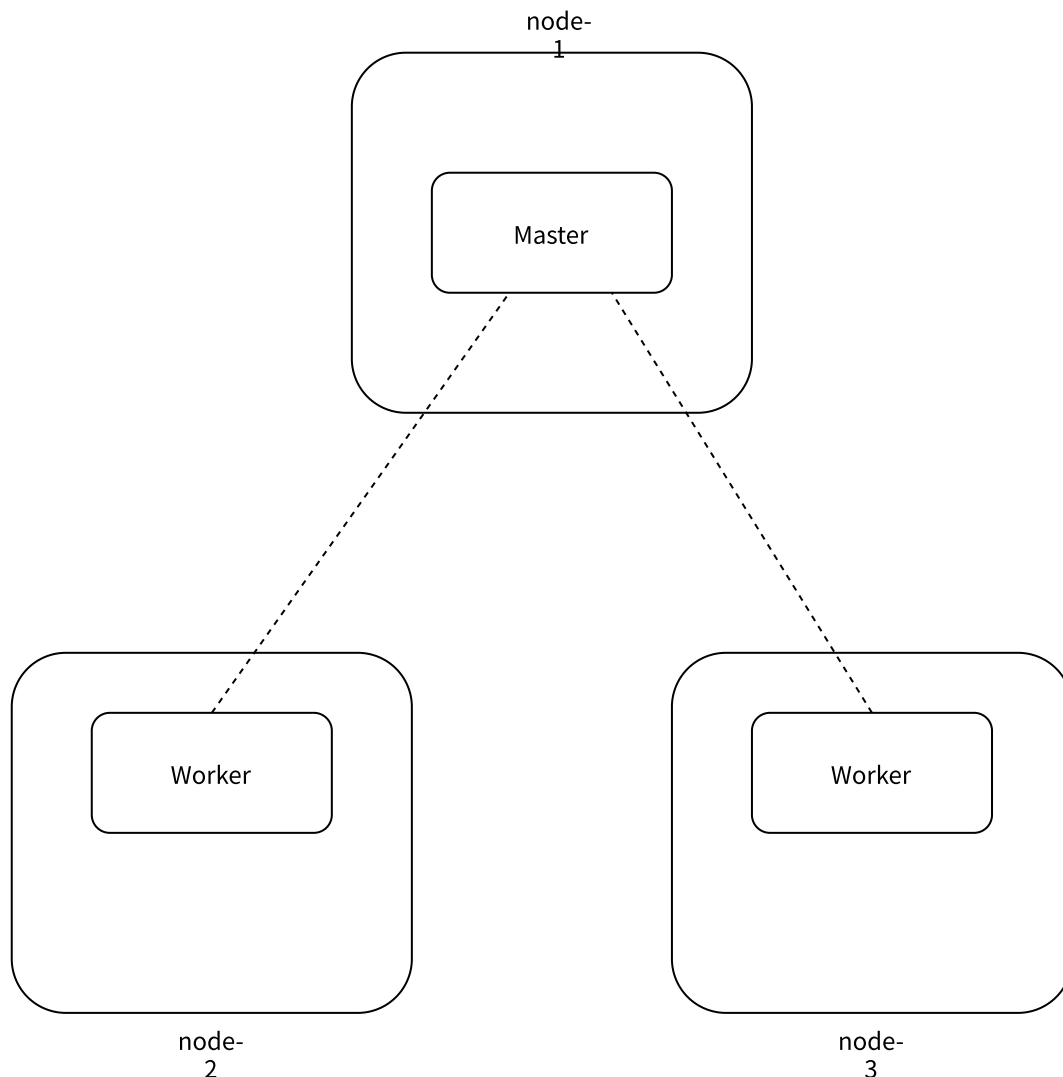
2.6 Spark和Yarn角色对比

Spark StandAlone的Client模式	YARN
Master	ResourceManager
Worker	NodeManager
Executor	YarnChild
SparkSubmit (Driver)	ApplicationMaster

3. StandAlone模式环境搭建

3.1 搭建步骤

环境准备：三台Linux，一个安装Master，其他两台机器安装Worker



1. 下载spark安装包，下载地址：<https://spark.apache.org/downloads.html>

Download Apache Spark™

1. Choose a Spark release: spark 版本
2. Choose a package type: 兼容 hadoop 的版本
3. Download Spark: [spark-3.2.3-bin-hadoop3.2.tgz](#) 下载地址
4. Verify this release using the 3.2.3 [signatures](#), [checksums](#) and [project release KEYS](#) by following these [procedures](#).

Note that Spark 3 is pre-built with Scala 2.12 in general and Spark 3.2+ provides additional pre-built distribution with Scala 2.13.

2. 上传spark安装包到Linux服务器上
3. 解压spark安装包

```
1 tar -zxvf spark-3.2.3-bin-hadoop3.2.tgz -C /bigdata/
```

4. 进入到spark按照包目录并将conf目录下的spark-env.sh.template重命名为spark-env.sh，再修改

```
1 export JAVA_HOME=/usr/local/jdk1.8.0_251/  
2 export SPARK_MASTER_HOST=node-1.51doit.cn
```

5. 将conf目录下的workers.template重命名为workers并修改，指定Worker的所在节点

```
1 node-2.51doit.cn  
2 node-3.51doit.cn
```

6. 将配置好的spark拷贝到其他节点

```
1 for i in {2..3}; do scp -r spark-3.2.3-bin-hadoop3.2 node-${i}.51doit.cn:$PWD; don
```

3.2 启动Spark集群

- 在Spark的安装目录执行启动脚本

```
1 sbin/start-all.sh
```

- 执行jps命令查看Java进程

```
1 jps
```

在ndoe-1上可以看见Master进程，在其他的节点上可以看见到Worker进程

- 访问Master的web管理界面，端口8080

← → ↺ 不安全 | node-1.51doit.cn:8080

APACHE
spark3.2.3

Spark Master at spark://node-1.51doit.cn:7077

URL: spark://node-1.51doit.cn:7077

Alive Workers: 2

Cores in use: 4 Total, 0 Used

Memory in use: 2.0 GiB Total, 0.0 B Used

Resources in use:

Applications: 0 Running, 0 Completed

Drivers: 0 Running, 0 Completed

Status: ALIVE

Workers (2)

Worker Id	Address	State	Cores	Memory	Resources
worker-20221222181128-172.16.100.102-35589	172.16.100.102:35589	ALIVE	2 (0 Used)	1024.0 MiB (0.0 B Used)	
worker-20221222181128-172.16.100.103-41037	172.16.100.103:41037	ALIVE	2 (0 Used)	1024.0 MiB (0.0 B Used)	

Running Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

Completed Applications (0)

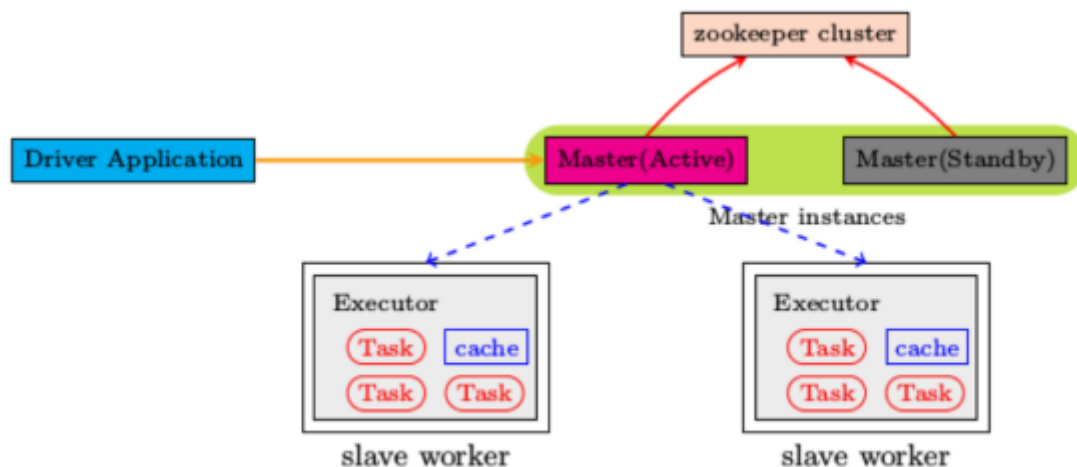
Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

3.3 一些重要参数

```
1 export SPARK_WORKER_CORES=4      #指定worker可用的逻辑核数
2 export SPARK_WORKER_MEMORY=2g    #指定worker可用的内存大小
```

3.4 standalone模式高可用部署

spark的standalone模式可以启动两个以上的Master，但是需要依赖zookeeper进行协调，所有的节点启动后，都向zk注册



修改配置文件spark-env.sh

```
1 # 注释掉master的地址，所有节点都先连接zookeeper
2 # export SPARK_MASTER_HOST=node-1.51doit.cn
3 export SPARK_DAEMON_JAVA_OPTS="-Dspark.deploy.recoveryMode=ZOOKEEPER -
  Dspark.deploy.zookeeper.url=node-1.51doit.cn:2181,node-2.51doit.cn:2181,node-
  3.51doit.cn:2181 -Dspark.deploy.zookeeper.dir=/spark"
```

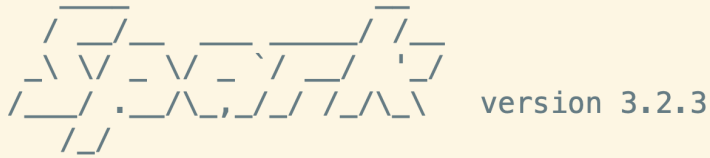
4. 启动Spark Shell编程

4.1 什么是Spark Shell

spark shell是spark中的交互式命令行客户端，可以在spark shell中使用scala编写spark程序，启动后默认已经创建了SparkContext，别名为sc

4.2 启动Spark Shell

```
library for your platform... using builtin-java classes where applicable
Spark context Web UI available at http://node-1.51doit.cn:4040
Spark context available as 'sc' (master = spark://node-1.51doit.cn:7077,node-2.51doit.cn:7077, app id = app-20221222191524-0000).
Spark session available as 'spark'.
Welcome to
```



```
Using Scala version 2.12.15 (Java HotSpot(TM) 64-Bit Server VM, Java 1.8.0_241)
Type in expressions to have them evaluated.
Type :help for more information.
```

```
scala>
```

```
1 /bigdata/spark-3.2.3-bin-hadoop3.2/bin/spark-shell \  
2 --master spark://node-1.51doit.cn:7077 --executor-memory 1g \  
3 --total-executor-cores 3
```

如果Master配置了HA高可用，需要指定两个Master（因为这两个Master任意一个都可能是Active状态）

```
1 /bigdata/spark-3.2.3-bin-hadoop3.2/bin/spark-shell \  
2 --master spark://node-1.51doit.cn:7077,node-2.51doit.cn:7077 \  
3 --executor-memory 1g \  
4 --total-executor-cores 3
```

参数说明：

--master 指定masterd地址和端口，协议为spark://，端口是RPC的通信端口

--executor-memory 指定每一个executor的使用的内存大小

--total-executor-cores指定整个application总共使用了cores

- 在shell中编写第一个spark程序

```
1 sc.textFile("hdfs://node-1.51doit.cn:9000/words.txt").flatMap(_.split("
  ")).map(_ ,
  1)).reduceByKey(_+_).sortBy(_._2,false).saveAsTextFile("hdfs://node-
  1.51doit.cn:9000/out")
```

5. Spark编程入门

5.1 Scala编写Spark的WorkCount

5.1.1 创建一个Maven项目

5.1.2 在pom.xml中添加依赖和插件

```
1 <!-- 定义的一些常量 -->
2 <properties>
3     <maven.compiler.source>8</maven.compiler.source>
4     <maven.compiler.target>8</maven.compiler.target>
5     <encoding>UTF-8</encoding>
6     <spark.version>3.2.3</spark.version>
7     <scala.version>2.12.15</scala.version>
8 </properties>
9
10 <dependencies>
11     <!-- scala的依赖 -->
12     <dependency>
13         <groupId>org.scala-lang</groupId>
14         <artifactId>scala-library</artifactId>
15         <version>${scala.version}</version>
16     </dependency>
17
18     <!-- spark core 即为spark内核，其他高级组件都要依赖spark core -->
19     <dependency>
20         <groupId>org.apache.spark</groupId>
21         <artifactId>spark-core_2.12</artifactId>
22         <version>${spark.version}</version>
23     </dependency>
24
25 </dependencies>
26
27 <!-- 配置Maven的镜像库 -->
```

```
28 <!-- 依赖下载国内镜像库 -->
29 <repositories>
30   <repository>
31     <id>nexus-aliyun</id>
32     <name>Nexus aliyun</name>
33     <layout>default</layout>
34     <url>http://maven.aliyun.com/nexus/content/groups/public</url>
35     <snapshots>
36       <enabled>>false</enabled>
37       <updatePolicy>never</updatePolicy>
38     </snapshots>
39     <releases>
40       <enabled>true</enabled>
41       <updatePolicy>never</updatePolicy>
42     </releases>
43   </repository>
44 </repositories>
45
46 <!-- maven插件下载国内镜像库 -->
47 <pluginRepositories>
48   <pluginRepository>
49     <id>ali-plugin</id>
50     <url>http://maven.aliyun.com/nexus/content/groups/public/</url>
51     <snapshots>
52       <enabled>>false</enabled>
53       <updatePolicy>never</updatePolicy>
54     </snapshots>
55     <releases>
56       <enabled>true</enabled>
57       <updatePolicy>never</updatePolicy>
58     </releases>
59   </pluginRepository>
60 </pluginRepositories>
61
62 <build>
63   <pluginManagement>
64     <plugins>
65       <!-- 编译scala的插件 -->
66       <plugin>
67         <groupId>net.alchim31.maven</groupId>
68         <artifactId>scala-maven-plugin</artifactId>
69         <version>3.2.2</version>
70       </plugin>
71       <!-- 编译java的插件 -->
72       <plugin>
73         <groupId>org.apache.maven.plugins</groupId>
74         <artifactId>maven-compiler-plugin</artifactId>
```

```
75         <version>3.5.1</version>
76     </plugin>
77 </plugins>
78 </pluginManagement>
79 <plugins>
80     <plugin>
81         <groupId>net.alchim31.maven</groupId>
82         <artifactId>scala-maven-plugin</artifactId>
83         <executions>
84             <execution>
85                 <id>scala-compile-first</id>
86                 <phase>process-resources</phase>
87                 <goals>
88                     <goal>add-source</goal>
89                     <goal>compile</goal>
90                 </goals>
91             </execution>
92             <execution>
93                 <id>scala-test-compile</id>
94                 <phase>process-test-resources</phase>
95                 <goals>
96                     <goal>testCompile</goal>
97                 </goals>
98             </execution>
99         </executions>
100     </plugin>
101
102     <plugin>
103         <groupId>org.apache.maven.plugins</groupId>
104         <artifactId>maven-compiler-plugin</artifactId>
105         <executions>
106             <execution>
107                 <phase>compile</phase>
108                 <goals>
109                     <goal>compile</goal>
110                 </goals>
111             </execution>
112         </executions>
113     </plugin>
114
115     <!-- 打jar插件 -->
116     <plugin>
117         <groupId>org.apache.maven.plugins</groupId>
118         <artifactId>maven-shade-plugin</artifactId>
119         <version>2.4.3</version>
120         <executions>
121             <execution>
```

```

122         <phase>package</phase>
123         <goals>
124             <goal>shade</goal>
125         </goals>
126         <configuration>
127             <filters>
128                 <filter>
129                     <artifact>*:*</artifact>
130                     <excludes>
131                         <exclude>META-INF/*.SF</exclude>
132                         <exclude>META-INF/*.DSA</exclude>
133                         <exclude>META-INF/*.RSA</exclude>
134                     </excludes>
135                 </filter>
136             </filters>
137         </configuration>
138     </execution>
139 </executions>
140 </plugin>
141 </plugins>
142 </build>

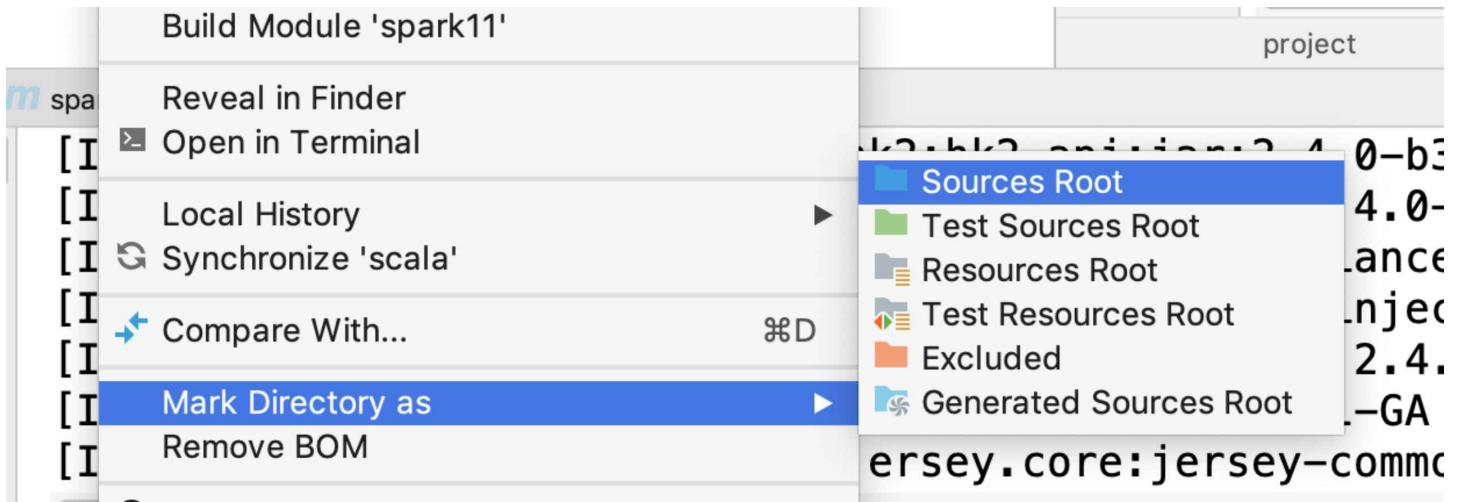
```

- 注意：将粘贴的内容拷贝到指定的位置



5.1.3 创建一个scala目录

选择scala目录，右键，将目录转成源码包，或者点击maven的刷新按钮



5.1.4 编写Spark程序

```
1 import org.apache.spark.rdd.RDD
2 import org.apache.spark.{SparkConf, SparkContext}
3
4 /**
5  * 1.创建SparkContext
6  * 2.创建RDD
7  * 3.调用RDD的Transformation (s) 方法
8  * 4.调用Action
9  * 5.释放资源
10 */
11 object WordCount {
12
13   def main(args: Array[String]): Unit = {
14
15     val conf: SparkConf = new SparkConf().setAppName("WordCount")
16     //创建SparkContext, 使用SparkContext来创建RDD
17     val sc: SparkContext = new SparkContext(conf)
18     //spark写Spark程序, 就是对抽象的神奇的大集合【RDD】编程, 调用它高度封装的API
19     //使用SparkContext创建RDD
20     val lines: RDD[String] = sc.textFile(args(0))
21
22     //Transformation 开始 //
23     //切分压平
24     val words: RDD[String] = lines.flatMap(_.split(" "))
25     //将单词和一组组合放在元组中
26     val wordAndOne: RDD[(String, Int)] = words.map((_, 1))
27     //分组聚合, reduceByKey可以先局部聚合再全局聚合
28     val reduced: RDD[(String, Int)] = wordAndOne.reduceByKey(_+_ )
29     //排序
30     val sorted: RDD[(String, Int)] = reduced.sortBy(_._2, false)
```

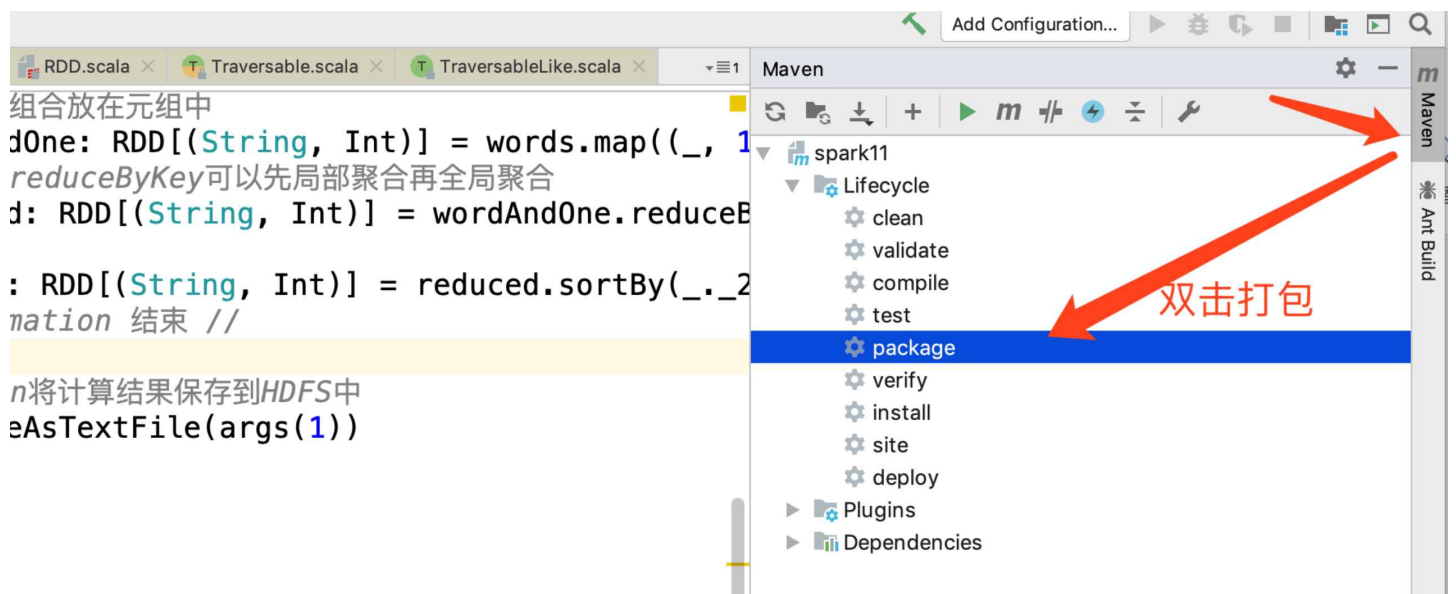
```

31    //Transformation 结束 //
32
33    //调用Action将计算结果保存到HDFS中
34    sorted.saveAsTextFile(args(1))
35    //释放资源
36    sc.stop()
37 }
38 }

```

5.1.5 使用maven打包

- 使用idea图形界面打包：



- 使用maven命令打包（两种方式任选其一）

```
1 mvn clean package
```

5.1.6 提交任务

- 上传jar包到服务器，然后使用sparksubmit命令提交任务

```

1 /bigdata/spark-3.2.3-bin-hadoop3.2/bin/spark-submit \
2 --master spark://node-1.51doit.cn:7077 \
3 --executor-memory 1g --total-executor-cores 4 \

```

```
4 --class cn._51doit.spark.day01.WordCount \  
5 /root/spark-in-action-1.0.jar hdfs://node-1.51doit.cn:9000/words.txt hdfs://node
```

参数说明:

--master 指定master地址和端口, 协议为spark://, 端口是RPC的通信端口

--executor-memory 指定每一个executor的使用的内存大小

--total-executor-cores指定整个application总共使用了cores

--class 指定程序的main方法全类名

jar包路径 args0 args1

5.2 Java编写Spark的WordCount

5.2.1 使用匿名实现类方式

```
1 import org.apache.spark.SparkConf;  
2 import org.apache.spark.api.java.JavaPairRDD;  
3 import org.apache.spark.api.java.JavaRDD;  
4 import org.apache.spark.api.java.JavaSparkContext;  
5 import org.apache.spark.api.java.function.FlatMapFunction;  
6 import org.apache.spark.api.java.function.Function2;  
7 import org.apache.spark.api.java.function.PairFunction;  
8 import scala.Tuple2;  
9  
10 import java.util.Arrays;  
11 import java.util.Iterator;  
12  
13 public class JavaWordCount {  
14  
15     public static void main(String[] args) {  
16         SparkConf sparkConf = new SparkConf().setAppName("JavaWordCount");  
17         //创建JavaSparkContext  
18         JavaSparkContext jsc = new JavaSparkContext(sparkConf);  
19         //使用JavaSparkContext创建RDD  
20         JavaRDD<String> lines = jsc.textFile(args[0]);  
21         //调用Transformation (s)  
22         //切分压平  
23         JavaRDD<String> words = lines.flatMap(new FlatMapFunction<String, String
```

```

24         @Override
25         public Iterator<String> call(String line) throws Exception {
26             return Arrays.asList(line.split(" ")).iterator();
27         }
28     });
29     //将单词和一组合在一起
30     JavaPairRDD<String, Integer> wordAndOne = words.mapToPair(
31         new PairFunction<String, String, Integer>() {
32             @Override
33             public Tuple2<String, Integer> call(String word) throws Exce
34                 return Tuple2.apply(word, 1);
35         }
36     });
37     //分组聚合
38     JavaPairRDD<String, Integer> reduced = wordAndOne.reduceByKey(
39         new Function2<Integer, Integer, Integer>() {
40             @Override
41             public Integer call(Integer v1, Integer v2) throws Exception {
42                 return v1 + v2;
43             }
44         });
45     //排序, 先调换KV的顺序VK
46     JavaPairRDD<Integer, String> swapped = reduced.mapToPair(
47         new PairFunction<Tuple2<String, Integer>, Integer, String>() {
48             @Override
49             public Tuple2<Integer, String> call(Tuple2<String, Integer> tp) thro
50                 return tp.swap();
51         }
52     );
53     //再排序
54     JavaPairRDD<Integer, String> sorted = swapped.sortByKey(false);
55     //再调换顺序
56     JavaPairRDD<String, Integer> result = sorted.mapToPair(
57         new PairFunction<Tuple2<Integer, String>, String, Integer>() {
58             @Override
59             public Tuple2<String, Integer> call(Tuple2<Integer, String> tp) thro
60                 return tp.swap();
61         }
62     );
63     //触发Action, 将数据保存到HDFS
64     result.saveAsTextFile(args[1]);
65     //释放资源
66     jsc.stop();
67 }
68 }
69

```

5.2.2 使用Lambda表达式方式

```
1 import org.apache.spark.SparkConf;
2 import org.apache.spark.api.java.JavaPairRDD;
3 import org.apache.spark.api.java.JavaRDD;
4 import org.apache.spark.api.java.JavaSparkContext;
5 import scala.Tuple2;
6
7 import java.util.Arrays;
8
9 public class JavaLambdaWordCount {
10
11     public static void main(String[] args) {
12         SparkConf conf = new SparkConf().setAppName("JavaLambdaWordCount");
13         //创建SparkContext
14         JavaSparkContext jsc = new JavaSparkContext(conf);
15         //创建RDD
16         JavaRDD<String> lines = jsc.textFile(args[0]);
17         //切分压平
18         JavaRDD<String> words = lines.flatMap(line -> Arrays.asList(line.split("
19 //将单词和一组合
20 JavaPairRDD<String, Integer> wordAndOne = words.mapToPair(word -> Tuple2
21 //分组聚合
22 JavaPairRDD<String, Integer> reduced = wordAndOne.reduceByKey((a, b) ->
23 //调换顺序
24 JavaPairRDD<Integer, String> swapped = reduced.mapToPair(tp -> tp.swap()
25 //排序
26 JavaPairRDD<Integer, String> sorted = swapped.sortByKey(false);
27 //调换顺序
28 JavaPairRDD<String, Integer> result = sorted.mapToPair(tp -> tp.swap());
29 //将数据保存到HDFS
30 result.saveAsTextFile(args[1]);
31 //释放资源
32 jsc.stop();
33     }
34 }
```

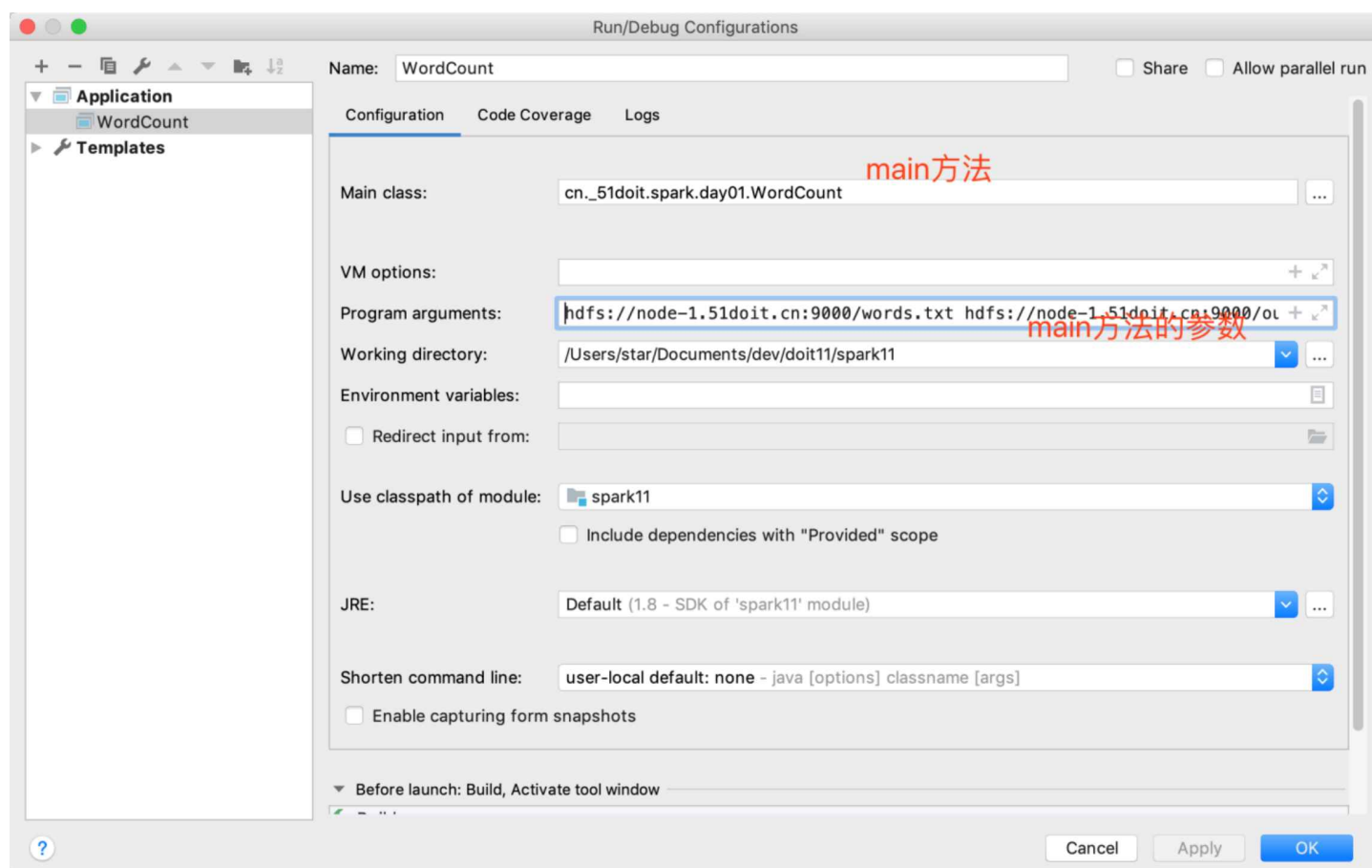
5.3 本地运行Spark和Debug

spark程序每次都打包上在提交到集群上比较麻烦且不方便调试，Spark还可以进行Local模式运行，方便测试和调试

5.3.1 在本地运行

```
1 //Spark程序local模型运行,local[*]是本地运行,并开启多个线程
2 val conf: SparkConf = new SparkConf()
3   .setAppName("WordCount")
4   .setMaster("local[*]") //设置为local模式执行
```

- 输入运行参数



5.3.2 读取HDFS中的数据

由于往HDFS中的写入数据存在权限问题，所以在代码中设置用户为HDFS目录的所属用户

- 1 //往HDFS中写入数据，将程序的所属用户设置成更HDFS一样的用户
- 2 `System.setProperty("HADOOP_USER_NAME", "root")`

5.4 使用PySpark

5.4.1 配置python环境

- ① 在所有节点上按照python3，版本必须是python3.6及以上版本

```
1 yum install -y python3
```

Dependencies Resolved				
Package	Arch	Version	Repository	Size
Installing:				
python3	x86_64	3.6.8-13.el7	base	69 k
Installing for dependencies:				
libtirpc	x86_64	0.2.4-0.16.el7	base	89 k
python3-libs	x86_64	3.6.8-13.el7	base	7.0 M
python3-pip	noarch	9.0.3-7.el7_7	updates	1.8 M
python3-setuptools	noarch	39.2.0-10.el7	base	629 k
Transaction Summary				

- ② 修改所有节点的环境变量

```
1 export JAVA_HOME=/usr/local/jdk1.8.0_251
2 export PYSPARK_PYTHON=python3
3 export HADOOP_HOME=/bigdata/hadoop-3.2.1
4 export HADOOP_CONF_DIR=/bigdata/hadoop-3.2.1/etc/hadoop
5 export PATH=$PATH:$JAVA_HOME/bin:$HADOOP_HOME/bin
```

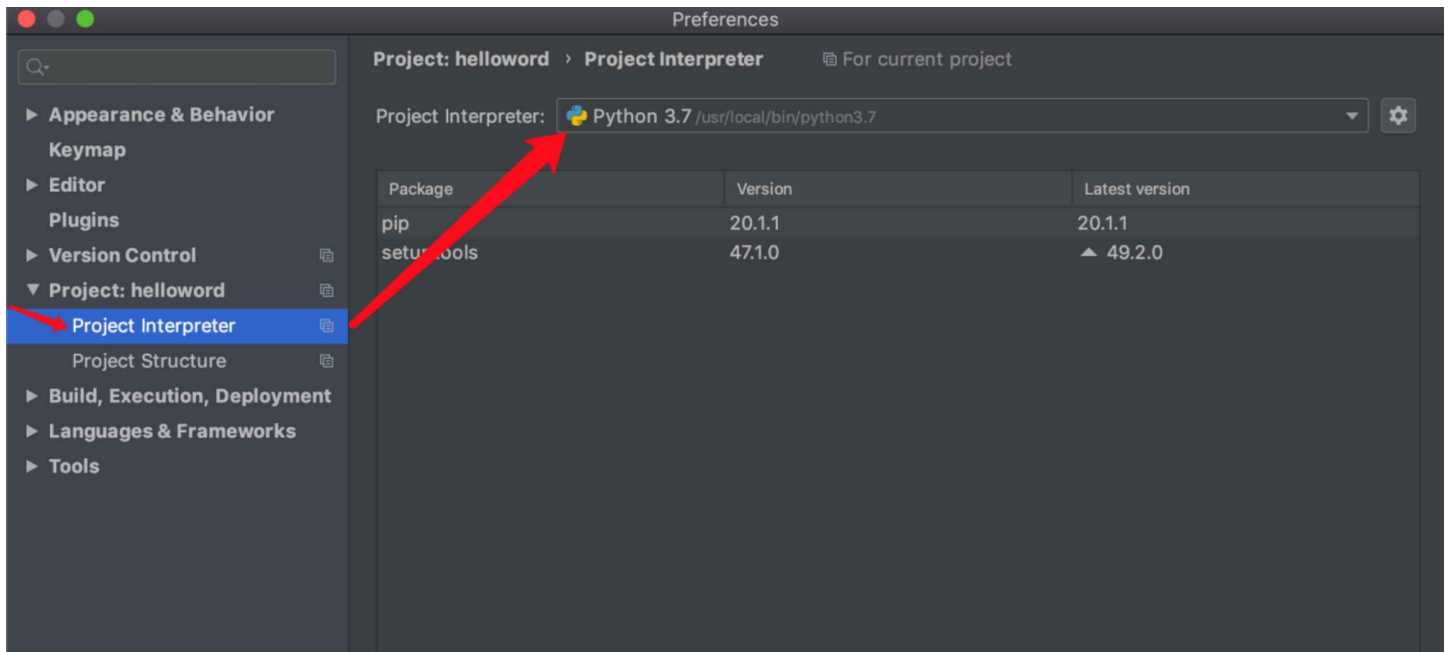
5.4.2 使用pyspark shell

```
1 /bigdata/spark-3.2.3-bin-hadoop3.2/bin/pyspark \
```

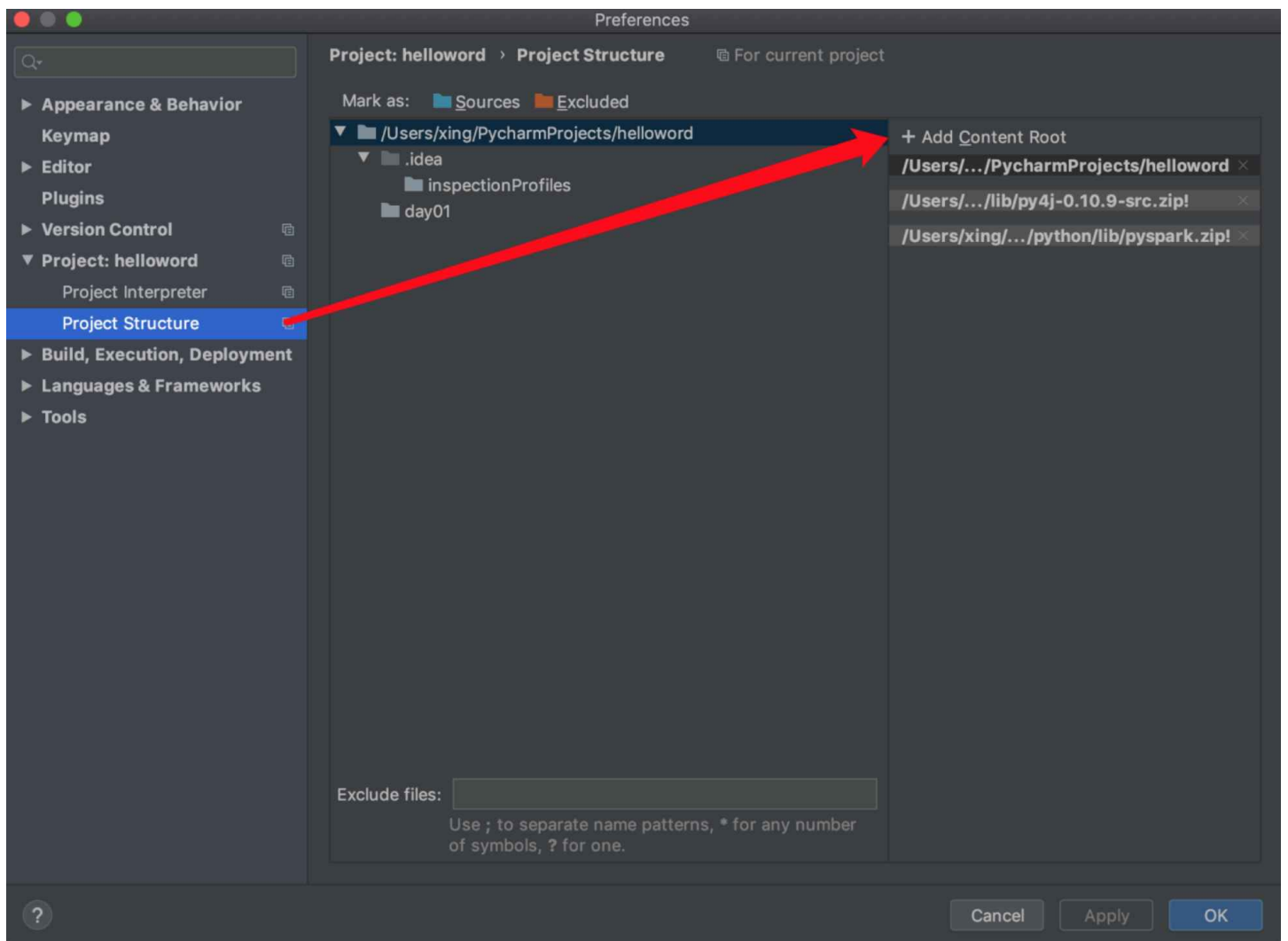
```
2 --master spark://node-1.51doit.cn:7077 \  
3 --executor-memory 1g --total-executor-cores 10
```

5.4.3 配置PyCharm开发环境

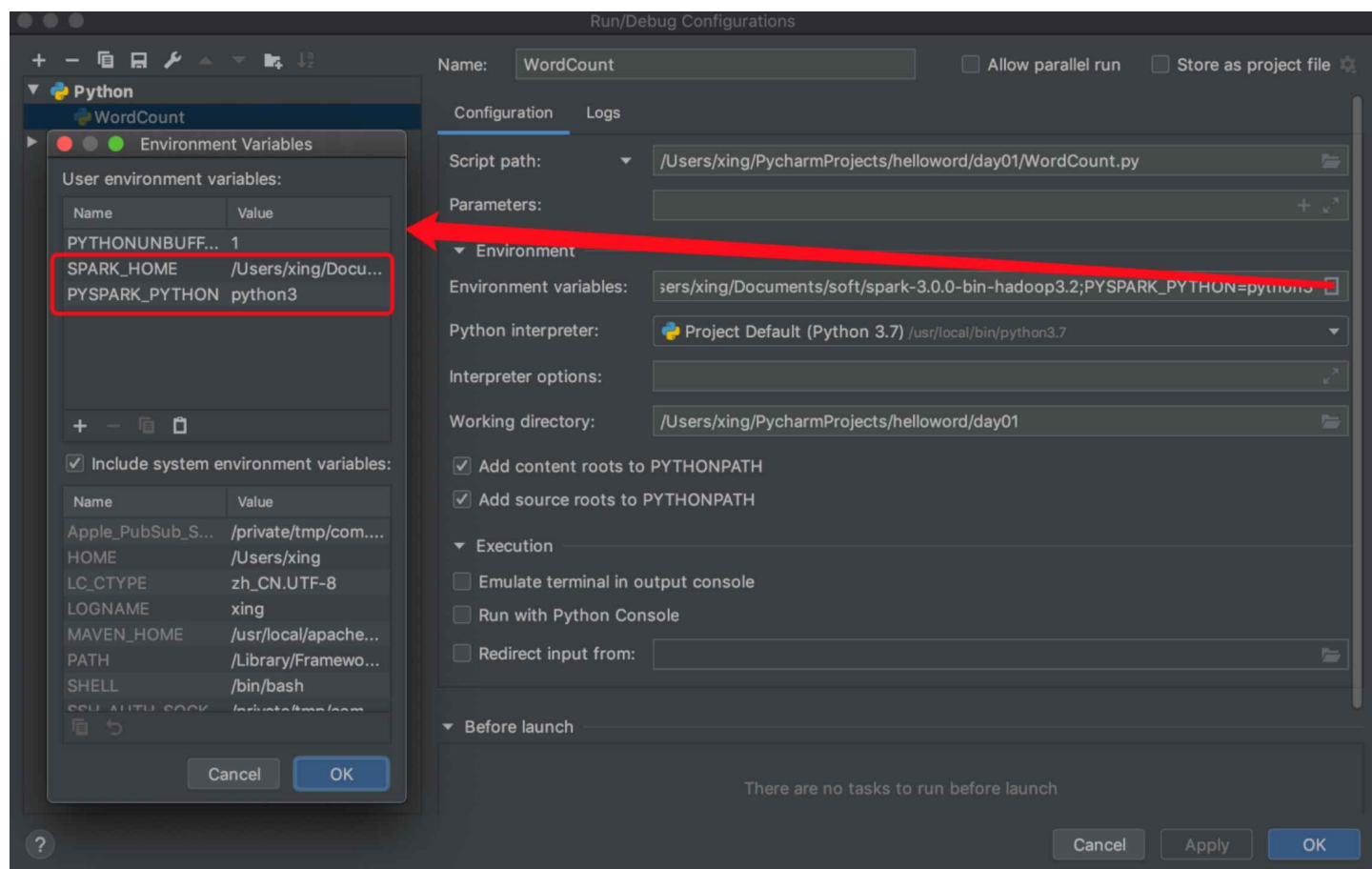
①配置python的环境



②配置pyspark的依赖



③添加环境变量



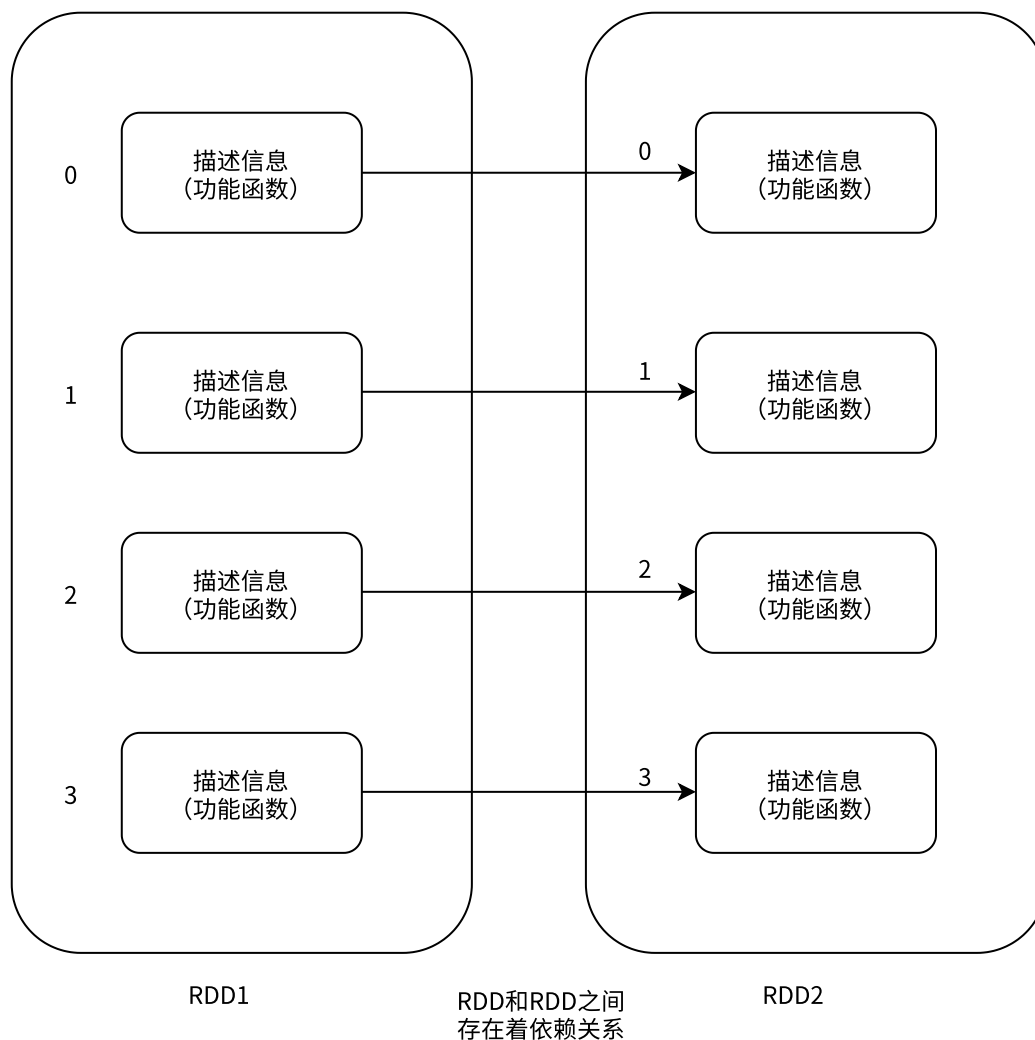
6. RDD的使用

6.1 什么是RDD

RDD的全称为Resilient Distributed Dataset，是一个弹性、可复原的分布式数据集，是Spark中最基本的抽象，是一个不可变的、有多个分区的、可以并行计算的集合。**RDD中并不装真正要计算的数据，而装的是描述信息，描述以后从哪里读取数据，调用了用什么方法，传入了什么函数，以及依赖关系等。**

6.2 RDD的特点

- **有一些列连续的分区**：分区编号从0开始，分区的数量决定了对应阶段Task的并行度
- **有一个函数作用在每个输入切片上**：每一个分区都会生成一个Task，对该分区的数据进行计算，这个函数就是具体的计算逻辑
- **RDD和RDD之间存在一系列依赖关系**：RDD调用Transformation后会生成一个新的RDD，子RDD会记录父RDD的依赖关系，包括宽依赖（有shuffle）和窄依赖（没有shuffle）
- **（可选的）K-V的RDD在Shuffle会有分区器**，默认使用HashPartitioner
- **（可选的）如果从HDFS中读取数据，会有一个最优位置**：spark在调度任务之前会读取NameNode的元数据信息，获取数据的位置，移动计算而不是移动数据，这样可以提高计算效率。



6.3 RDD的算子（方法）分类

- Transformation：即转换算子，调用转换算子会生成一个新的RDD，Transformation是Lazy的，不会触发job执行。
- Action：行动算子，调用行动算子会触发job执行，本质上是调用了sc.runJob方法，该方法从最后一个RDD，根据其依赖关系，从后往前，划分Stage，生成TaskSet。

6.4 创建RDD的方法

- 从HDFS指定的目录创建RDD

```
1 val lines: RDD[String] = sc.textFile("hdfs://node-1.51doit.cn:9000/log")
```

- 通过并行化方式，将Driver端的集合转成RDD

```
1 val rdd1: RDD[Int] = sc.parallelize(Array(1,2,3,4,5,6,7,8,9))
```

6.5 查看RDD的分区数量

```
1 val rdd1: RDD[Int] = sc.parallelize(Array(1,2,3,4,5,6,7,8,9))
2 rdd1.partitions.length
```

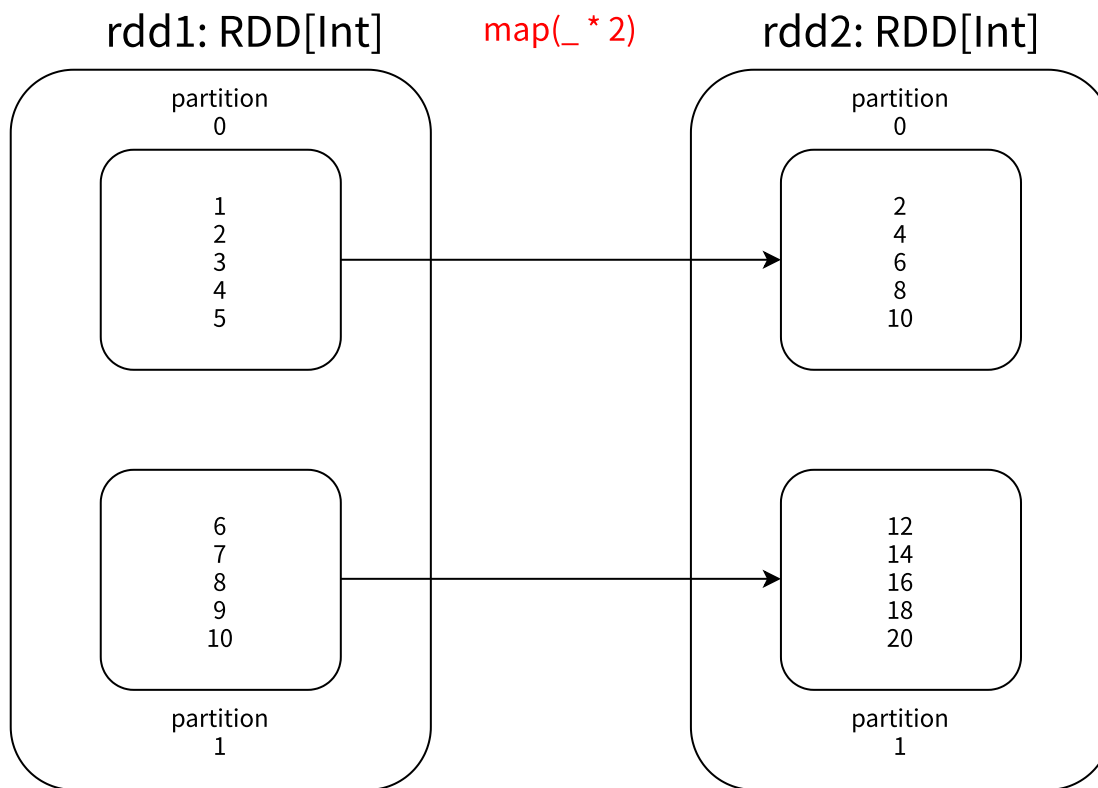
6.6 RDD的Transformation算子

6.6.1 map

map算子的功能为做映射，即将原来的RDD中对应的每一个元素，应用外部传入的函数进行运算，返回一个新的RDD

```
1 val rdd1: RDD[Int] = sc.parallelize(List(1,2,3,4,5,6,7,8,9,10), 2)
2 val rdd2: RDD[Int] = rdd1.map(_ * 2)
```

这是一个逻辑图，是对运算逻辑的表述，因为RDD中并不包含真正要计算的数据，而是保存的描述信息

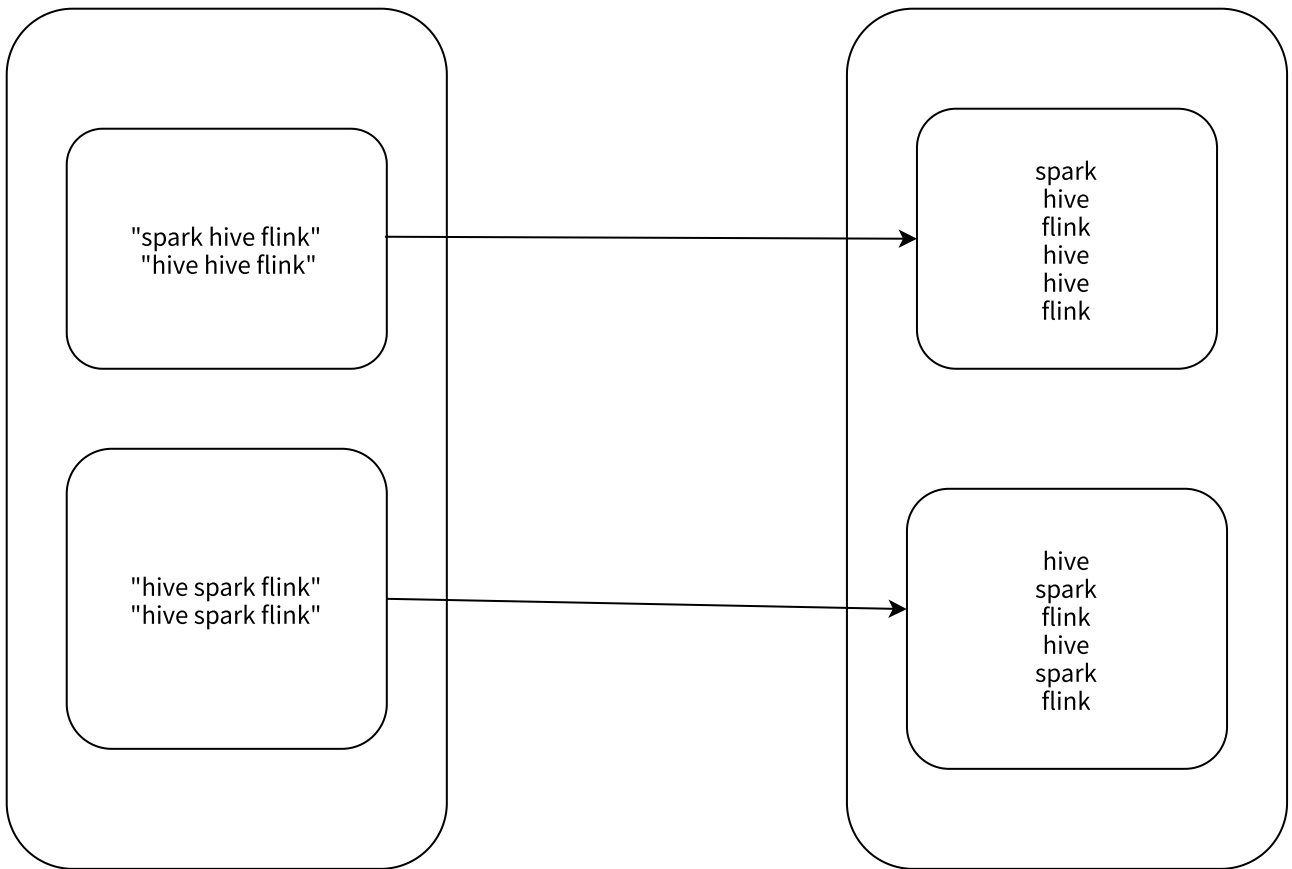


6.6.2 flatMap

flatMap算子的功能为扁平化映射，即将原来RDD中对应的每一个元素应用外部的运算逻辑进行运算，然后再将返回的数据进行压平，类似先map，然后再flatten的操作，最后返回一个新的RDD

```
1 val arr = Array(  
2   "spark hive flink",  
3   "hive hive flink",  
4   "hive spark flink",  
5   "hive spark flink"  
6 )  
7 val rdd1: RDD[String] = sc.makeRDD(arr, 2)  
8 val rdd2: RDD[String] = rdd1.flatMap(_.split(" "))
```

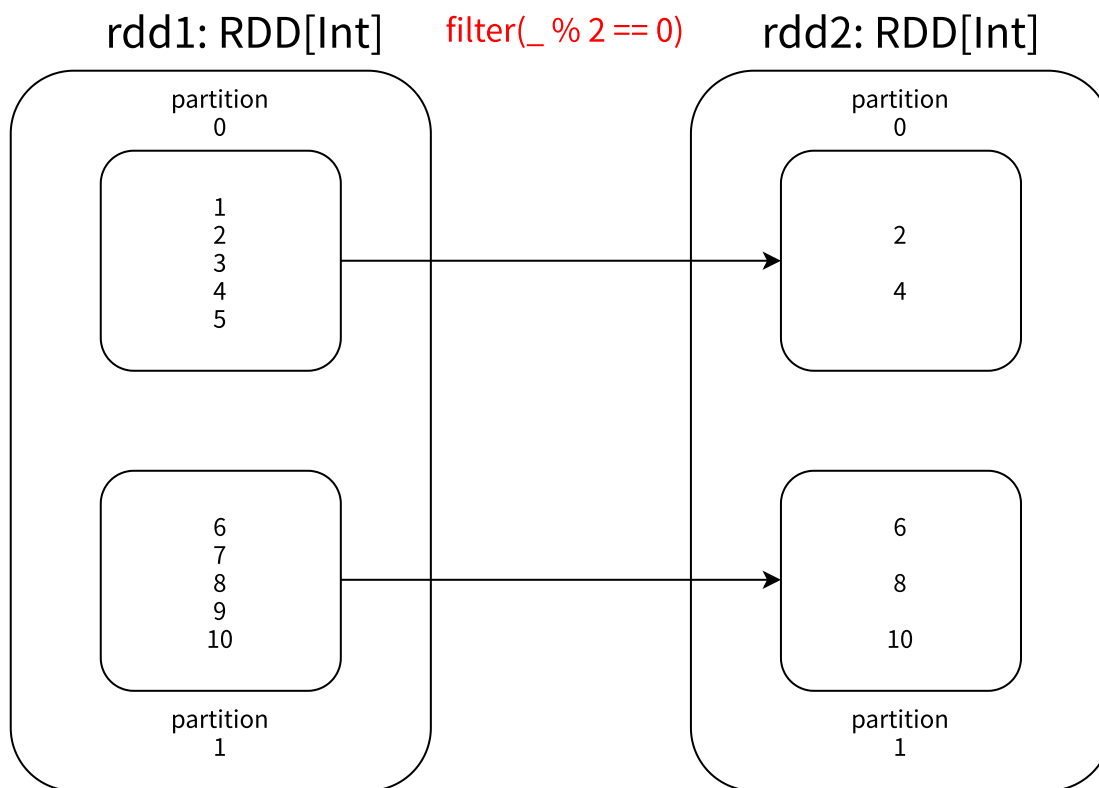
flatMap的逻辑拖



6.6.3 filter

filter的功能为过滤，即将原来RDD中对应的每一个元素，应用外部传入的过滤逻辑，然后返回一个新的RDD

```
1 val rdd1: RDD[Int] = sc.parallelize(List(1,2,3,4,5,6,7,8,9,10), 2)
2 val rdd2: RDD[Int] = rdd1.filter(_ % 2 == 0)
```



6.6.4 mapPartitions

将数据以分区为的形式返回进行map操作，一个分区对应一个迭代器，该方法和map方法类似，只不过该方法的参数由RDD中的每一个元素变成了RDD中每一个分区的迭代器，如果在映射的过程中需要频繁创建额外的对象，使用mapPartitions要比map高效的过。

```
1 val rdd1 = sc.parallelize(List(1, 2, 3, 4, 5), 2)
2 var r1: RDD[Int] = rdd1.mapPartitions(it => it.map(x => x * 10))
```

! map和mapPartitions的区别，mapPartitions一定会比map效率更高吗？

不一定：如果对RDD中的数据进行简单的映射操作，例如变大写，对数据进行简单的运算，map和mapPartitions的效果是一样的，但是如果是使用到了外部共享的对象或数据库连接，mapPartitions效率会更高一些。

原因：map出入的函数是一条一条的进行处理，如果使用数据库连接，会每来一条数据创建一个连接，导致性能过低，而mapPartitions传入的函数参数是迭代器，是以分区为单位进行

操作，可以事先创建好一个连接，反复使用，操作一个分区中的多条数据。

特别提醒：如果使用mapPartitions方法不当，即将迭代器中的数据toList，就是将数据都放到内存中，可能会出现内存溢出的情况。

6.6.5 mapPartitionsWithIndex

类似于mapPartitions, 不过函数要输入两个参数，第一个参数为分区的索引，第二个是对应分区的迭代器。函数的返回的是一个经过该函数转换的迭代器。

```
1 val rdd1 = sc.parallelize(List(1,2,3,4,5,6,7,8,9), 2)
2 val rdd2 = rdd1.mapPartitionsWithIndex((index, it) => {
3   it.map(e => s"partition: $index, val: $e")
4 })
```

6.6.6 keys

RDD中的数据为对偶元组类型，调用keys方法后返回一个新的RDD，该RDD的对应的数据为原来对偶元组的全部key

```
1 val lst = List(
2   ("spark", 1), ("hadoop", 1), ("hive", 1), ("spark", 1),
3   ("spark", 1), ("flink", 1), ("hbase", 1), ("spark", 1),
4   ("kafka", 1), ("kafka", 1), ("kafka", 1), ("kafka", 1),
5   ("hadoop", 1), ("flink", 1), ("hive", 1), ("flink", 1)
6 )
7 //通过并行化的方式创建RDD，分区数量为4
8 val wordAndOne: RDD[(String, Int)] = sc.parallelize(lst, 4)
9 val keyRDD: RDD[String] = wordAndOne.keys
```

6.6.7 values

RDD中的数据为对偶元组类型，调用values方法后返回一个新的RDD，该RDD的对应的数据为原来对偶元组的全部values

```
1 val lst = List(
2   ("spark", 1), ("hadoop", 1), ("hive", 1), ("spark", 1),
3   ("spark", 1), ("flink", 1), ("hbase", 1), ("spark", 1),
```

```

4  ("kafka", 1), ("kafka", 1), ("kafka", 1), ("kafka", 1),
5  ("hadoop", 1), ("flink", 1), ("hive", 1), ("flink", 1)
6  )
7  //通过并行化的方式创建RDD, 分区数量为4
8  val wordAndOne: RDD[(String, Int)] = sc.parallelize(lst, 4)
9  val valueRDD: RDD[Int] = wordAndOne.values
10 //

```

6.6.8 mapValues

RDD中的数据为对偶元组类型，将value应用传入的函数进行运算后再与key组合成元组返回一个新的RDD

```

1  val lst = List(("spark", 5), ("hive", 3), ("hbase", 4), ("flink", 8))
2  val rdd1: RDD[(String, Int)] = sc.parallelize(lst, 2)
3  //将每一个元素的次数乘以10再跟key组合在一起
4  //val rdd2 = rdd1.map(t => (t._1, t._2 * 10))
5  val rdd2 = rdd1.mapValues(_ * 10)

```

6.6.9 flatMapValues

RDD中的数据为对偶元组类型，将value应用传入的函数进行flatMap打平后再与key组合成元组返回一个新的RDD

```

1  val lst = List(("spark", "1,2,3"), ("hive", "4,5"), ("hbase", "6"), ("flink", "7")
2  val rdd1: RDD[(String, String)] = sc.parallelize(lst, 2)
3  //将value打平, 再将打平后的每一个元素与key组合("spark", "1,2,3") => ("spark", 1), ("s
4  val rdd2: RDD[(String, Int)] = rdd1.flatMapValues(_.split(",").map(_.toInt))
5  //      val rdd2 = rdd1.flatMap(t => {
6  //          t._2.split(",").map(e => (t._1, e.toInt))
7  //      })

```

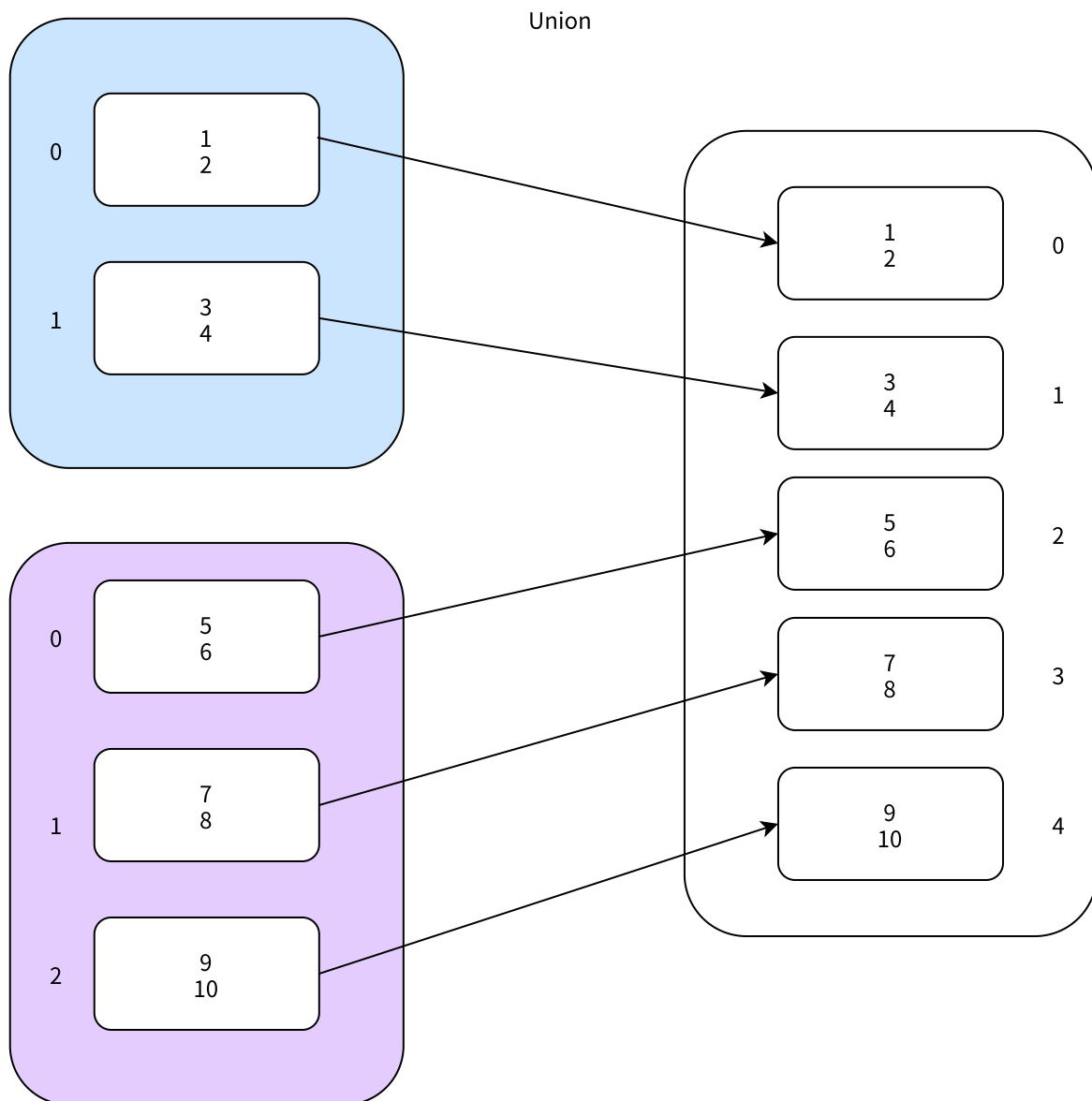
6.6.10 union

将两个类型一样的RDD合并到一起，返回一个新的RDD，新的RDD的分区数量是原来两个RDD的分区数量之和

```

1 //两个RDD进行union, 对应的数据类型必须一样
2 //Union不会去重
3 val rdd1 = sc.parallelize(List(1,2,3,4), 2)
4 val rdd2 = sc.parallelize(List(5, 6, 7, 8, 9,10), 3)
5 val rdd3 = rdd1.union(rdd2)
6 println(rdd3.partitions.length)

```



6.6.11 reduceByKey

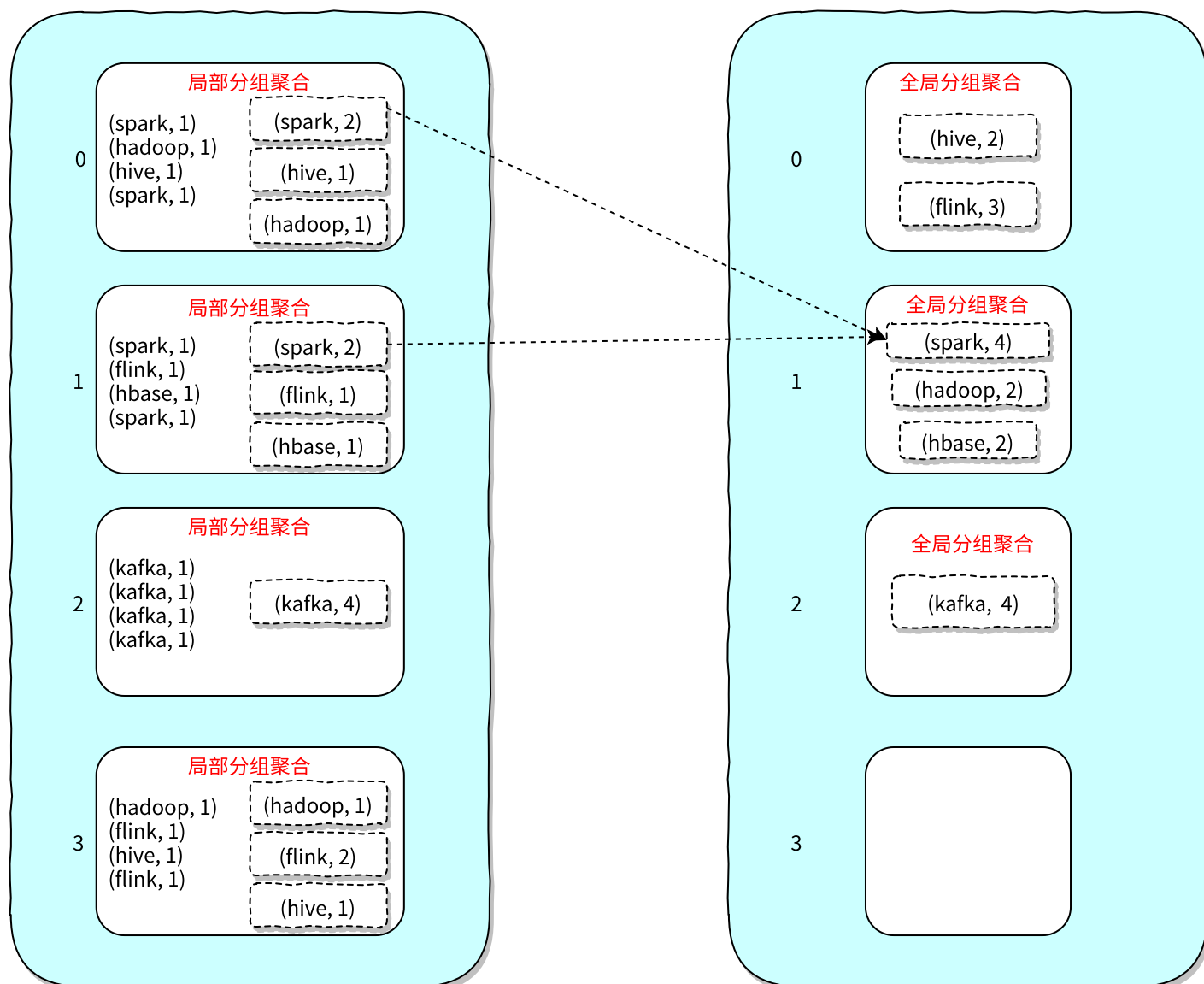
将数据按照相同的key进行聚合，特点是先在每个分区中进行局部分组聚合，然后将每个分区聚合的结果从上游拉取到下游再进行全局分组聚合

```

1 val lst = List(
2   ("spark", 1), ("hadoop", 1), ("hive", 1), ("spark", 1),
3   ("spark", 1), ("flink", 1), ("hbase", 1), ("spark", 1),
4   ("kafka", 1), ("kafka", 1), ("kafka", 1), ("kafka", 1),
5   ("hadoop", 1), ("flink", 1), ("hive", 1), ("flink", 1)
6 )
7 //通过并行化的方式创建RDD，分区数量为4
8 val wordAndOne: RDD[(String, Int)] = sc.parallelize(lst, 4)
9 val reduced: RDD[(String, Int)] = wordAndOne.reduceByKey(_ + _)

```

RduceByKey对应的逻辑图



6.6.12 combineByKey

```
1 val lst = List(  
2   ("spark", 1), ("hadoop", 1), ("hive", 1), ("spark", 1),  
3   ("spark", 1), ("flink", 1), ("hbase", 1), ("spark", 1),  
4   ("kafka", 1), ("kafka", 1), ("kafka", 1), ("kafka", 1),  
5   ("hadoop", 1), ("flink", 1), ("hive", 1), ("flink", 1)  
6 )  
7 //通过并行化的方式创建RDD, 分区数量为4  
8 val wordAndOne: RDD[(String, Int)] = sc.parallelize(lst, 4)  
9 //调用combineByKey传入三个函数  
10 //val reduced = wordAndOne.combineByKey(x => x, (a: Int, b: Int) => a + b, (m: I  
11 val f1 = (x: Int) => {  
12   val stage = TaskContext.get().stageId()  
13   val partition = TaskContext.getPartitionId()  
14   println(s"f1 function invoked in state: $stage, partition: $partition")  
15   x  
16 }  
17 //在每个分区内, 将key相同的value进行局部聚合操作  
18 val f2 = (a: Int, b: Int) => {  
19   val stage = TaskContext.get().stageId()  
20   val partition = TaskContext.getPartitionId()  
21   println(s"f2 function invoked in state: $stage, partition: $partition")  
22   a + b  
23 }  
24 //第三个函数是在下游完成的  
25 val f3 = (m: Int, n: Int) => {  
26   val stage = TaskContext.get().stageId()  
27   val partition = TaskContext.getPartitionId()  
28   println(s"f3 function invoked in state: $stage, partition: $partition")  
29   m + n  
30 }  
31 val reduced = wordAndOne.combineByKey(f1, f2, f3)
```

combineByKey要传入三个函数:

第一个函数: 在上游执行, 该key在当前分区第一次出现时, 对value处理的运算逻辑

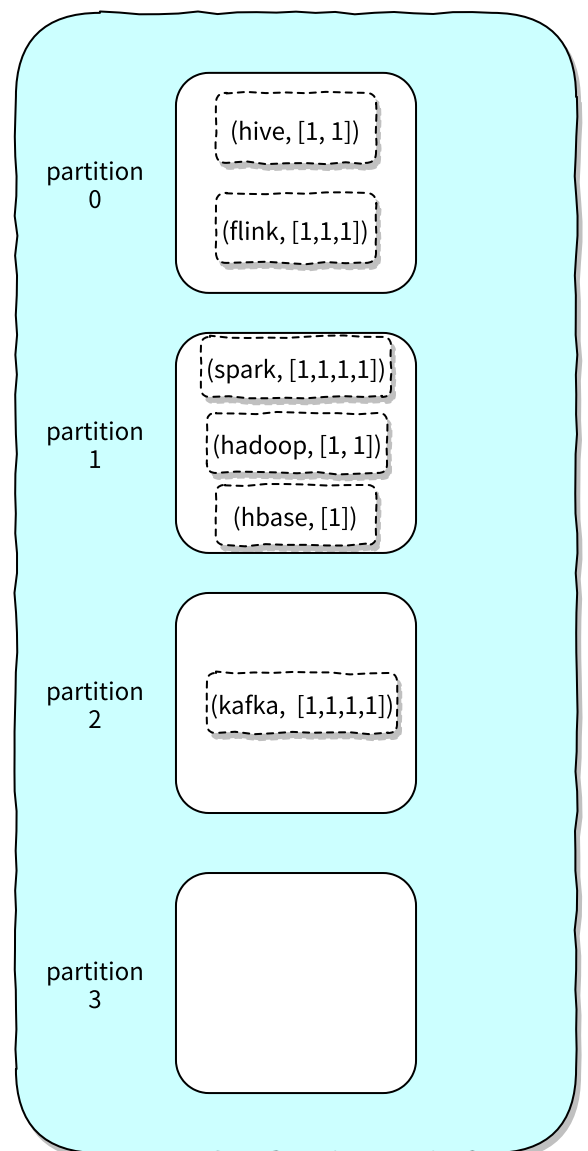
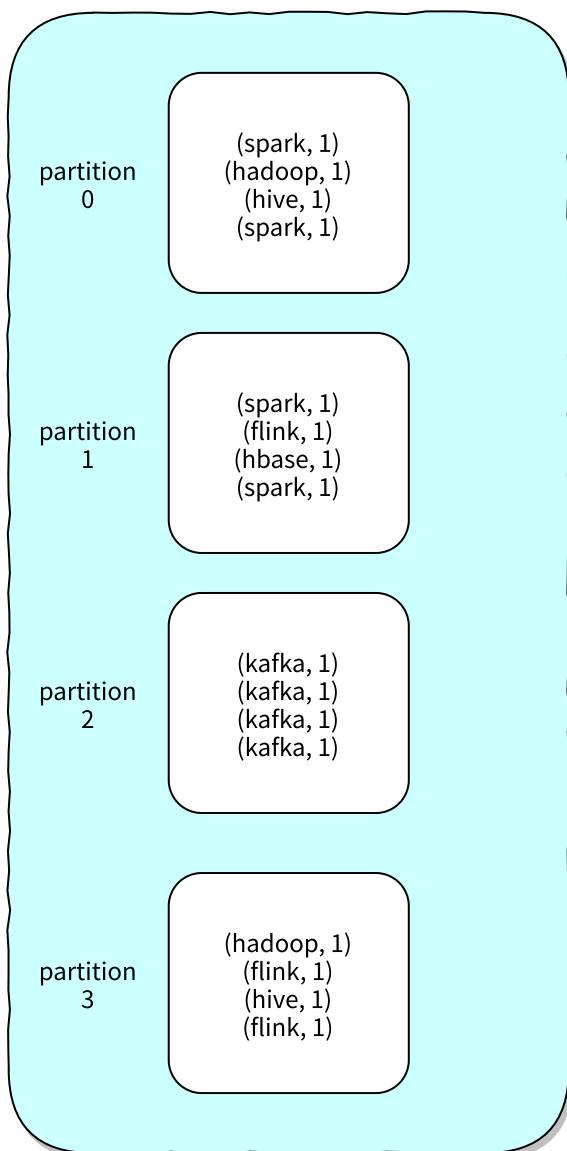
第二个函数: 在上游执行, 当该key在当前分区再次出现时, 将以前相同key的value进行运算的逻辑

第三个函数: 在下游执行, 将来自不同分区, 相同key的数据通过网络拉取过来, 然后进行全局聚合的逻辑

6.6.13 groupByKey

按照key进行分组，底层使用的是ShuffledRDD，mapSideCombine = false，传入的三个函数只有前两个被调用了，并且是在下游执行的

```
1  val lst = List(  
2    ("spark", 1), ("hadoop", 1), ("hive", 1), ("spark", 1),  
3    ("spark", 1), ("flink", 1), ("hbase", 1), ("spark", 1),  
4    ("kafka", 1), ("kafka", 1), ("kafka", 1), ("kafka", 1),  
5    ("hadoop", 1), ("flink", 1), ("hive", 1), ("flink", 1)  
6  )  
7  //通过并行化的方式创建RDD，分区数量为4  
8  val wordAndOne: RDD[(String, Int)] = sc.parallelize(lst, 4)  
9  //按照key进行分组  
10 val grouped: RDD[(String, Iterable[Int])] = wordAndOne.groupByKey()
```



6.6.14 foldByKey

与reduceByKey类似，只不过是可指定初始值，每个分区应用一次初始值，先在每个进行局部聚合，然后再全局聚合，局部聚合的逻辑与全局聚合的逻辑相同。

```
1  val lst: Seq[(String, Int)] = List(  
2    ("spark", 1), ("hadoop", 1), ("hive", 1), ("spark", 1),  
3    ("spark", 1), ("flink", 1), ("hbase", 1), ("spark", 1),  
4    ("kafka", 1), ("kafka", 1), ("kafka", 1), ("kafka", 1),  
5    ("hadoop", 1), ("flink", 1), ("hive", 1), ("flink", 1)  
6  )  
7  //通过并行化的方式创建RDD，分区数量为4  
8  val wordAndOne: RDD[(String, Int)] = sc.parallelize(lst, 4)  
9  
10 //与reduceByKey类似，只不过是可指定初始值，每个分区应用一次初始值  
11 val reduced: RDD[(String, Int)] = wordAndOne.foldByKey(0)(_ + _)
```

6.6.15 aggregateByKey

与reduceByKey类似，并且可以指定初始值，每个分区应用一次初始值，传入两个函数，分别是局部聚合的计算逻辑、全局聚合的逻辑。

```
1  
2  val lst: Seq[(String, Int)] = List(  
3    ("spark", 1), ("hadoop", 1), ("hive", 1), ("spark", 1),  
4    ("spark", 1), ("flink", 1), ("hbase", 1), ("spark", 1),  
5    ("kafka", 1), ("kafka", 1), ("kafka", 1), ("kafka", 1),  
6    ("hadoop", 1), ("flink", 1), ("hive", 1), ("flink", 1)  
7  )  
8  //通过并行化的方式创建RDD，分区数量为4  
9  val wordAndOne: RDD[(String, Int)] = sc.parallelize(lst, 4)  
10 //在第一个括号中传入初始化，第二个括号中传入两个函数，分别是局部聚合的逻辑和全局聚合的逻辑  
11 val reduced: RDD[(String, Int)] = wordAndOne.aggregateByKey(0)(_ + _, _ + _)
```

6.6.16 ShuffledRDD

reduceByKey、combineByKey、aggregateByKey、foldByKey底层都是使用的ShuffledRDD，并且mapSideCombine = true

```
1 val f1 = (x: Int) => {
2   val stage = TaskContext.get().stageId()
3   val partition = TaskContext.getPartitionId()
4   println(s"f1 function invoked in state: $stage, partition: $partition")
5   x
6 }
7 //在每个分区内，将key相同的value进行局部聚合操作
8 val f2 = (a: Int, b: Int) => {
9   val stage = TaskContext.get().stageId()
10  val partition = TaskContext.getPartitionId()
11  println(s"f2 function invoked in state: $stage, partition: $partition")
12  a + b
13 }
14 //第三个函数是在下游完成的
15 val f3 = (m: Int, n: Int) => {
16   val stage = TaskContext.get().stageId()
17   val partition = TaskContext.getPartitionId()
18   println(s"f3 function invoked in state: $stage, partition: $partition")
19   m + n
20 }
21 //指定分区器为HashPartitioner
22 val partitioner = new HashPartitioner(wordAndOne.partitions.length)
23 val shuffledRDD = new ShuffledRDD[String, Int, Int](wordAndOne, partitioner)
24 //设置聚合亲器并关联三个函数
25 val aggregator = new Aggregator[String, Int, Int](f1, f2, f3)
26 shuffledRDD.setAggregator(aggregator) //设置聚合器
27 shuffledRDD.setMapSideCombine(true) //设置map端聚合
```

6.6.17 distinct

distinct是对RDD中的元素进行取重，底层使用的是reduceByKey实现的，先局部去重，然后再全局去重

```
1 val arr = Array(
2   "spark", "hive", "spark", "flink",
3   "spark", "hive", "hive", "flink",
4   "flink", "flink", "flink", "spark"
5 )
6 val rdd1: RDD[String] = sc.parallelize(arr, 3)
```

```
7 //去重
8 val rdd2: RDD[String] = rdd1.distinct()
```

distinct的底层实现如下：

```
1 val rdd11: RDD[(String, Null)] = rdd1.map((_, null))
2 val rdd12: RDD[String] = rdd11.reduceByKey((a, _) => a).keys
```

6.6.18 partitionBy

按照指定的分区器进行分区，底层使用的是ShuffledRDD

```
1 val lst: Seq[(String, Int)] = List(
2   ("spark", 1), ("hadoop", 1), ("hive", 1), ("spark", 1),
3   ("spark", 1), ("flink", 1), ("hbase", 1), ("spark", 1),
4   ("kafka", 1), ("kafka", 1), ("kafka", 1), ("kafka", 1),
5   ("hadoop", 1), ("flink", 1), ("hive", 1), ("flink", 1)
6 )
7 //通过并行化的方式创建RDD，分区数量为4
8 val wordAndOne: RDD[(String, Int)] = sc.parallelize(lst, 4)
9 val partitioner = new HashPartitioner(wordAndOne.partitions.length)
10 //按照指定的分区进行分区
11 val partitioned: RDD[(String, Int)] = wordAndOne.partitionBy(partitioner)
```

6.6.19 repartitionAndSortWithinPartitions

按照值的分区器进行分区，并且将数据按照指定的排序规则在分区内排序，底层使用的是ShuffledRDD，设置了指定的分区器和排序规则

```
1 val lst: Seq[(String, Int)] = List(
2   ("spark", 1), ("hadoop", 1), ("hive", 1), ("spark", 1),
3   ("spark", 1), ("flink", 1), ("hbase", 1), ("spark", 1),
4   ("kafka", 1), ("kafka", 1), ("kafka", 1), ("kafka", 1),
5   ("hadoop", 1), ("flink", 1), ("hive", 1), ("flink", 1)
6 )
7 //通过并行化的方式创建RDD，分区数量为4
8 val wordAndOne: RDD[(String, Int)] = sc.parallelize(lst, 4)
```

```
9 val partitioner = new HashPartitioner(wordAndOne.partitions.length)
10 //按照指定的分区进行分区，并且将数据按照指定的排序规则在分区内排序
11 val partitioned = wordAndOne.repartitionAndSortWithinPartitions(partitioner)
```

repartitionAndSortWithinPartitions的底层实现：

```
1 new ShuffledRDD[K, V, V](self, partitioner).setKeyOrdering(ordering)
```

6.6.20 sortBy

```
1 val lines: RDD[String] = sc.textFile("hdfs://node-1.51doit.cn:9000/words")
2 //切分压平
3 val words: RDD[String] = lines.flatMap(_.split(" "))
4 //将单词和1组合
5 val wordAndOne: RDD[(String, Int)] = words.map((_, 1))
6 //分组聚合
7 val reduced: RDD[(String, Int)] = wordAndOne.reduceByKey(_ + _)
8 //按照单词出现的次数，从高到低进行排序
9 val sorted: RDD[(String, Int)] = reduced.sortBy(_._2, false)
```

6.6.21 sortByKey

按照指定的key排序规则进行全局排序

```
1 val lines: RDD[String] = sc.textFile("hdfs://node-1.51doit.cn:9000/words")
2 //切分压平
3 val words: RDD[String] = lines.flatMap(_.split(" "))
4 //将单词和1组合
5 val wordAndOne: RDD[(String, Int)] = words.map((_, 1))
6 //分组聚合
7 val reduced: RDD[(String, Int)] = wordAndOne.reduceByKey(_ + _)
8 //按照单词出现的次数，从高到低进行排序
9 //val sorted: RDD[(String, Int)] = reduced.sortBy(_._2, false)
10 //val keyed: RDD[(Int, (String, Int))] = reduced.keyBy(_._2).sortByKey()
11 val sorted = reduced.map(t => (t._2, t)).sortByKey(false)
```

! sortBy、sortByKey是Transformation，但是为什么会生成job？

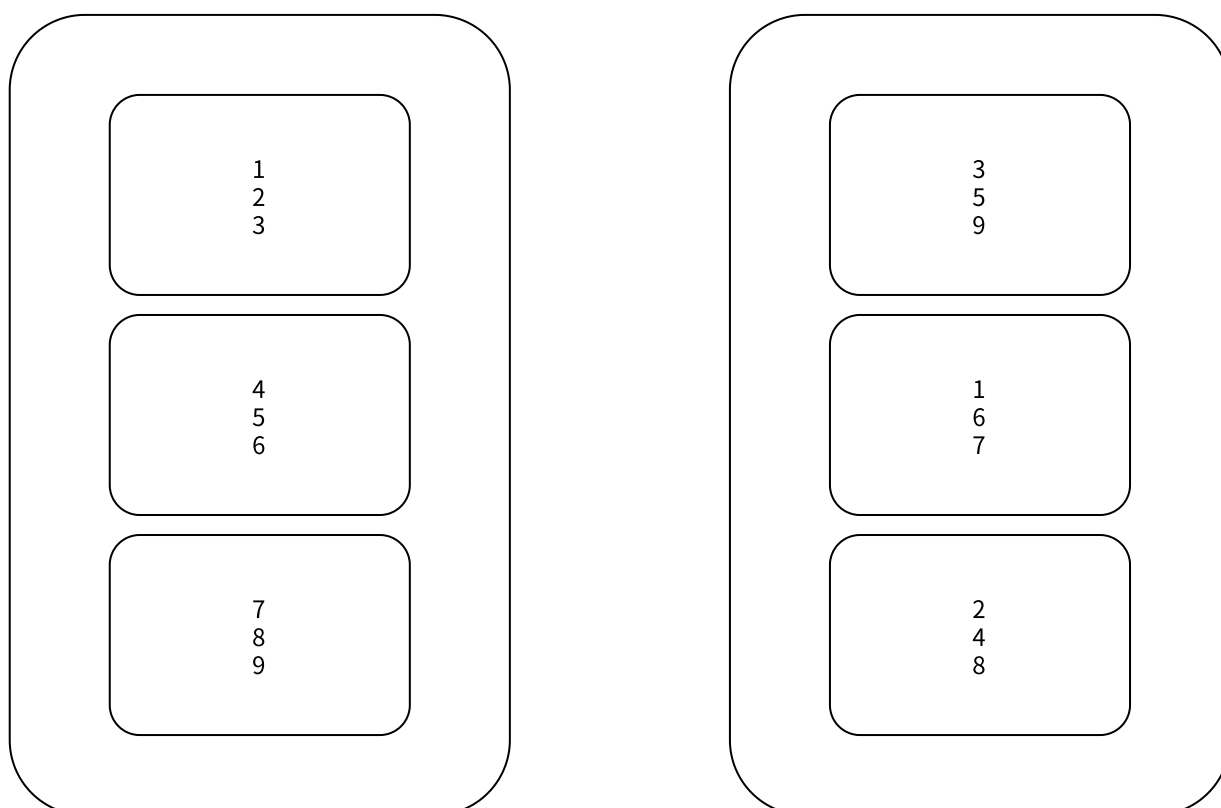
因为sortBy、sortByKey需要实现全局排序，使用的是RangePartitioner，在构建RangePartitioner时，会对数据进行采样，所有会触发Action，根据采样的结果来构建RangePartitioner。

RangePartitioner可以保证数据按照一定的范围全局有序，同时在shuffle的同时，有设置了setKeyOrdering，这样就又可以保证数据在每个分区内有序了！

6.6.22 repartition

repartition的功能是重新分区，一定会shuffle，即将数据打散。repartition的功能是改变分区数量（可以增大、减少、不变）可以将数据相对均匀的重新分区，可以改善数据倾斜的问题

```
1 val rdd1 = sc.parallelize(List(1,2,3,4,5,6,7,8,9,10), 3)
2 //repartition方法一定shuffle
3 //不论将分区数量变多、变少、或不变，都shuffle
4 val rdd2 = rdd1.repartition(3)
```



repartition的底层调用的是coalesce，shuffle = true

```
1 coalesce(numPartitions, shuffle = true)
```

6.6.23 coalesce

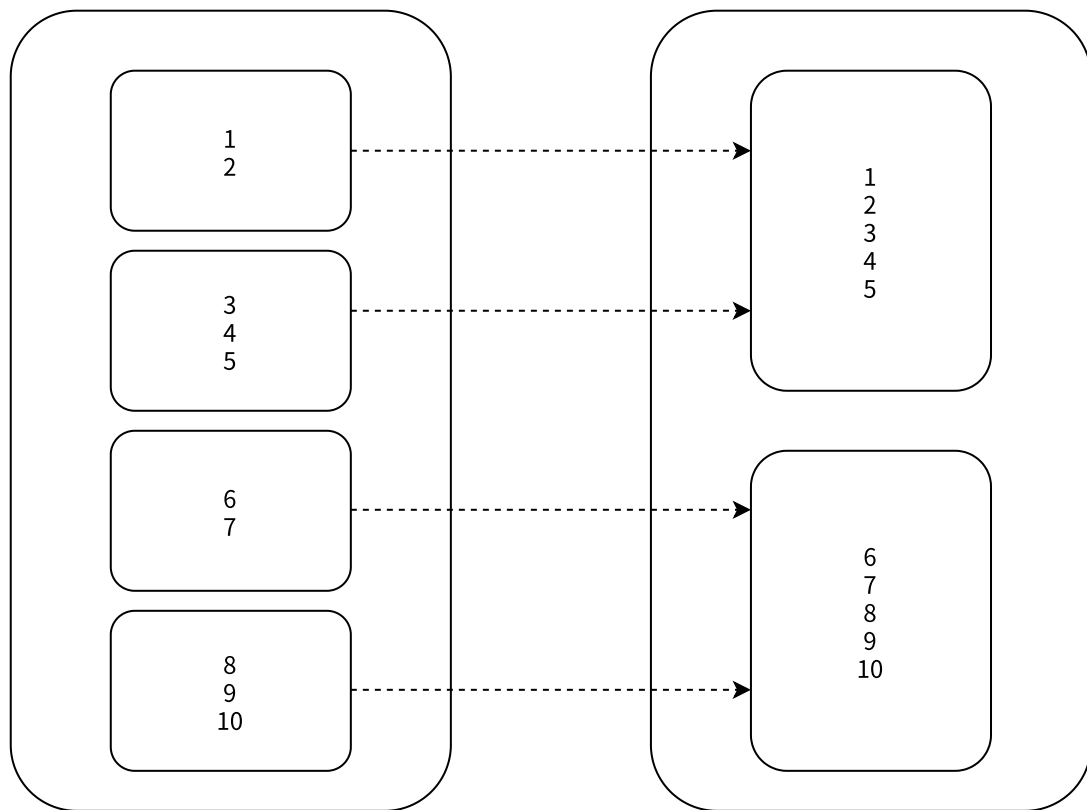
coalesce可以shuffle，也可以不shuffle，如果将分区数量减少，并且shuffle = false，就是将分区进行合并

- shuffle = true

```
1 val rdd1 = sc.parallelize(List(1,2,3,4,5,6,7,8,9,10), 3)
2 //shuffle = true
3 val rdd2 = rdd1.coalesce(3, true)
4 //与repartition(3)功能一样
```

- shuffle = false

```
1 val rdd1 = sc.parallelize(List(1,2,3,4,5,6,7,8,9,10), 4)
2 //shuffle = false
3 val rdd2 = rdd1.coalesce(2, false)
```

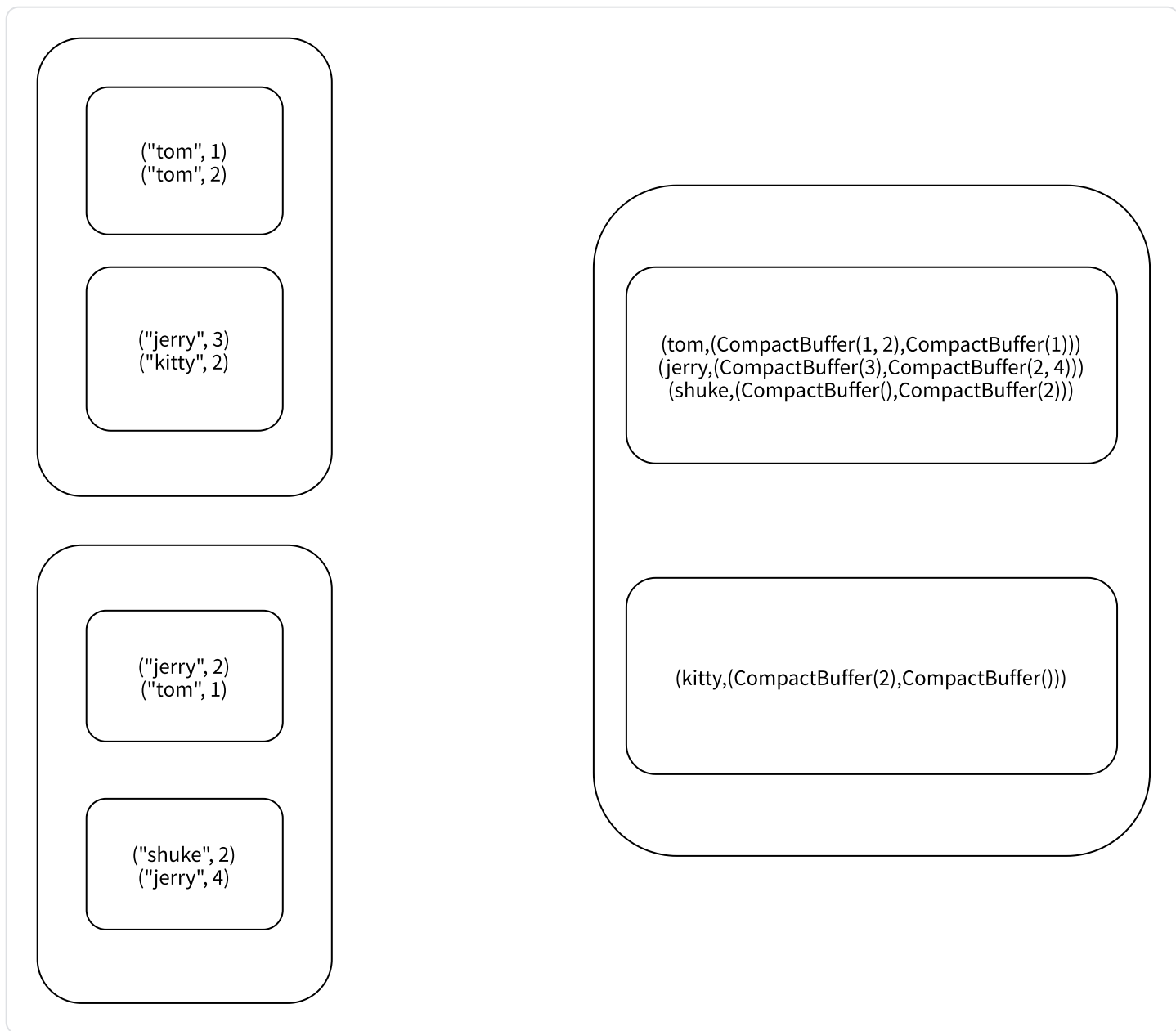


6.6.24 cogroup

协同分组，即将多个RDD中对应的数据，使用相同的分区器（HashPartitioner），将来自多个RDD中的key相同的数据通过网络传入到同一台机器的同一个分区中(*groupByKey*、*groupByKey*只能对一个RDD进行分组,)

注意:调用cogroup方法，两个RDD中对应的数据都必须是对偶元组类型，并且key类型一定相同

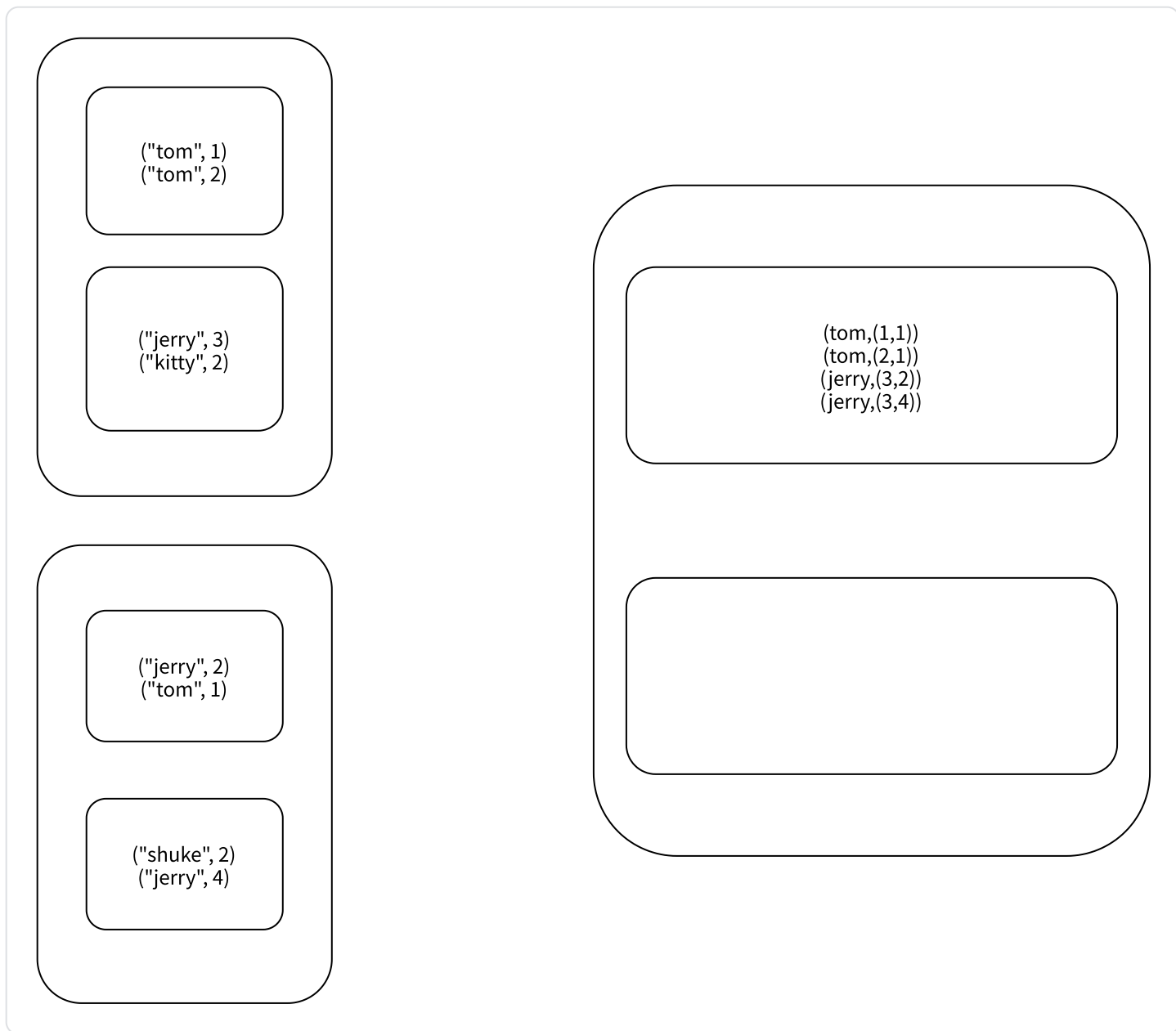
```
1 //通过并行化的方式创建一个RDD
2 val rdd1 = sc.parallelize(List(("tom", 1), ("tom", 2), ("jerry", 3), ("kitty", 2)
3 //通过并行化的方式再创建一个RDD
4 val rdd2 = sc.parallelize(List(("jerry", 2), ("tom", 1), ("shuke", 2), ("jerry",
5 //将两个RDD都进行分组
6 val grouped: RDD[(String, (Iterable[Int], Iterable[Int]))] = rdd1.cogroup(rdd2)
```



6.6.25 join

两个RDD进行join，相当于SQL中的内关联join

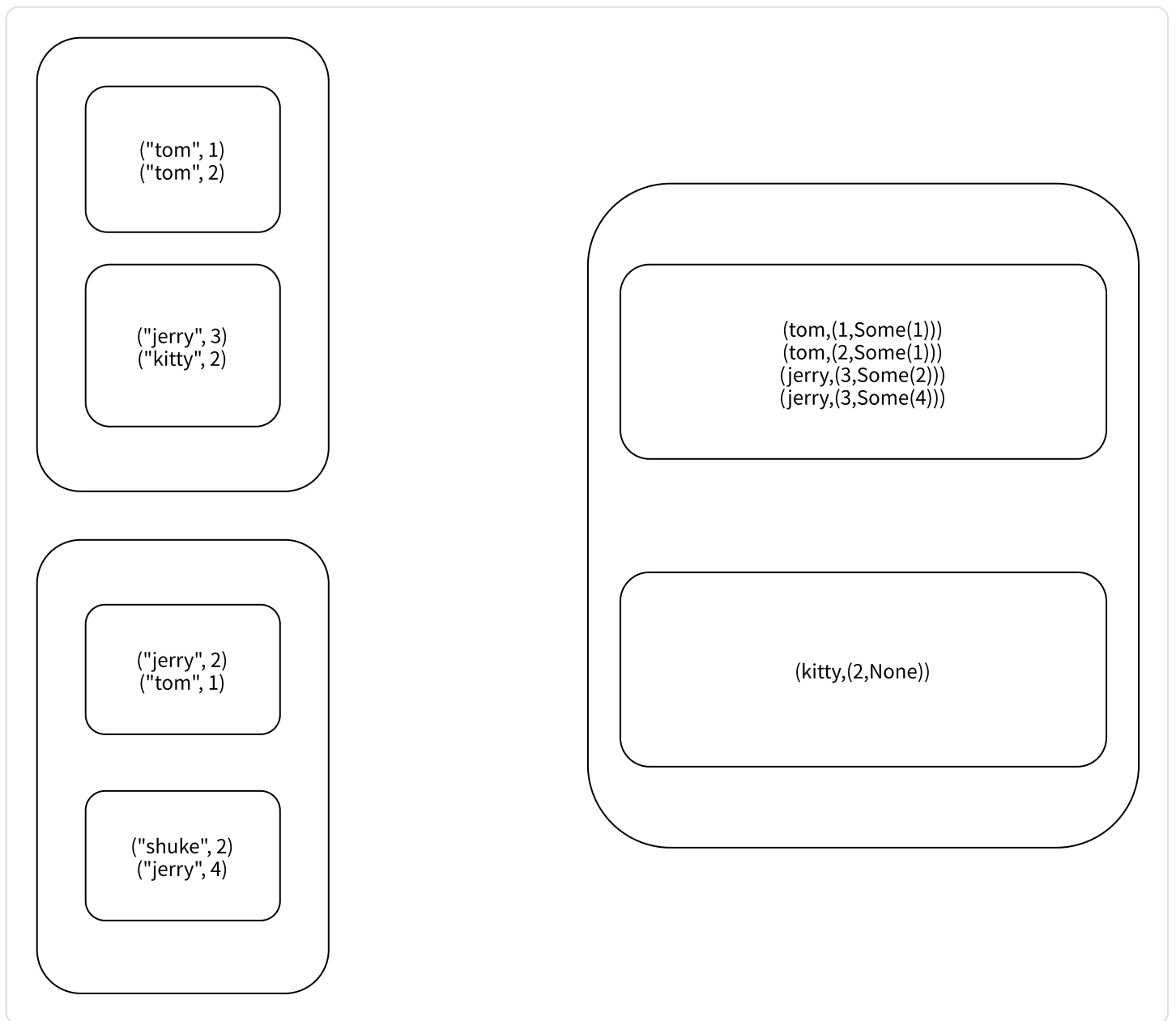
```
1 //通过并行化的方式创建一个RDD
2 val rdd1 = sc.parallelize(List(("tom", 1), ("tom", 2), ("jerry", 3), ("kitty", 2)
3 //通过并行化的方式再创建一个RDD
4 val rdd2 = sc.parallelize(List(("jerry", 2), ("tom", 1), ("shuke", 2), ("jerry",
5 val rdd3: RDD[(String, (Int, Int))] = rdd1.join(rdd2)
```



6.6.26 leftOuterJoin

左外连接，相当于SQL中的左外关联

```
1 //通过并行化的方式创建一个RDD
2 val rdd1 = sc.parallelize(List(("tom", 1), ("tom", 2), ("jerry", 3), ("kitty", 2)
3 //通过并行化的方式再创建一个RDD
4 val rdd2 = sc.parallelize(List(("jerry", 2), ("tom", 1), ("shuke", 2), ("jerry",
5 val rdd3: RDD[(String, (Int, Option[Int]))] = rdd1.leftOuterJoin(rdd2)
```



6.6.27 rightOuterJoin

右外连接，相当于SQL中的右外关联

```
1 //通过并行化的方式创建一个RDD
2 val rdd1 = sc.parallelize(List(("tom", 1), ("tom", 2), ("jerry", 3), ("kitty", 2)
3 //通过并行化的方式再创建一个RDD
4 val rdd2 = sc.parallelize(List(("jerry", 2), ("tom", 1), ("shuke", 2), ("jerry",
5 val rdd3: RDD[(String, (Option[Int], Int))] = rdd1.rightOuterJoin(rdd2)
```

("tom", 1)
("tom", 2)

("jerry", 3)
("kitty", 2)

("jerry", 2)
("tom", 1)

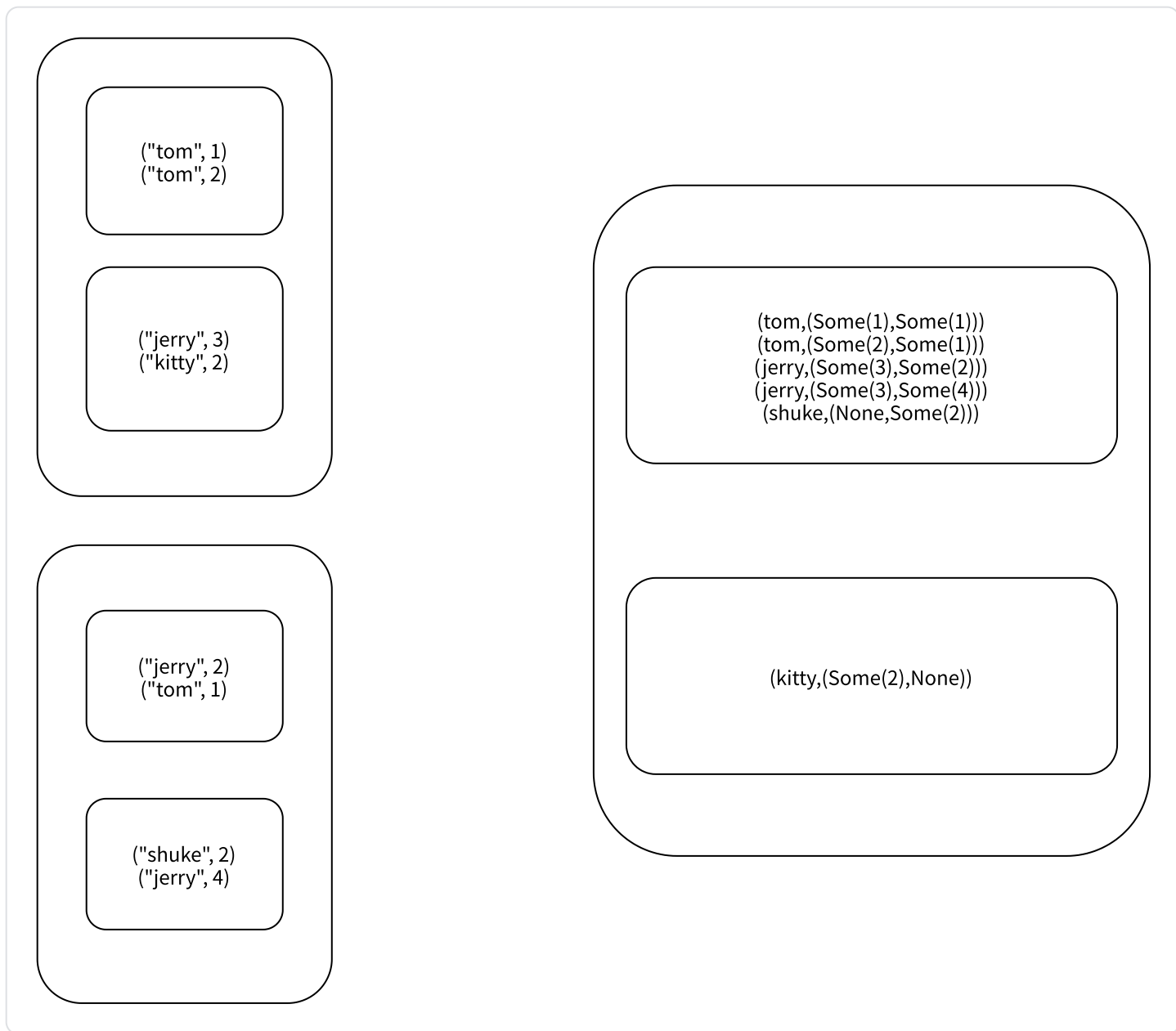
("shuke", 2)
("jerry", 4)

(tom,(Some(1),1))
(tom,(Some(2),1))
(jerry,(Some(3),2))
(jerry,(Some(3),4))
(shuke,(None,2))

6.6.28 fullOuterJoin

全连接，相当于SQL中的全关联

```
1 //通过并行化的方式创建一个RDD
2 val rdd1 = sc.parallelize(List(("tom", 1), ("tom", 2), ("jerry", 3), ("kitty", 2)
3 //通过并行化的方式再创建一个RDD
4 val rdd2 = sc.parallelize(List(("jerry", 2), ("tom", 1), ("shuke", 2), ("jerry",
5 val rdd3: RDD[(String, (Option[Int], Option[Int]))] = rdd1.fullOuterJoin(rdd2)
```



6.6.29 intersection

求交集，底层使用的是`cogroup`实现的

```
1 val rdd1 = sc.parallelize(List(1,2,3,4,4,6), 2)
2 val rdd2 = sc.parallelize(List(3,4,5,6,7,8), 2)
3 //求交集
4 val rdd3: RDD[Int] = rdd1.intersection(rdd2)
5
6 //使用cogroup实现intersection的功能
7 val rdd11 = rdd1.map((_, null))
8 val rdd22 = rdd2.map((_, null))
9 val rdd33: RDD[(Int, (Iterable[Null], Iterable[Null]))] = rdd11.cogroup(rdd22)
10 val rdd44: RDD[Int] = rdd33.filter { case (_, (it1, it2)) => it1.nonEmpty && it2
```

6.6.30 subtract

求两个RDD的差集，将第一个RDD中的数据，如果在第二个RDD中出现了，就从第一个RDD中移除

```
1 val rdd1 = sc.parallelize(List("A", "B", "C", "D", "E"))
2 val rdd2 = sc.parallelize(List("A", "B"))
3
4 val rdd3: RDD[String] = rdd1.subtract(rdd2)
5 //返回 C D E
```

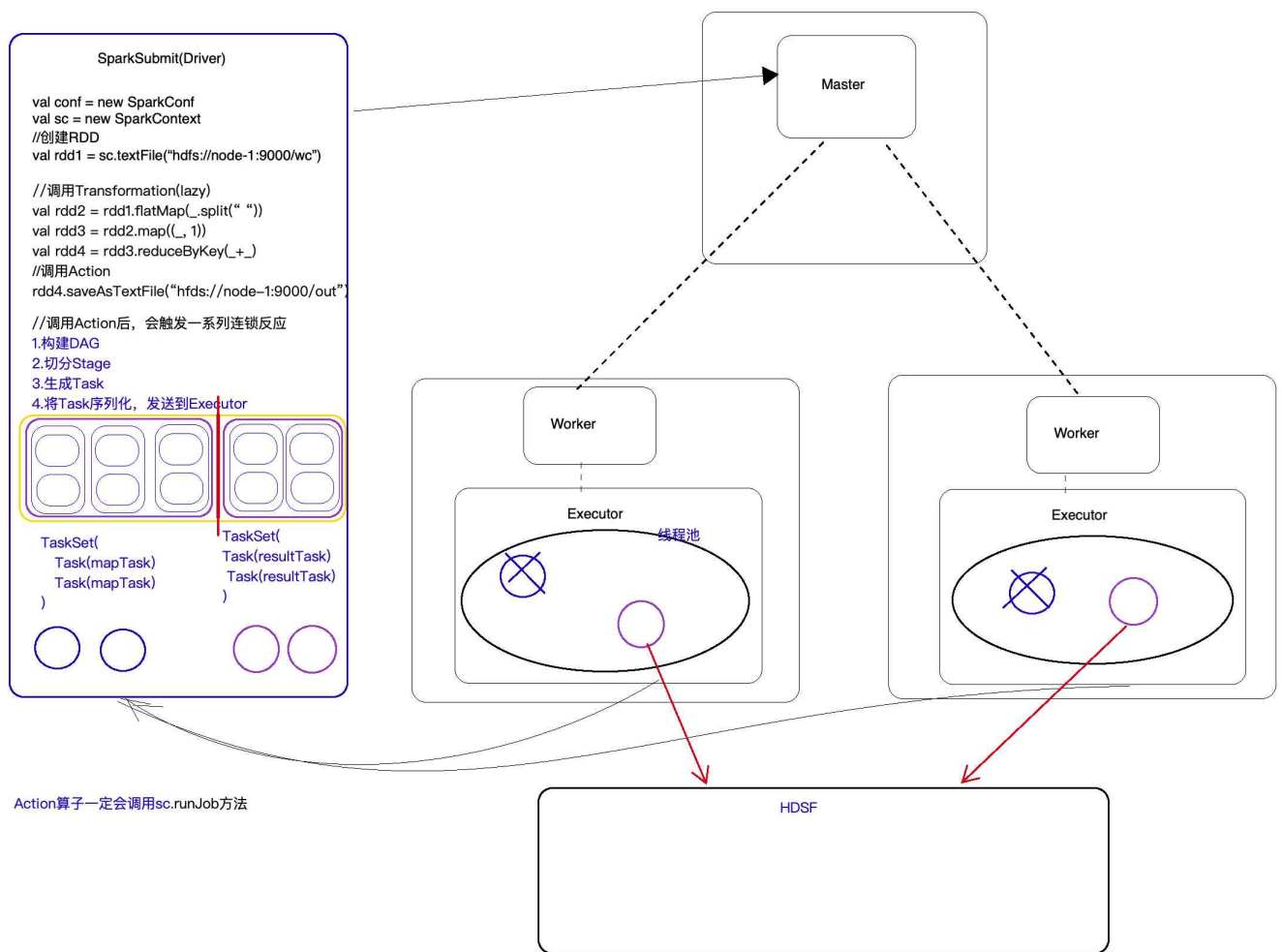
6.6.31 cartesian

笛卡尔积

```
1 val rdd1 = sc.parallelize(List("tom", "jerry"), 2)
2 val rdd2 = sc.parallelize(List("tom", "kitty", "shuke"), 3)
3 val rdd3 = rdd1.cartesian(rdd2)
```

6.7 RDD的Action算子

Action算子会触发Job的生成，底层调用的是sparkContext.runJob方法，根据最后一个RDD，从后往前，切分Stage，生成Task



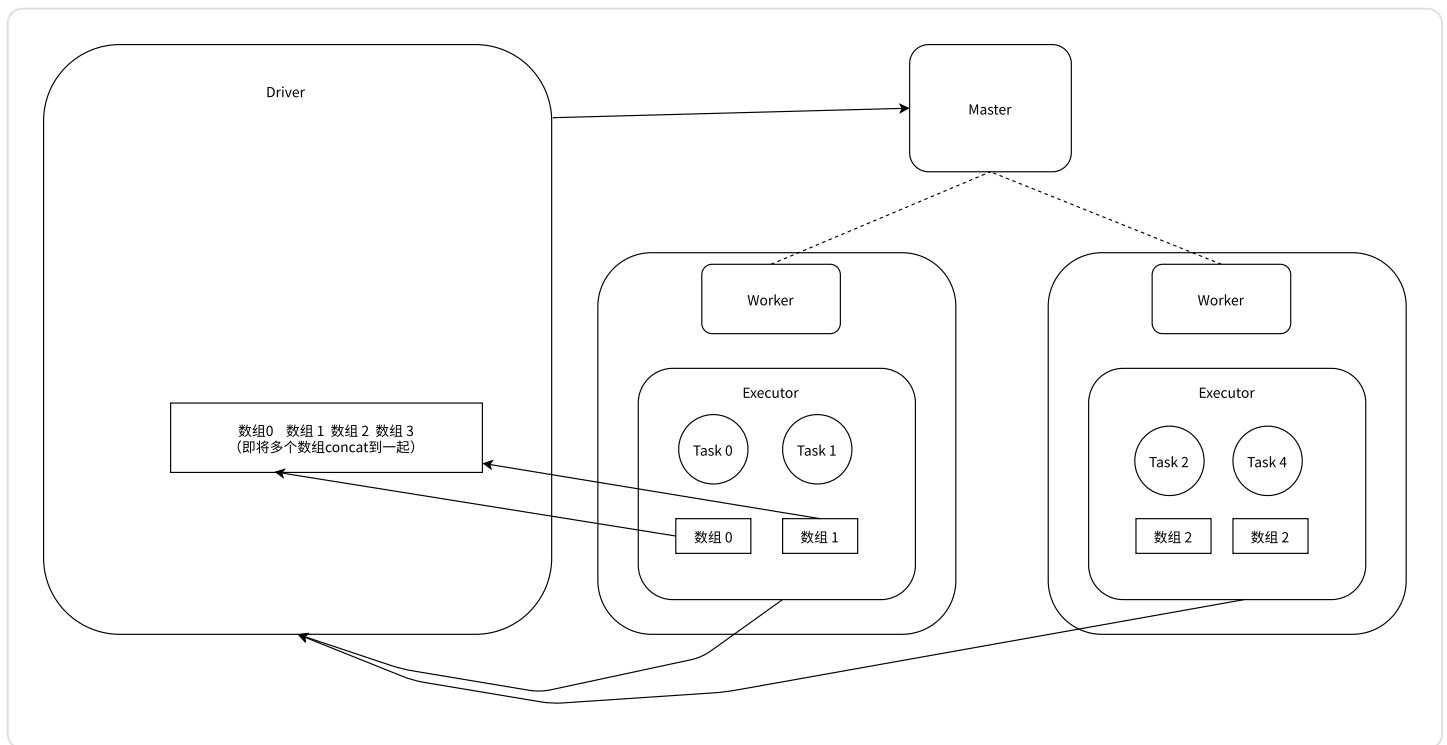
6.7.1 saveAsTextFile

将数据以文本的形式保存到文件系统中，一个分区对应一个结果文件，可以指定hdfs文件系统，也可以指定本地文件系统（本地文件系统要写file://协议），数据的写入是下Executor中Task写入的，是多个Task并行写入的。

```
1 val rdd1 = sc.parallelize(List(1,2,3,4,5), 2)
2 rdd1.saveAsTextFile("hdfs://node-1.51doit.cn:9000/out2")
```

6.7.2 collect

每个分区对应的Task，将数据在Executor中，将数据以集合的形式保存到内存中，然后将每个分区对应的数据以数组形式通过网络收集回Driver端，数据按照分区编号有序返回



```
1
2 val rdd1 = sc.parallelize(List(1,2,3,4,5,6,7,8,9,10), 4)
3 val rdd2 = rdd1.map(_ * 10)
4 //调用collect方法, 是一个Action
5 val res: Array[Int] = rdd2.collect()
6 println(res.toBuffer)
```

collect底层实现:

```
1 def collect(): Array[T] = withScope {
2   //this代表最后一个RDD, 即触发Action的RDD
3   //(iter: Iterator[T]) => iter.toArray 函数代表对最后一个进行的处理逻辑, 即将每个分区
4   //最后将没法分区对应的数组通过网络传输到Driver端
5   val results = sc.runJob(this, (iter: Iterator[T]) => iter.toArray)
6   //在Driver端, 将多个数组合并成一个数组
7   Array.concat(results: _*)
8 }
```

! 使用collect方法的注意事项:

如果Driver的内存相对较小, 并且每个分区对应的数据比较大, 通过网络传输的数据, 返回到Driver, 当返回到Driver端的数据达到了一定大小, 就不收集了, 即将一部分无法收集的数据丢弃

如果需要将大量的数据收集到Driver端，那么可以在提交任务的时候指定Driver的内存大小（`-driver-memory 2g`）

6.7.3 aggregate

aggregate方式是Action，可以将多个分区的数据进行聚合运算，例如进行相加，比较大小等

! aggregate方法可以指定一个初始值，初始值在每个分区进行聚合时会应用一次，全局聚合时会使用一次

```
1 val rdd1 = sc.parallelize(List(1,2,3,4,5,6,7,8,9,10), 4)
2
3 //f1是在Executor端执行的
4 val f1 = (a: Int, b: Int) => {
5   println("f1 function invoked ~~~~")
6   a + b
7 }
8
9 //f2实在Driver端执行的
10 val f2 = (m: Int, n: Int) => {
11   println("f2 function invoked !!!!")
12   m + n
13 }
14
15 //返回的结果为55
16 val r1: Int = rdd1.aggregate(0)(f1, f2)
17
18 //返回的结果为50055
19 val r2: Int = rdd1.aggregate(10000)(f1, f2)
20
```

```
1 val rdd1 = sc.parallelize(List("a", "b", "c", "d"), 2)
2 val r: String = rdd1.aggregate("&")(_ + _, _ + _)
3
4 //返回的回的有两种：应为task的分布式并行运行的，先返回的结果在前面
5 // &&cd&ab 或 &&ab&cd
```

6.7.4 reduce

将数据先在每个分区内进行局部聚合，然后将每个分区返回的结果在Driver端进行全局聚合

```
1
2 val rdd1 = sc.parallelize(List(1,2,3,4,5,6,7,8,9,10), 4)
3 val f1 = (a: Int, b: Int) => {
4   println("f1 function invoked ~~~~")
5   a + b
6 }
7 //f1这个函数即在Executor中执行，又在Driver端执行
8 //reduce方法局部聚合的逻辑和全局聚合的逻辑是一样的
9 //局部聚合是在每个分区内完成 (Executor)
10 //全局聚合实在Driver完成的
11 val r = rdd1.reduce(f1)
```

6.7.5 sum

sum方法是Action，实现的逻辑只能是相加

```
1 val rdd1 = sc.parallelize(List(1,2,3,4,5,6,7,8,9,10), 4)
2 //sum底层调用的是fold，该方法是一个柯里化方法，第一个括号传入的初始值是0.0
3 //第二个括号传入的函数(_ + _)，局部聚合和全局聚合都是相加
4 val r = rdd1.sum()
```

6.7.6 fold

fold跟reduce类似，只不过fold是一个柯里化方法，第一个参数可以指定一个初始值

```
1 val rdd1 = sc.parallelize(List(1,2,3,4,5,6,7,8,9,10), 4)
2 //fold与reduce方法类似，该方法是一个柯里化方法，第一个括号传入的初始值是0.0
3 //第二个括号传入的函数(_ + _)，局部聚合和全局聚合都是相加
4 val r = rdd1.fold(0)(_ + _)
```

6.7.7 min、max

将整个RDD中全部对应的数据求最大值或最小值，底层的实现是：现在每个分区内求最大值或最小值，然后将每个分区返回的数据在Driver端再进行比较（min、max没有shuffle）

```
1 val rdd1 = sc.parallelize(List(5,7 ,9,6,1 ,8,2, 4,3,10), 4)
2 //没有shuffle
3 val r: Int = rdd1.max()
```

6.7.8 count

返回rdd元素的数量，先在每个分区内求数据的条数，然后再将每个分区返回的条数在Driver进行求和

```
1 val rdd1 = sc.parallelize(List(5,7 ,9,6,1 ,8,2, 4,3,10), 4)
2 //在每个分区内先计算每个分区对应的数据条数（使用的是边遍历，边计数）
3 //然后再将每个分区返回的条数，在Driver进行求和
4 val r: Long = rdd1.count()
```

6.7.9 take

返回一个由数据集的前n个元素组成的数组，即从RDD的0号分区开始取数据，take可能触发一到多次Action（可能生成多个Job）因为首先从0号分区取数据，如果取够了，就直接返回，没有取够，再触发Action，从后面的分区继续取数据，直到取够指定的条数为止

```
1 val rdd1 = sc.parallelize(List(5,7 ,9,6,1 ,8,2, 4,3,10), 4)
2 //可能会触发一到多次Action
3 val res: Array[Int] = rdd1.take(2)
```

6.7.10 first

返回RDD中的第一个元素，类似于take(1)，first返回的不是数组

```
1 val rdd1 = sc.parallelize(List(5,7 ,9,6,1 ,8,2, 4,3,10), 4)
2 //返回RDD中对应的第一条数据
```

```
3 val r: Int = rdd1.first()
```

6.7.11 top

将RDD中数据按照降序或者指定的排序规则，返回前n个元素

```
1 val rdd1 = sc.parallelize(List(  
2   5, 7, 6, 4,  
3   9, 6, 1, 7,  
4   8, 2, 8, 5,  
5   4, 3, 10, 9  
6 ), 4)  
7  
8 val res1: Array[Int] = rdd1.top(2)  
9 //指定排序规则，如果没有指定，使用默认的排序规则  
10 implicit val ord = Ordering[Int].reverse  
11 val res2: Array[Int] = rdd1.top(2)  
12 val res3: Array[Int] = rdd1.top(2)(Ordering[Int].reverse)
```

top底层调用的使用takeOrdered

```
1 def top(num: Int)(implicit ord: Ordering[T]): Array[T] = withScope {  
2   takeOrdered(num)(ord.reverse)  
3 }
```

6.7.12 takeOrdered

top底层丢的是takeOrdered，takeOrdered更灵活，可以传指定排序规则。底层是先在每个分区内求topN，然后将每个分区返回的结果再在Driver端求topN

! 在每个分区内进行排序，使用的是有界优先队列，特点是数据添加到其中，就会按照指定的排序规则排序，并且允许数据重复，最多只存放最大或最小的N个元素

```

1 def takeOrdered(num: Int)(implicit ord: Ordering[T]): Array[T] = withScope {
2   if (num == 0) {
3     Array.empty
4   } else {
5     val mapRDDs = mapPartitions { items =>
6       // Priority keeps the largest elements, so let's reverse the ordering.
7       //使用有界优先队列
8       val queue = new BoundedPriorityQueue[T](num)(ord.reverse)
9       queue += collectionUtils.takeOrdered(items, num)(ord)
10      Iterator.single(queue)
11    }
12    if (mapRDDs.partitions.length == 0) {
13      Array.empty
14    } else {
15      mapRDDs.reduce { (queue1, queue2) =>
16        queue1 += queue2 //将多个有界优先队列进行+=，返回两个有界优先队列最大的N个
17        queue1
18      }.toArray.sorted(ord)
19    }
20  }
21 }

```

6.7.13 foreach

将数据一条一条的取出来进行处理，函数没有返回

```

1 val sc = SparkUtil.getContext("FlowCount", true)
2
3 val rdd1 = sc.parallelize(List(
4   5, 7, 6, 4,
5   9, 6, 1, 7,
6   8, 2, 8, 5,
7   4, 3, 10, 9
8 ), 4)
9
10 rdd1.foreach(e => {
11   println(e * 10) //函数是在Executor中执行
12 })

```

使用foreach将数据写入到MySQL中，不好，效率低

```

1 rdd1.foreach(e => {
2     //但是不好，为什么？
3     //每写一条数据用一个连接对象，效率太低了
4     val connection = DriverManager.getConnection("jdbc:mysql://node-1.51doit.cn:33
5     val preparedStatement = connection.prepareStatement("Insert into tb_res values
6     preparedStatement.setInt(1, e)
7     preparedStatement.executeUpdate()
8 })

```

6.7.14 foreachPartition

和foreach类似，只不过是以分区位单位，一个分区对应一个迭代器，应用外部传的函数，函数没有返回值，通常使用该方法将数据写入到外部存储系统中，一个分区获取一个连接，效率更高

```

1 rdd1.foreachPartition(it => {
2     //先创建好一个连接对象
3     val connection = DriverManager.getConnection("jdbc:mysql://node-1.51doit.cn:33
4     val preparedStatement = connection.prepareStatement("Insert into tb_res values
5     //一个分区中的多条数据用一个连接进行处理
6     it.foreach(e => {
7         preparedStatement.setInt(1, e)
8         preparedStatement.executeUpdate()
9     })
10    //用完后关闭连接
11    preparedStatement.close()
12    connection.close()
13 })

```

6.8 RDD特殊的算子

6.8.1 cache、persist

将数据缓存到内存，第一次触发Action，才会将数据放入内存，以后在触发Action，可以复用前面内存中缓存的数据，可以提升技术效率

cache和persist的使用场景：一个application多次触发Action，为了复用前面RDD的数据，避免反复读取HDFS（数据源）中的数据和重复计算，可以将数据缓存到内存或磁盘【executor所在的磁盘】，第一次触发action才放入到内存或磁盘，以后会缓存的RDD进行操作可以复用缓存的数据。

一个RDD多次触发Action缓存才有意义，如果将数据缓存到内存，内存不够，以分区位单位，只缓存部分分区的数据，cache底层调用persist，可以指定更加丰富的存储基本，支持多种StageLevel，可以将数据序列化,默认放入内存使用的是java对象存储，但是占用空间大，优点速度快，也可以使用其他的序列化方式

cache和persist方法，严格来说，不是Transformation，应为没有生成新的RDD，只是标记当前rdd要cache或persist

6.8.2 checkpoint

checkpoint使用场景：适合复杂的计算【机器学习、迭代计算】，为了避免中间结果数据丢失重复计算，可以将宝贵的中间结果保存到hdfs中，保证中间结果安全。

在调用rdd的checkpint方法之前，一定要指定checkpoint的目录sc.setCheckPointDir，指的HDFS存储目录，为保证中间结果安全，将数据保存到HDFS中

第一次触发Action，才做checkpoint，会额外触发一个job，这个job的目的就是将结果保存到HDFS中如果RDD做了checkpoint，这个RDD以前的依赖关系就不在使用了，触发多次Action，checkpoint才有意义，多用于迭代计算

checkpoint严格的说，不是Transformation，只是标记当前RDD要做checkpoint

7. Spark中的一些重要概念

7.1 Application

使用SparkSubmit提交的个计算应用，一个Application中可以触发多次Action，触发一次Action产生一个Job，一个Application中可以有一到多个Job

7.2 Job

Driver向Executor提交的作业，触发一次Acition形成一个完整的DAG，一个DAG对应一个Job，一个Job中有一到多个Stage，一个Stage中有一到多个Task

7.3 DAG

概念：有向无环图，是对多个RDD转换过程和依赖关系的描述，触发Action就会形成一个完整的DAG，一个DAG对应一个Job

7.4 Stage

概念：任务执行阶段，Stage执行是有先后顺序的，先执行前的，在执行后面的，一个Stage对应一个TaskSet，一个TaskSet中的Task的数量取决于Stage中最后一个RDD分区的数量

7.5 Task

概念：Spark中任务最小的执行单元，Task分类两种，即ShuffleMapTask和ResultTask

Task其实就是类的实例，有属性（从哪里读取数据），有方法（如何计算），Task的数量决定并行度，同时也要考虑可用的cores

7.6 TaskSet

保存同一种计算逻辑多个Task的集合，一个TaskSet中的Task计算逻辑都一样，计算的数据不一样

8. 任务执行原理分析

8.1 WordCount程序有多个RDD

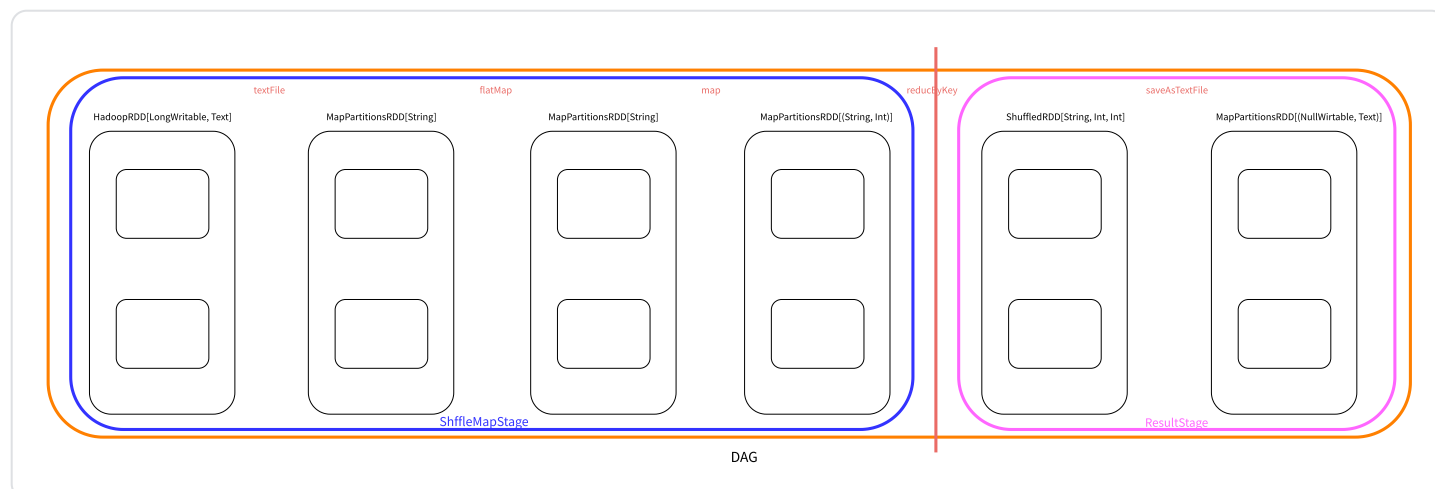
该Job中，有多少个RDD，多少个Stage，多少个TaskSet，多个Task，Task的类型有哪些？

```
1  object WordCount {
2
3  def main(args: Array[String]): Unit = {
4      val conf: SparkConf = new SparkConf()
5      val sc: SparkContext = new SparkContext(conf)
6
7      sc.textFile(args(0))
8          .flatMap(_.split(" "))
9          .map((_, 1))
10         .reduceByKey(_+_ )
11         .saveAsTextFile(args(1))
12
13     sc.stop()
14 }
15 }
```

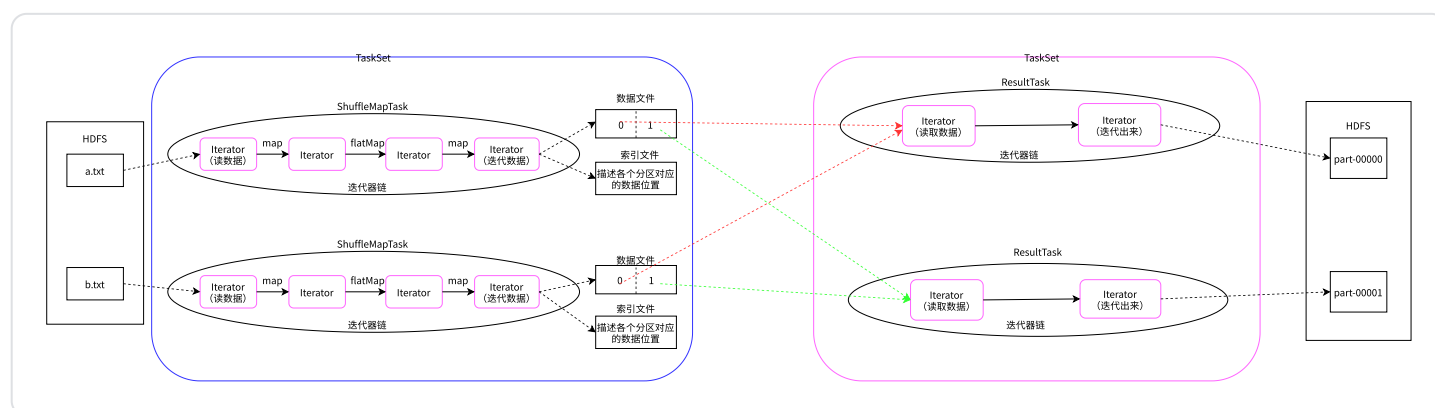
8.2 RDD的数量分析

- 读取hdfs中的目录有两个输入切片，最原始的HadoopRDD的分区为2，以后没有改变RDD的分区数量，RDD的分区数量都是2
- 在调用reduceByKey方法时，有shuffle产生，要划分Stage，所以有两个Stage
- 第一个Stage的并行度为2，所以有2个Task，并且为ShuffleMapTask。第二个Stage的并行度也为2，所以也有2个Task，并且为ResultTask，所以一共有4个Task

spark的任务上次的逻辑计划图



下面的物理执行计划图，会生成Task，生成的Task如下



8.3 Stage和Task的类型

Stage有两种类型，分别是ShuffleMapStage和ResultStage，ShuffleMapStage生成的Task叫做ShuffleMapTask，ResultStage生成的Task叫做ResultTask

- ShuffleMapTask

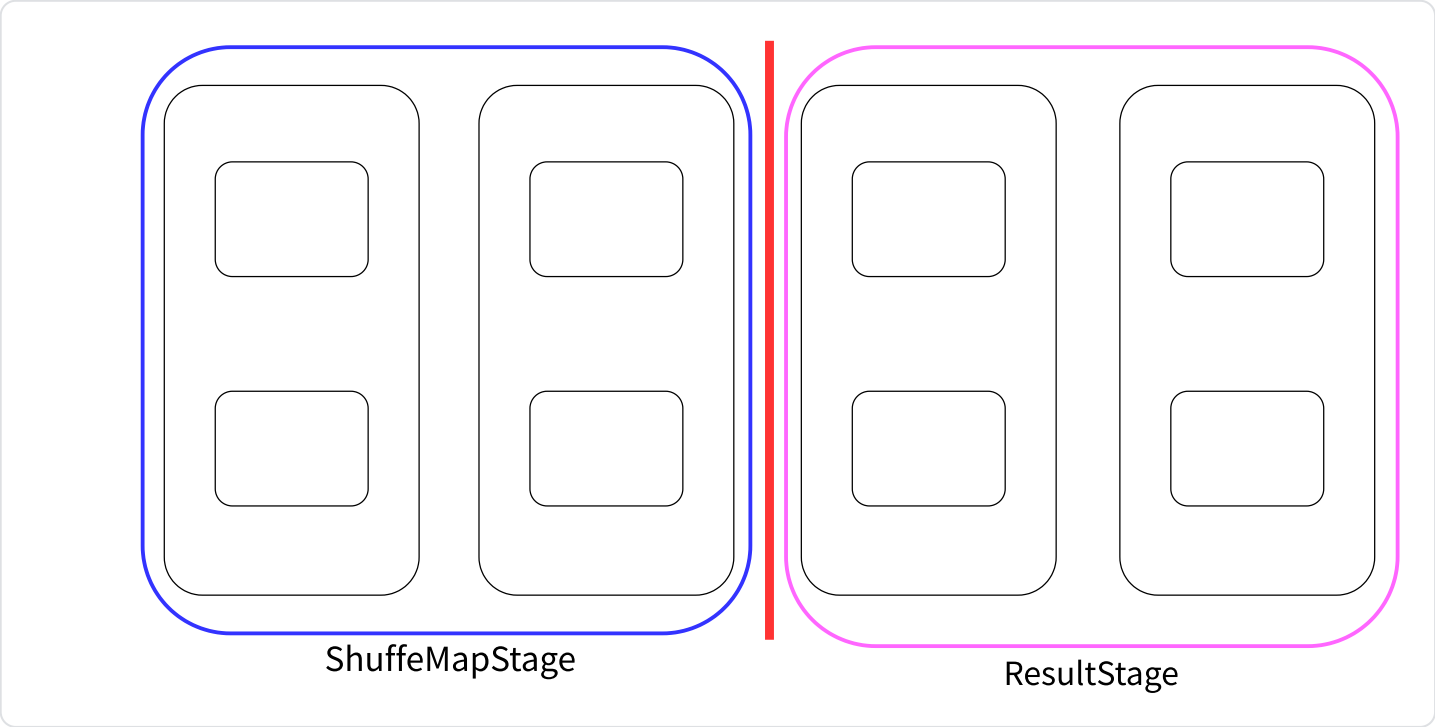
- 1.可以读取各种数据源的数据
- 2.可以读取Shuffle的中间结果（Shuffle Read）
- 3.为shuffle做准备，即应用分区器，将数据溢写磁盘（ShuffleWrite），后面一定还会有其他的Stage

- ResultTask

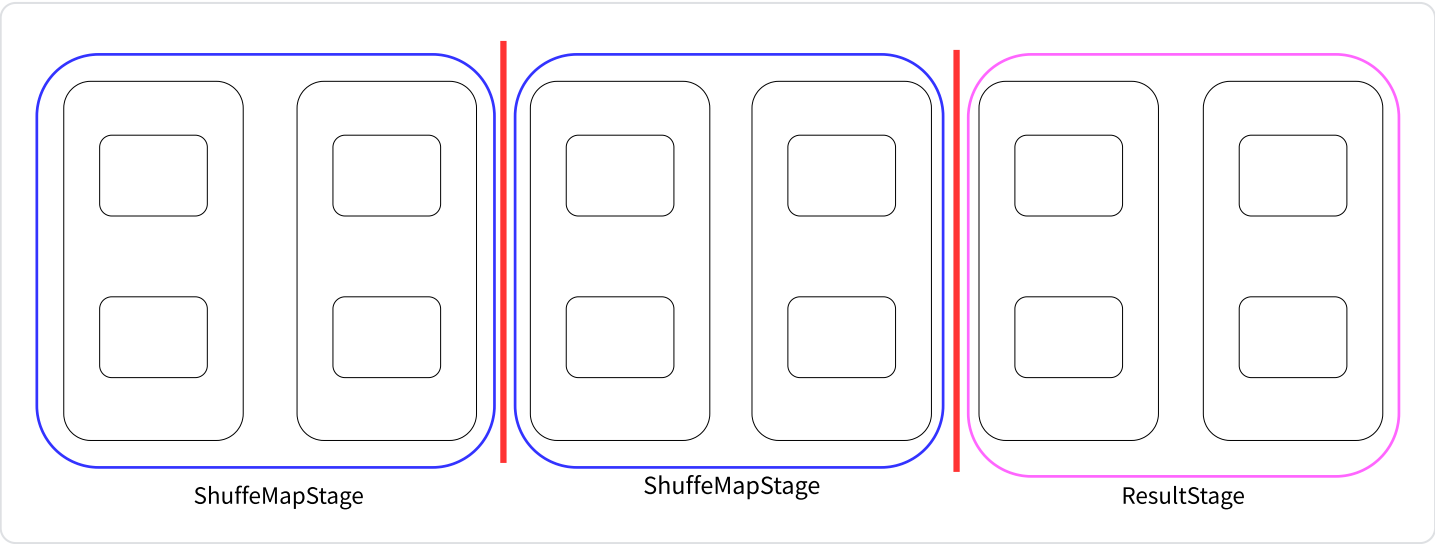
- 1.可以读取各种数据源的数据
- 2.可以读取Shuffle的中间结果（Shuffle Read）
- 3.是整个job中最后一个阶段对应的Task，一定会产生结果数据（就是将产生的结果返回的Driver或写入到外部的存储系统）

多种情况：

第一种：

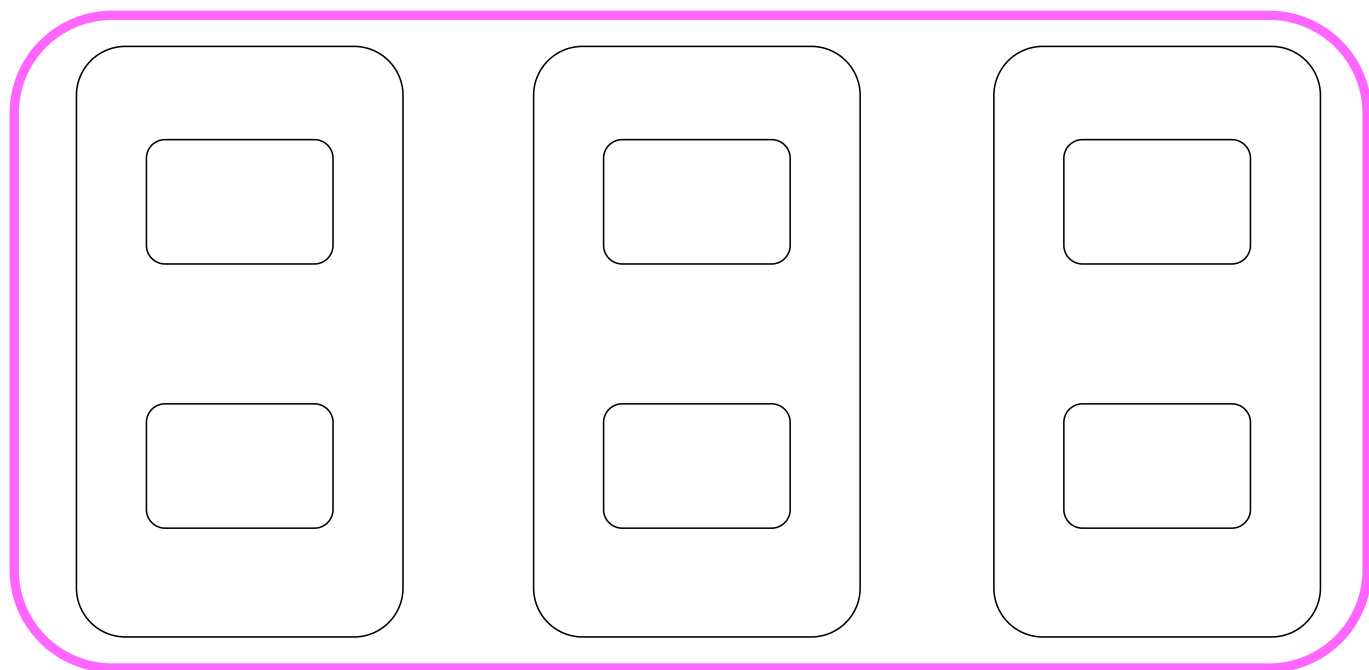


第二种



第三种：

计算任务比较简单，没有shuffle



ResultStage

9. Shuffle的深入理解

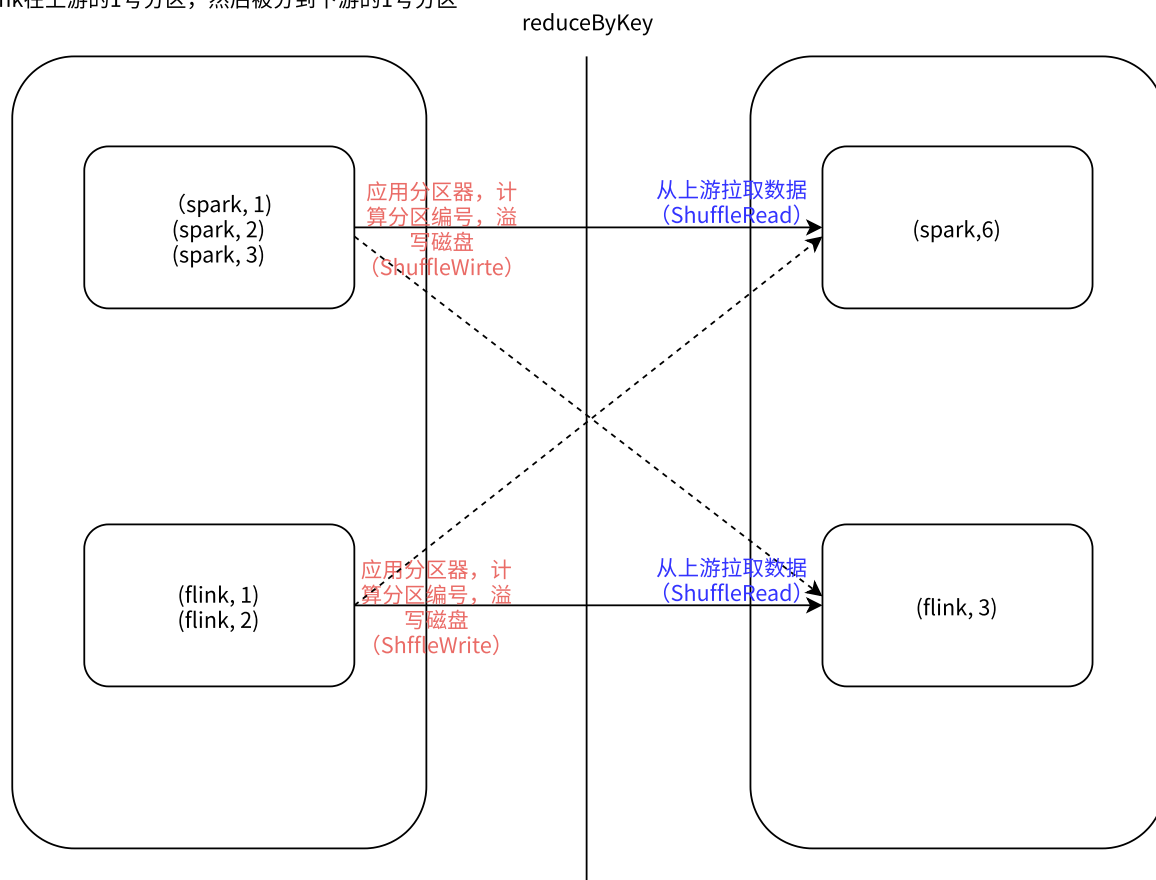
什么是Shuffle，本意为洗牌，在数据处理领域里面，意为将数打散。

问题：shuffle一定有网络传输吗？有网络传输的一定是Shuffle吗？

9.1 Shuffle的概念

通过网络将数据传输到多台机器，数据被打散，但是有网络传输，不一定就有shuffle，Shuffle的功能是将具有相同规律的数据按照指定的分区器的分区规则，通过网络，传输到指定的机器的一个分区中，需要注意的是，不是上游的Task发送给下游的Task，而是下游的Task到上游拉取数据。

假设spark在上游的0号分区，然后被分到下游的0号分区
flink在上游的1号分区，然后被分到下游的1号分区



Shuffle的本质是，上游的Task有没有应用分区器，溢写磁盘，下游的Task有没有到上游拉取数据

9.2 reduceByKey一定会Shuffle吗

不一定，如果一个RDD事先使用了HashPartitioner分区先进行分区，然后再调用reduceByKey方法，使用的也是HashPartitioner，并且没有改变分区数量，调用redcueByKey就不shuffle

❗ 如果自定义分区器，多次使用自定义的分区器，并且没有改变分区的数量，为了减少shuffle的次数，提高计算效率，需要重新自定义分区器的equals方法

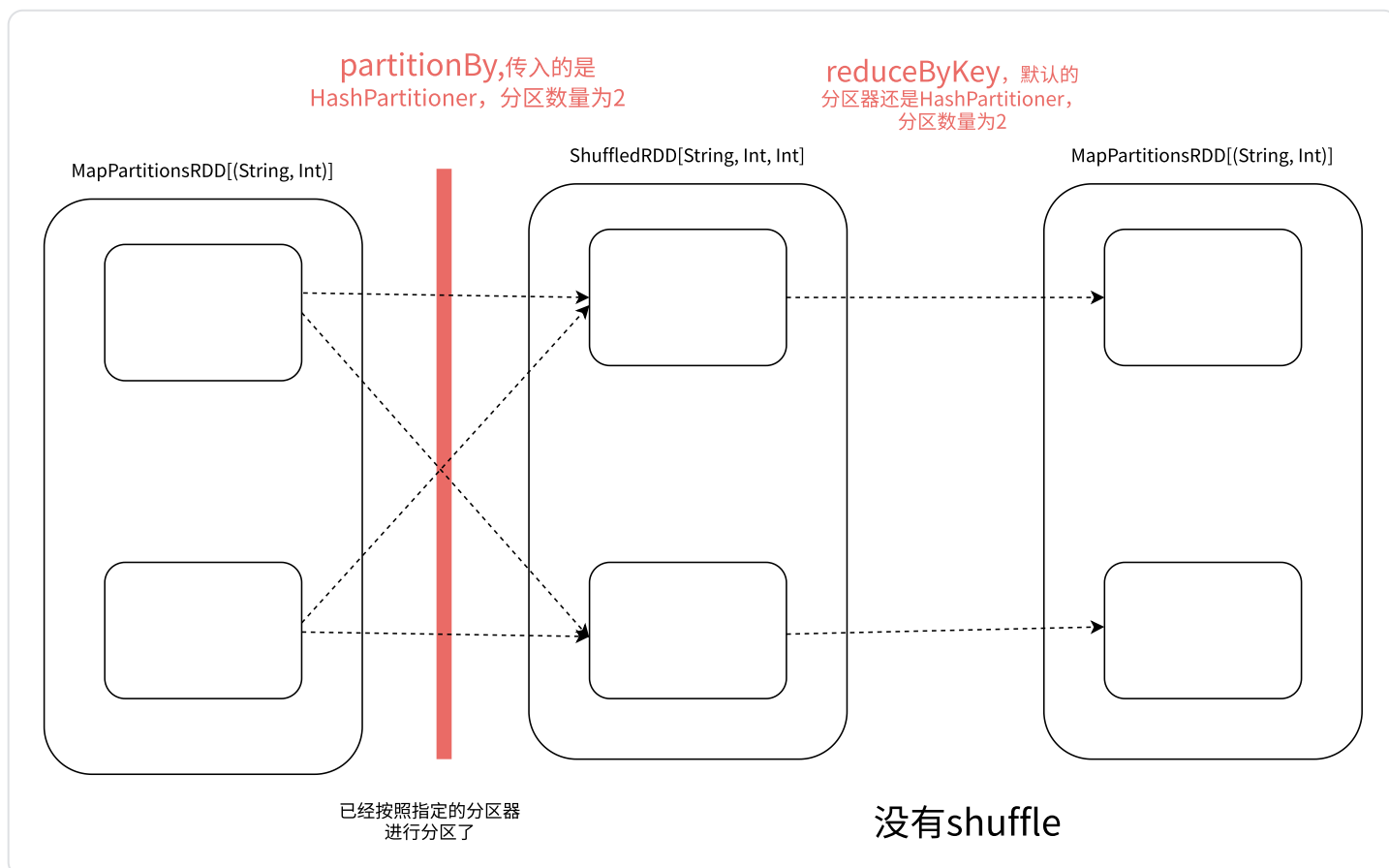
例如：

```
1 //创建RDD，并没有立即读取数据，而是触发Action才会读取数据
2 val lines = sc.textFile("hdfs://node-1.51doit.cn:9000/words")
3
4 val wordAndOne = lines.flatMap(_.split(" ")).map((_, 1))
5 //先使用HashPartitioner进行partitionBy
6 val partitioner = new HashPartitioner(wordAndOne.partitions.length)
```

```

7 val partitioned = wordAndOne.partitionBy(partitioner)
8 //然后再调用reduceByKey
9 val reduced: RDD[(String, Int)] = partitioned.reduceByKey(_ + _)
10
11 reduced.saveAsTextFile("hdfs://node-1.51doit.cn:9000/out-36-82")

```



9.3 join一定会Shuffle吗

不一定，join一般会shuffle，但是如果两个要join的rdd实现都使用相同的分区去进行分区了，并且join时，依然使用相同类型的分区器，并且没有改变分区数据，那么不shuffle

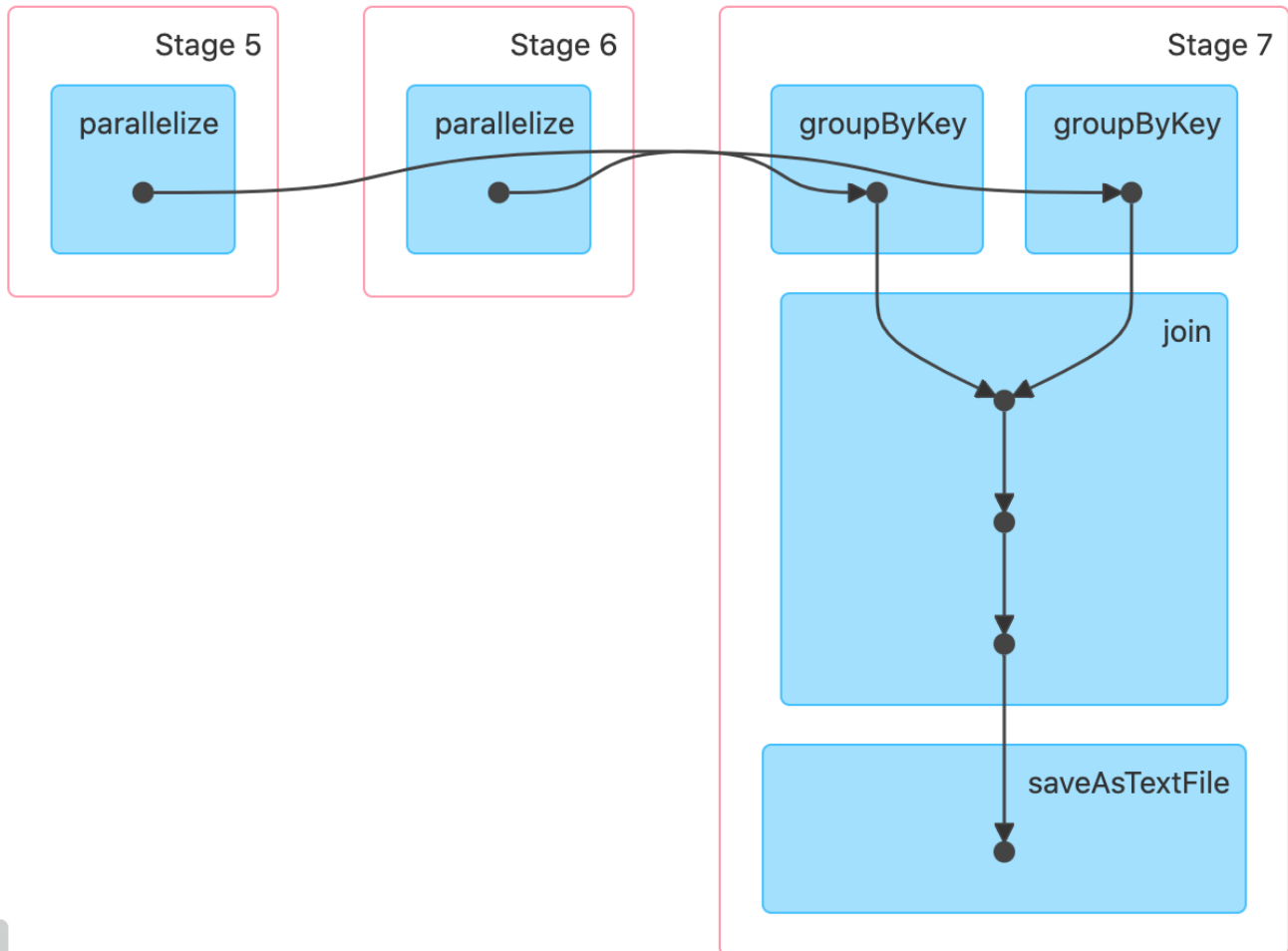
```

1 //通过并行化的方式创建一个RDD
2 val rdd1 = sc.parallelize(List(("tom", 1), ("tom", 2), ("jerry", 3), ("kitty", 2)
3 //通过并行化的方式再创建一个RDD
4 val rdd2 = sc.parallelize(List(("jerry", 2), ("tom", 1), ("shuke", 2), ("jerry",
5 //该join一定有shuffle, 并且是3个Stage
6 val rdd3: RDD[(String, (Int, Int))] = rdd1.join(rdd2)
7
8
9 val rdd11 = rdd1.groupByKey()

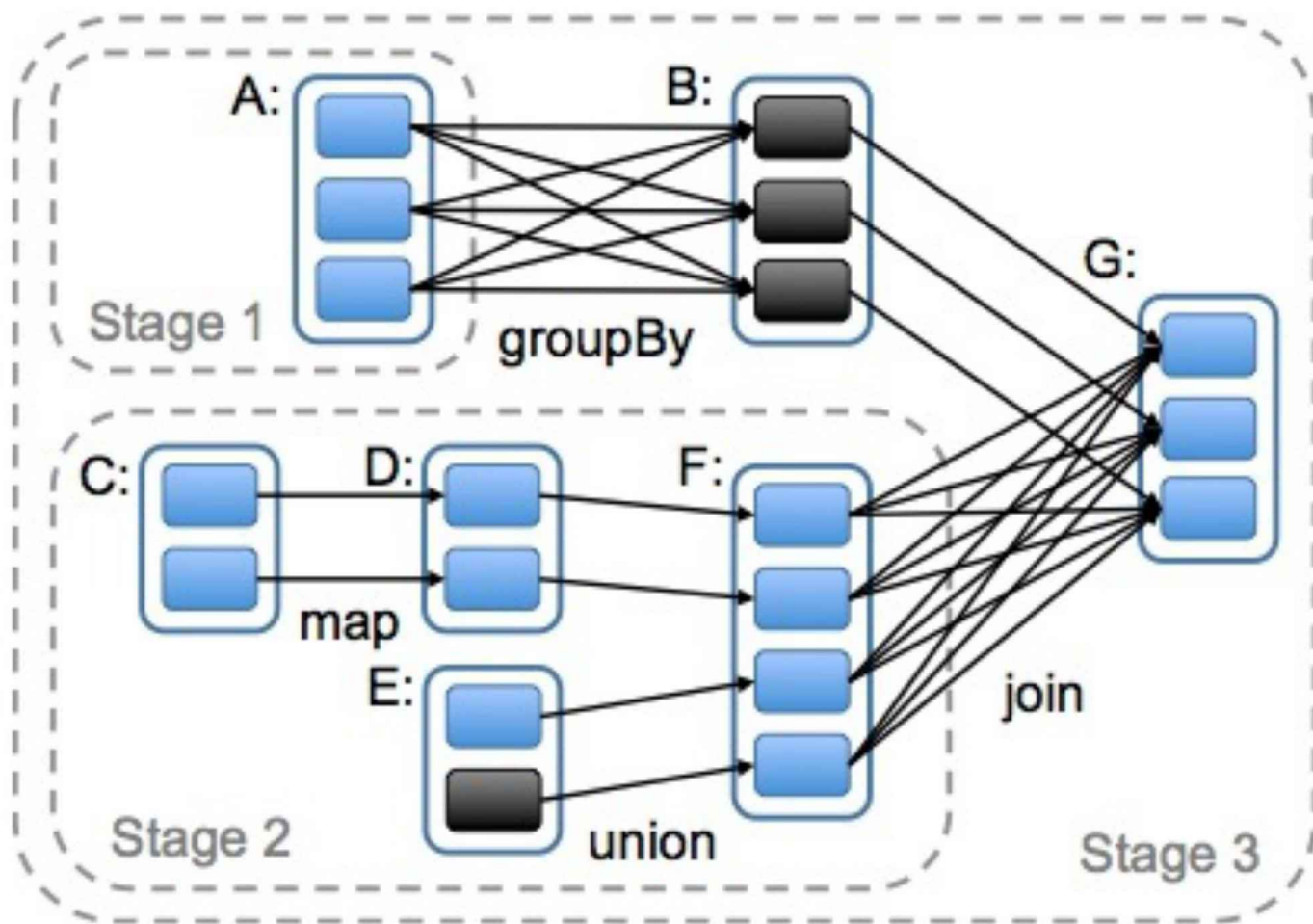
```

```
10 val rdd22 = rdd2.groupByKey()
11 //下面的join, 没有shuffle
12 val rdd33 = rdd11.join(rdd22)
13
14 rdd33.saveAsTextFile("hdfs://node-1.51doit.cn:9000/out-36-86")
```

▼ DAG Visualization



分析一下下面的图片，有几次shuffle，有几个Stage



上面的分支没有shuffle，因为实现已经使用groupBy进行分区了（使用了HashPartitioner，分区数量为3），在join是，使用的分区器也是HashPartitioner，分区数量为3，所有不shuffle

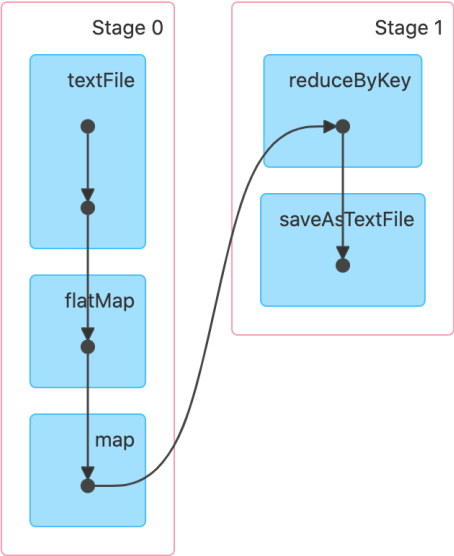
下面的分支没有实现进行分区（即使使用了HashPartitioner进行分区，但是jion后的分区数量发生了变化），所有要shuffle

9.4 shuffle数据的复用

spark在shuffle时，会应用分区器，当读取达到一定大小或整个分区的数据被处理完，会将数据溢写磁盘（数据文件和索引文件），溢写持磁盘的数据，会保存在Executor所在机器的本地磁盘（默认是保存在/temp目录，也可以配置到其他目录），只要application一直运行，shuffle的中间结果数据就会被保存。如果以后再次触发Action，使用到了以前shuffle的中间结果，那么就不会从源头重新计算而是，而是复用shuffle中间结果，所有说，shuffle是一种特殊的persist，以后再次触发Action，就会跳过前面的Stage，直接读取shuffle的数据，这样可以提高程序的执行效率。

正常情况：

▼ DAG Visualization



再次触发Action

 3.2.3

Jobs

Stages

Storage

Environment

Executors

Spark shell application UI

Spark Jobs (?)

User: root
Total Uptime: 7.7 min
Scheduling Mode: FIFO
Completed Jobs: 2

▶ Event Timeline

▼ Completed Jobs (2)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Job Id ▼	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
1	count at <console>:24 count at <console>:24	2022/12/31 17:46:17	0.9 s	1/1 (1 skipped)	2/2 (2 skipped)
0	runJob at SparkHadoopWriter.scala:83 runJob at SparkHadoopWriter.scala:83	2022/12/31 17:44:23	45 s	2/2	4/4

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

DAAG Visualization

Stage 2 (skipped)

Stage 3

reduceByKey

前面的 Stage 是 skipped 状态，因为以前触发一次 Action，shuffle 的中间结果溢写到磁盘了，再次触发 Action，会复用前面 shuffle 的中间结果，避免重复运算

Completed Stages (1)

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
3	count at <console>:24 +details	2022/12/31 17:46:17	0.8 s	2/2			256.0 B	

如果由于机器宕机或磁盘问题，部分shuffle的中间数据丢失，以后再次触发Action，使用到了shuffle中间结果数据，但是部数据无法访问，spark会根据RDD的依赖关系（RDD的血统）重新生成对应分区的Task，重新计算丢失的数据！

```
(FileSegmentManagedBuffer.java:112)
    at org.apache.spark.storage.ShuffleBlockFetcherIterator.next(ShuffleBlockFetcherIterator.scala:800)
    ... 21 more
Caused by: java.io.FileNotFoundException: /tmp/spark-b46efd2b-1020-4491-9e8a-4353d73f9a12/executor-5eac118d-b32c-4f62-9708-9f9692638086/blockmgr-1b0546a1-1d22-446c-bb91-56392b12c12f/0c/shuffle_0_0_0.data (No such file or directory)
    at java.io.FileInputStream.open0(Native Method)
    at java.io.FileInputStream.open(FileInputStream.java:195)
    at java.io.FileInputStream.<init>(FileInputStream.java:138)
    at org.apache.spark.network.buffer.FileSegmentManagedBuffer.createInputStream(FileSegmentManagedBuffer.java:101)
    ... 22 more
)
res2: Long = 5
```

如果将 shuffle 的中间结果数据删除，程序会报错误，但是不会退出，而是根据 RDD 的血统关系重新生成对应分区的 Task，然后重新计算丢失的数据，最后依然得到最终结果

Spark Jobs (?)

User: root
Total Uptime: 16 min
Scheduling Mode: FIFO
Completed Jobs: 3

Event Timeline

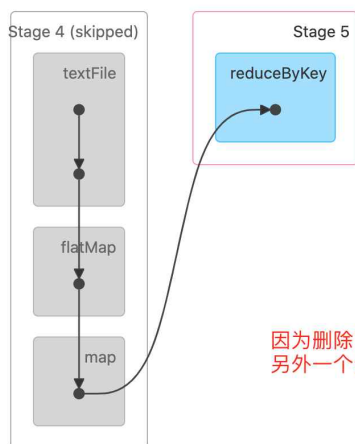
Completed Jobs (3)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
2	count at <console>:24 count at <console>:24	2022/12/31 17:52:37	3 s	2/1 (1 failed) (1 skipped)	3/2 (1 failed) (2 skipped) (1 killed: Stage finished)
1	count at <console>:24 count at <console>:24	2022/12/31 17:46:17	0.9 s	1/1 (1 skipped)	2/2 (2 skipped)
0	runJob at SparkHadoopWriter.scala:83 runJob at SparkHadoopWriter.scala:83	2022/12/31 17:44:23	45 s	2/2	4/4

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

DAG Visualization



因为删除了一份分区 shuffle 的中间结果文件，再次触发 Action，重新生成一个 Task 计算丢失分区对应的数据，另外一个分区对应的 shuffle 中间结果没有丢失，所以不用重头计算，就是 skipped 状态

Completed Stages (2)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
5 (retry 1)	count at <console>:24 +details	2022/12/31 17:52:39	0.7 s	2/2			256.0 B	
4 (retry 1)	map at <console>:23 +details	2022/12/31 17:52:38	0.7 s	1/1	928.0 B			128.0 B

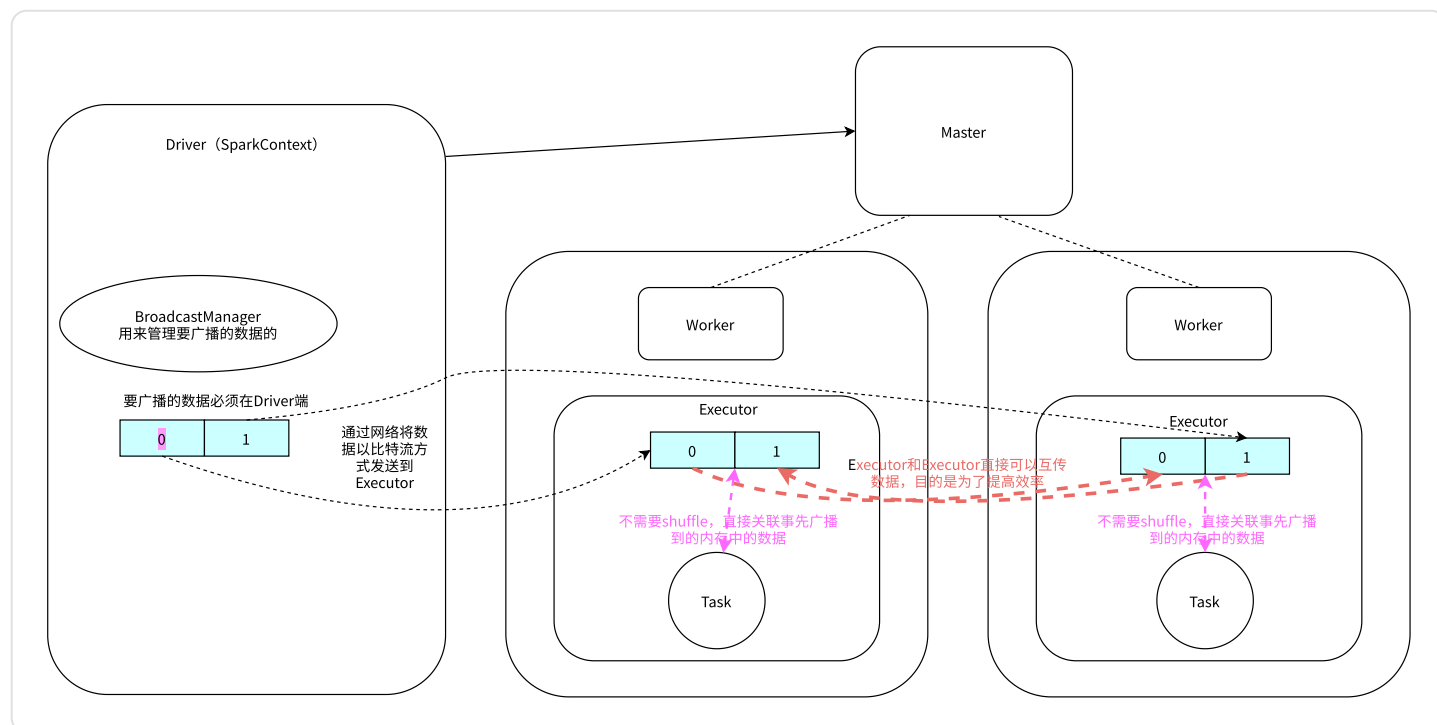
10. 广播变量

10.1 广播变量的使用场景

在很多计算场景，经常会遇到两个RDD进行JOIN，如果一个RDD对应的数据比较大，一个RDD对应的数据比较小，如果使用JOIN，那么会shuffle，导致效率变低。广播变量就是将相对较小的数据，先收集到Driver，然后再通过网络广播到属于该Application对应的每个Executor中，以后处理大量数据对

应的RDD关联数据，就不用shuffle了，而是直接在内存中关联已经广播好的数据，即通实现mapside join，可以将Driver端的数据广播到属于该application的Executor，然后通过Driver广播变量返回的引用，获取实现广播到Executor的数据

广播变量的特点：广播出去的数据就无法在改变了，在没有Executor中是只读的操作，在每个Executor中，多个Task使用一份广播变量



10.2 广播变量的实现原理

广播变量是通过BT的方式广播的（TorrentBroadcast），多个Executor可以相互传递数据，可以提高效率

sc.broadcast这个方法是阻塞的（同步的）

广播变量一旦广播出去就不能改变，为了以后可以定期的改变要关联的数据，可以定义一个object[单例对象]，在函数内使用，并且加一个定时器，然后定期更新数据

广播到Executor的数据，可以在Driver获取到引用，然后这个引用会伴随着每一个Task发送到Executor，然后通过这个引用，获取到事先广播好的数据

10.3 案例：根据IP计算归属地

10.3.1 需求

根据IP规则数据，计算出给定日志中ip地址对应的省份信息，由于IP地址的规则数据相对较小，所以可以将IP规则数据先广播出去，以后关联IP规则数据，就可以在内存中进行关联了，这样可以避免shuffle，提高执行效率！

10.3.2 代码实现

11. 序列化问题

11.1 序列化问题的场景

spark任务在执行过程中，由于编写的程序不当，任务在执行时，会出序列化问题，通常有以下两种情况，

- 封装数据的Bean没有实现序列化接口（Task已经生成了）
- 函数闭包问题，即函数的内部，使用到了外部没有实现序列化的引用（Task没有生成）

11.2 数据Bean未实现序列化接口

spark在运算过程中，由于很多场景必须要shuffle，即向数据溢写磁盘并且在网络间进行传输，但是由于封装数据的Bean没有实现序列化接口，就会导致出现序列化的错误！

```
1
2 object C02_CustomSort {
3
4   def main(args: Array[String]): Unit = {
5
6     val sc = SparkUtil.getContext(this.getClass.getSimpleName, true)
7     //使用并行化的方式创建RDD
8     val lines = sc.parallelize(
9       List(
10         "laoduan,38,99.99",
11         "nianhang,33,99.99",
12         "laozhao,18,9999.99"
13       )
14     )
15     val tfBoy: RDD[Boy] = lines.map(line => {
16       val fields = line.split(",")
17       val name = fields(0)
18       val age = fields(1).toInt
19       val fv = fields(2).toDouble
20       new Boy(name, age, fv) //将数据封装到一个普通的class中
```

```

21     })
22
23     implicit val ord = new Ordering[Boy] {
24         override def compare(x: Boy, y: Boy): Int = {
25             if (x.fv == y.fv) {
26                 x.age - y.age
27             } else {
28                 java.lang.Double.compare(y.fv, x.fv)
29             }
30         }
31     }
32     //sortBy会产生shuffle, 如果Boy没有实现序列化接口, Shuffle时会报错
33     val sorted: RDD[Boy] = tfBoy.sortBy(bean => bean)
34
35     val res = sorted.collect()
36
37     println(res.toBuffer)
38 }
39 }
40
41 //如果以后定义bean, 建议使用case class
42 class Boy(val name: String, var age: Int, var fv: Double) //extends Serializabl
43 {
44
45     override def toString = s"Boy($name, $age, $fv)"
46 }

```

11.3 函数闭包问题

11.3.1 闭包的现象

在调用RDD的Transformation和Action时，可能会传入自定义的函数，如果函数内部使用到了外部未被序列化的引用，就会报Task无法序列化的错误。原因是spark的Task是在Driver端生成的，并且需要通过网络传输到Executor中，Task本身实现了序列化接口，函数也实现了序列化接口，但是函数内部使用到的外部引用不支持序列化，就会函数导致无法序列化，从而导致Task没法序列化，就无法发送到Executor中了

SparkSubmit (Driver)

```
val sc = new SparkContext(conf)

//创建RDD
val lines = sc.textFile(args(1))
//创建一个没有实现序列化接口的object或class的实例
val rules = RuleObjectNotSer //没有实现序列化接口，并且是在在Driver端初始化的
//val rules = new RuleClassNotSer
//函数会在Executor中调用
val func = (line: String) => {
  val fields = line.split(",")
  val id = fields(0).toInt
  val code = fields(1)
  val name = rules.rulesMap.getOrElse(code, "未知") //闭包
  //获取当前线程ID
  val treadId = Thread.currentThread().getId
  //获取当前Task对应的分区编号
  val partitionId = TaskContext.getPartitionId()
  //获取当前Task运行时的所在机器的主机名
  val host = InetAddress.getLocalHost.getHostName
  (id, code, name, treadId, partitionId, host, rulesObj.toString)
}
lines.map(func) //在调用该方式时，就会检测函数是否可以序列化
因为在生成Task之前会对函数进行序列化检测，如果不支持序列化，
直接抛出异常，连Task都没有生成就报错了，是因为Task中的函数引
用的对象没法序列化，导致Task没法序列化，Task就没法发送到
Executor中
```

在调用RDD的Transformation或Action是传入函数，第一步就进行检测，即调用sc的clean方法
为了避免错误，在Driver初始化的object或class必须实现序列化接口，不然会报错误

```
1 def map[U: ClassTag](f: T => U): RDD[U] = withScope {
2   val cleanF = sc.clean(f) //检测函数是否可以序列化，如果可以直接将函数返回，如果不可以
3   new MapPartitionsRDD[U, T](this, (_, _, iter) => iter.map(cleanF))
4 }
```

```
1 private def ensureSerializable(func: AnyRef): Unit = {
2   try {
3     if (SparkEnv.get != null) {
4       //获取spark执行换的的序列化器，如果函数无法序列化，直接抛出异常，程序退出，根本就没
```

```

5     SparkEnv.get.closureSerializer.newInstance().serialize(func)
6   }
7 } catch {
8     case ex: Exception => throw new SparkException("Task not serializable", ex)
9 }
10 }

```

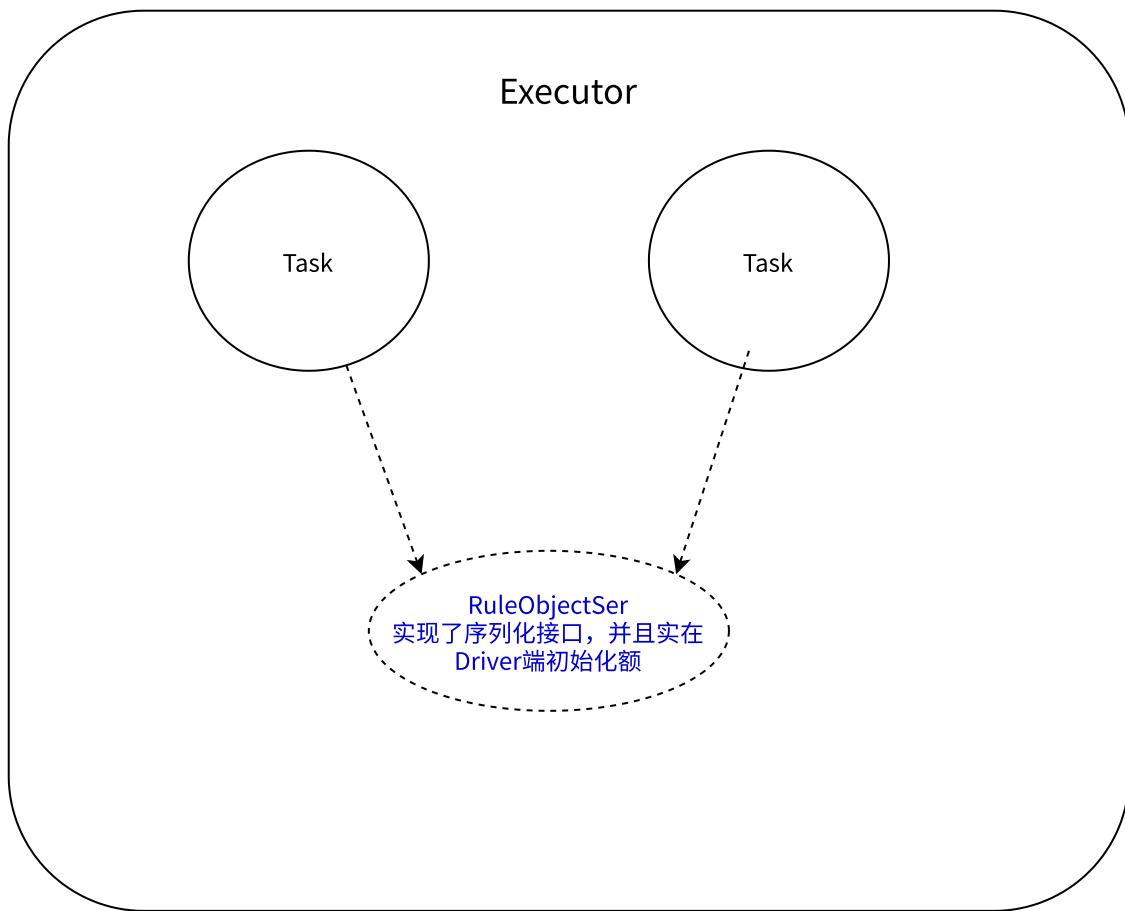
11.3.2 在Driver端初始化实现序列化的object

在一个Executor中，多个Task使用同一个object对象，因为在scala中，object就是单例对象，一个Executor中只有一个实例，Task会反序列化多次，但是引用的单例对象只反序列化一次

```

1  //从HDFS中读取数据，创建RDD
2  //HDFS指定的目录中有4个小文件，内容如下：
3  //1,ln
4  val lines = sc.textFile(args(1))
5  //函数外部定义的一个引用类型（变量）
6  //RuleObjectSer是一个静态对象，实在第一次使用的时候被初始化了（实在Driver被初始化的）
7  val rulesObj = RuleObjectSer
8
9  //函数实在Driver定义的
10 val func = (line: String) => {
11     val fields = line.split(",")
12     val id = fields(0).toInt
13     val code = fields(1)
14     val name = rulesObj.rulesMap.getOrElse(code, "未知") //闭包
15     //获取当前线程ID
16     val treadId = Thread.currentThread().getId
17     //获取当前Task对应的分区编号
18     val partitionId = TaskContext.getPartitionId()
19     //获取当前Task运行时的所在机器的主机名
20     val host = InetAddress.getLocalHost.getHostName
21     (id, code, name, treadId, partitionId, host, rulesObj.toString)
22 }
23
24 //处理数据，关联维度
25 val res = lines.map(func)
26 res.saveAsTextFile(args(2))

```



11.3.3 在Driver端初始化实现序列化的class

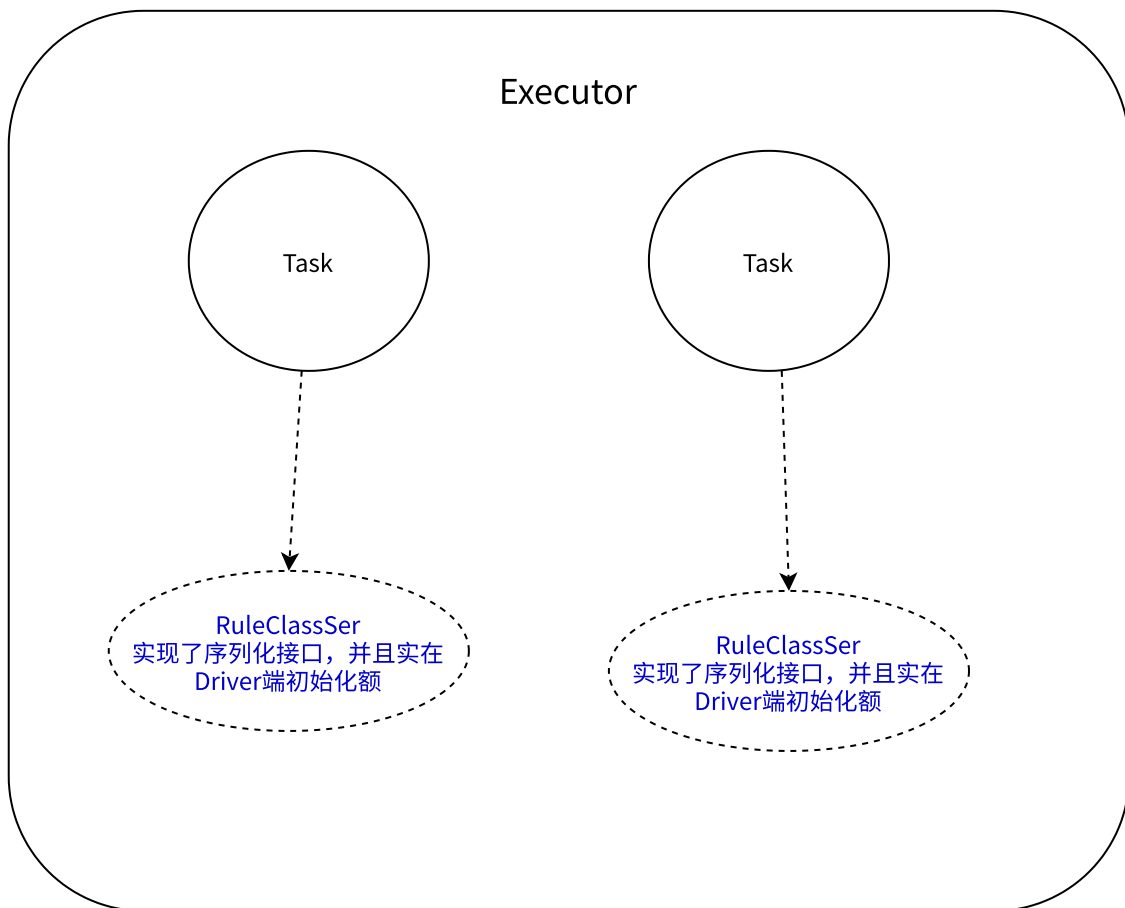
在一个Executor中，每个Task都会使用自己独享的class实例，因为在scala中，class就是多例，Task会反序列化多次，每个Task引用的class实例也会被序列化

```
1 //从HDFS中读取数据，创建RDD
2 //HDFS指定的目录中有4个小文件，内容如下：
3 //1,ln
4 val lines = sc.textFile(args(1))
5 //函数外部定义的一个引用类型（变量）
6 //RuleClassNotSer是一个类，需要new才能实现（实在Driver被初始化的）
7 val rulesClass = new RuleClassSer
8
9 //处理数据，关联维度
10 val res = lines.map(e => {
11   val fields = e.split(",")
12   val id = fields(0).toInt
13   val code = fields(1)
14   val name = rulesClass.rulesMap.getOrElse(code, "未知") //闭包
15   //获取当前线程ID
```

```

16  val treadId = Thread.currentThread().getId
17  //获取当前Task对应的分区编号
18  val partitionId = TaskContext.getPartitionId()
19  //获取当前Task运行时的所在机器的主机名
20  val host = InetAddress.getLocalHost.getHostName
21  (id, code, name, treadId, partitionId, host, rulesClass.toString)
22  })
23
24  res.saveAsTextFile(args(2))

```



11.3.4 在函数内部初始化未序列化的object

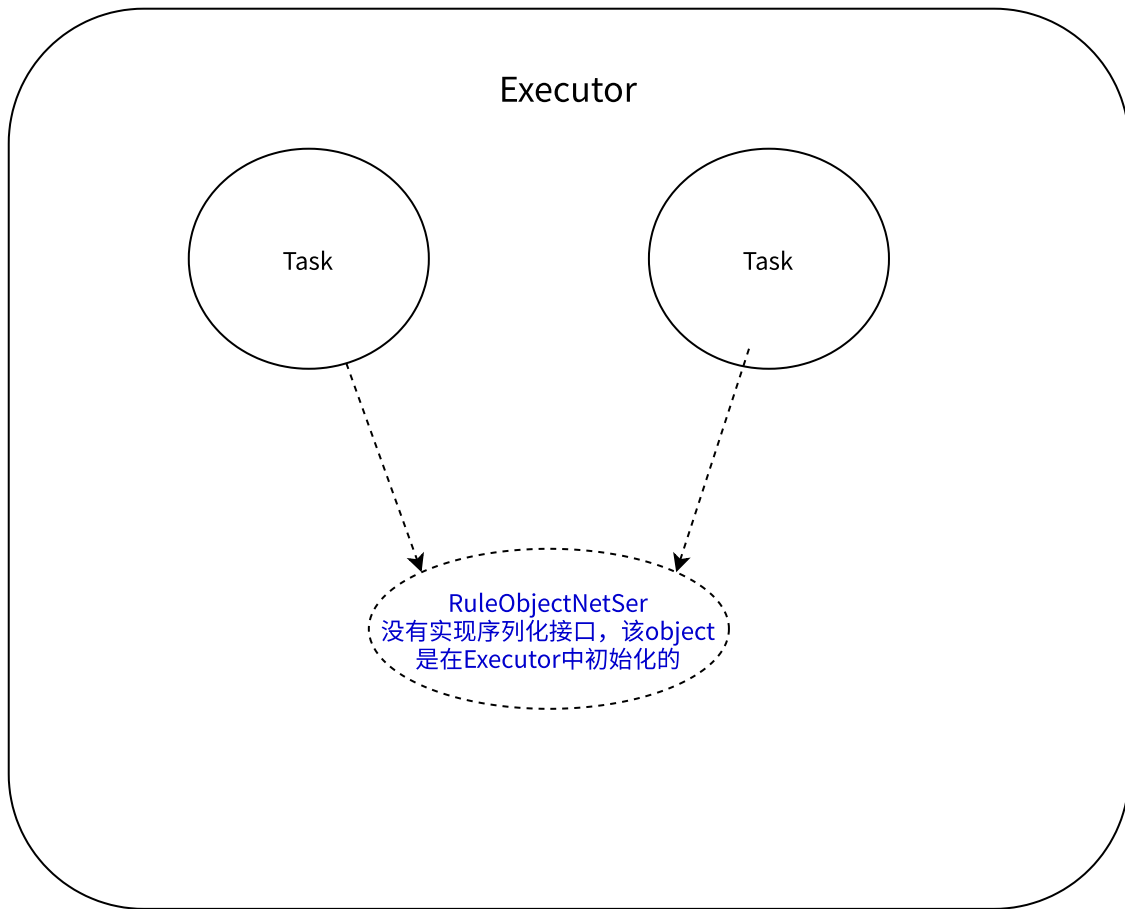
object没有实现序列化接口, 不会出现问题, 因为该object在函数内部被初始化的, 而不是在Driver初始化的

```

1  //从HDFS中读取数据, 创建RDD
2  //HDFS指定的目录中有4个小文件, 内容如下:
3  //1, ln

```

```
4 val lines = sc.textFile(args(1))
5 //不再Driver端初始化RuleObjectSer或RuleClassSer
6 //函数实在Driver定义的
7 val func = (line: String) => {
8     val fields = line.split(",")
9     val id = fields(0).toInt
10    val code = fields(1)
11    //在函数内部初始化没有实现序列化接口的RuleObjectNotSer
12    val name = RuleObjectNotSer.rulesMap.getOrElse(code, "未知")
13    //获取当前线程ID
14    val treadId = Thread.currentThread().getId
15    //获取当前Task对应的分区编号
16    val partitionId = TaskContext.getPartitionId()
17    //获取当前Task运行时的所在机器的主机名
18    val host = InetAddress.getLocalHost.getHostName
19    (id, code, name, treadId, partitionId, host, RuleObjectNotSer.toString)
20 }
21 //处理数据，关联维度
22 val res = lines.map(func)
23 res.saveAsTextFile(args(2))
24 sc.stop()
```



11.3.5 在函数内部初始化未序列化的class

这种方式非常不好，因为每来一条数据，new一个class的实例，会导致消耗更多资源，jvm会频繁GC

```
1 //从HDFS中读取数据，创建RDD
2 //HDFS指定的目录中有4个小文件，内容如下：
3 //1,ln
4 val lines = sc.textFile(args(1))
5
6 //处理数据，关联维度
7 val res = lines.map(e => {
8   val fields = e.split(",")
9   val id = fields(0).toInt
10  val code = fields(1)
11  //RuleClassNotSer是在Executor中被初始化的
12  val rulesClass = new RuleClassNotSer
13  //但是如果每来一条数据new一个RuleClassNotSer，不好，效率低，浪费资源，频繁GC
14  val name = rulesClass.rulesMap.getOrElse(code, "未知")
15  //获取当前线程ID
16  val treadId = Thread.currentThread().getId
17  //获取当前Task对应的分区编号
```

```

18  val partitionId = TaskContext.getPartitionId()
19  //获取当前Task运行时的所在机器的主机名
20  val host = InetAddress.getLocalHost.getHostName
21  (id, code, name, treadId, partitionId, host, rulesClass.toString)
22  })
23
24  res.saveAsTextFile(args(2))

```

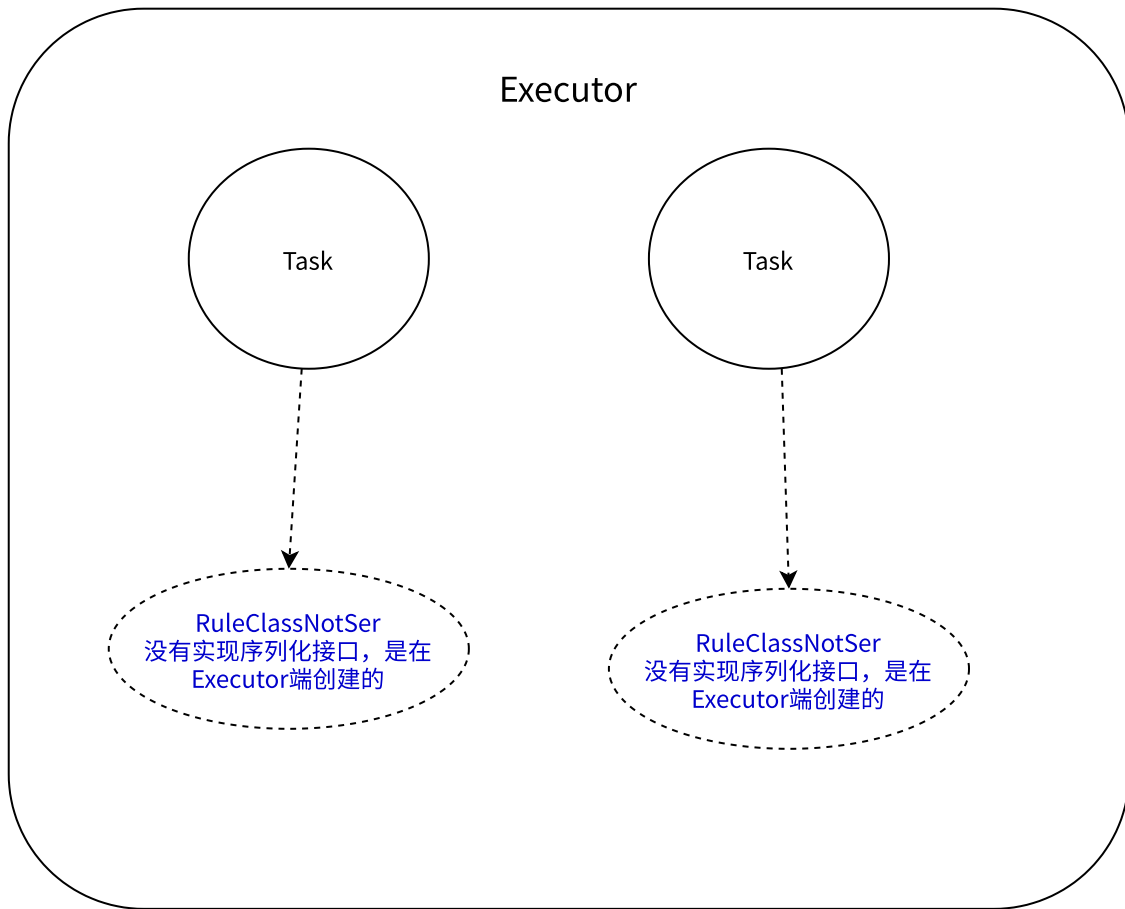
11.3.6 调用mapPartitions在函数内部初始化未序列化的class

一个分区使用一个class的实例，即每个Task都是自己的class实例

```

1  //从HDFS中读取数据，创建RDD
2  //HDFS指定的目录中有4个小文件，内容如下：
3  //1,1n
4  val lines = sc.textFile(args(1))
5  //处理数据，关联维度
6  val res = lines.mapPartitions(it => {
7    //RuleClassNotSer是在Executor中被初始化的
8    //一个分区的多条数据，使用同一个RuleClassNotSer实例
9    val rulesClass = new RuleClassNotSer
10   it.map(e => {
11     val fields = e.split(",")
12     val id = fields(0).toInt
13     val code = fields(1)
14     val name = rulesClass.rulesMap.getOrElse(code, "未知")
15     //获取当前线程ID
16     val treadId = Thread.currentThread().getId
17     //获取当前Task对应的分区编号
18     val partitionId = TaskContext.getPartitionId()
19     //获取当前Task运行时的所在机器的主机名
20     val host = InetAddress.getLocalHost.getHostName
21     (id, code, name, treadId, partitionId, host, rulesClass.toString)
22   })
23 })
24 res.saveAsTextFile(args(2))
25 sc.stop()

```



12. Task线程安全问题

12.1 现象和原理

在一个Executor可以同时运行多个Task，如果多个Task使用同一个共享的单例对象，如果对共享的数据同时进行读写操作，会导致线程不安全的问题，为了避免这个问题，可以加锁，但效率变低了，因为在一个Executor中同一个时间点只能有一个Task使用共享的数据，这样就变成了串行了，效率低！

12.2 案例

定义一个工具类object，格式化日期，因为SimpleDateFormat线程不安全，会出现异常

```
1 val conf = new SparkConf()
2   .setAppName("WordCount")
3   .setMaster("local[*]") //本地模式，开多个线程
4 //1. 创建SparkContext
5 val sc = new SparkContext(conf)
```

```

6
7 val lines = sc.textFile("data/date.txt")
8
9 val timeRDD: RDD[Long] = lines.map(e => {
10     //将字符串转成long类型时间戳
11     //使用自定义的object工具类
12     val time: Long = DateUtilObj.parse(e)
13     time
14 })
15
16 val res = timeRDD.collect()
17 println(res.toBuffer)

```

```

1 object DateUtilObj {
2
3     //多个Task使用了一个共享的SimpleDateFormat, SimpleDateFormat是线程不安全
4
5     val sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss")
6
7     //线程安全的
8     //val sdf: FastDateFormat = FastDateFormat.getInstance("yyyy-MM-dd HH:mm:ss")
9
10    def parse(str: String): Long = {
11        //2022-05-23 11:39:30
12        sdf.parse(str).getTime
13    }
14
15 }

```

上面的程序会出现错误，因为多个Task同时使用一个单例对象格式化日期，报错，如果加锁，程序会变慢，改进后的代码：

```

1 val conf = new SparkConf()
2     .setAppName("WordCount")
3     .setMaster("local[*]") //本地模式，开多个线程
4 //1.创建SparkContext
5 val sc = new SparkContext(conf)
6
7 val lines = sc.textFile("data/date.txt")
8
9 val timeRDD = lines.mapPartitions(it => {
10     //一个Task使用自己单独的DateUtilClass实例，缺点是浪费内存资源
11     val dataUtil = new DateUtilClass

```

```

12     it.map(e => {
13         dataUtil.parse(e)
14     })
15 })
16
17 val res = timeRDD.collect()
18 println(res.toBuffer)

```

```

1 class DateUtilClass {
2
3     val sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss")
4
5     def parse(str: String): Long = {
6         //2022-05-23 11:39:30
7         sdf.parse(str).getTime
8     }
9 }

```

改进后，一个Task使用一个DateUtilClass实例，不会出现线程安全的问题。

13. 累加器

累加器是Spark中用来做计数功能的，在程序运行过程当中，可以做一些额外的数据指标统计

需求：在处理数据的同时，统计一下指标数据，具体的需求为：将RDD中对应的每个元素乘以10，同时在统计每个分区中偶数的数据

13.1 不使用累加器的方案

需要多次触发Action，效率低，数据会被重复计算

```

1 /**
2  * 不使用累加器，而是触发两次Action
3  */
4 object C12_AccumulatorDemo1 {
5
6     def main(args: Array[String]): Unit = {
7
8         val conf = new SparkConf()
9             .setAppName("WordCount")

```

```

10     .setMaster("local[*]") //本地模式，开多个线程
11     //1.创建SparkContext
12     val sc = new SparkContext(conf)
13
14     val rdd1 = sc.parallelize(List(1,2,3,4,5,6,7,8,9), 2)
15     //对数据进行转换操作（将每个元素乘以10），同时还要统计每个分区的偶数的数量
16     val rdd2 = rdd1.map(_ * 10)
17     //第一次触发Action
18     rdd2.saveAsTextFile("out/111")
19
20     //附加的指标统计
21     val rdd3 = rdd1.filter(_ % 2 == 0)
22     //第二个触发Action
23     val c = rdd3.count()
24     println(c)
25 }
26 }

```

13.2 使用累加器的方法

触发一次Action，并且将附带的统计指标计算出来，可以使用Accumulator进行处理，Accumulator的本质是一个实现序列化接口class，每个Task都有自己的累加器，避免累加的数据发送冲突

```

1 object C14_AccumulatorDemo3 {
2
3     def main(args: Array[String]): Unit = {
4
5         val conf = new SparkConf()
6             .setAppName("WordCount")
7             .setMaster("local[*]") //本地模式，开多个线程
8         //1.创建SparkContext
9         val sc = new SparkContext(conf)
10
11         val rdd1 = sc.parallelize(List(1,2,3,4,5,6,7,8,9), 2)
12         //在Driver定义一个特殊的变量，即累加器
13         //Accumulator可以将每个分区的计数结果，通过网络传输到Driver，然后进行全局求和
14         val accumulator: LongAccumulator = sc.longAccumulator("even-acc")
15         val rdd2 = rdd1.map(e => {
16             if (e % 2 == 0) {
17                 accumulator.add(1) //闭包，在Executor中累计的
18             }
19             e * 10
20         })

```

```

21
22     //就触发一次Action
23     rdd2.saveAsTextFile("out/113")
24
25     //每个Task中累计的数据会返回到Driver吗?
26     println(accumulator.count)
27 }
28 }

```

14. StandAlone的两种执行模式

spark自动的StandAlone集群有两种运行方式，分别是client模式和cluster模式，默认使用的是client模式。两种运行模式的本质区别是，Driver运行在哪里了

14.1 什么是Driver

Driver本意是驱动的意思（类似叫法的有MySQL的连接驱动），在就是与集群中的服务建立连接，执行一些命令和请求的。但是在Spark的Driver指定就是SparkContext和里面创建的一些对象，所有可以总结为，SparkContext在哪里创建，Driver就在哪里。Driver中包含很多的对象实例，有SparkContext，DAGScheduler、TaskScheduler、ShuffleManager、BroadCastManager等，Driver是对这些对象的统称。

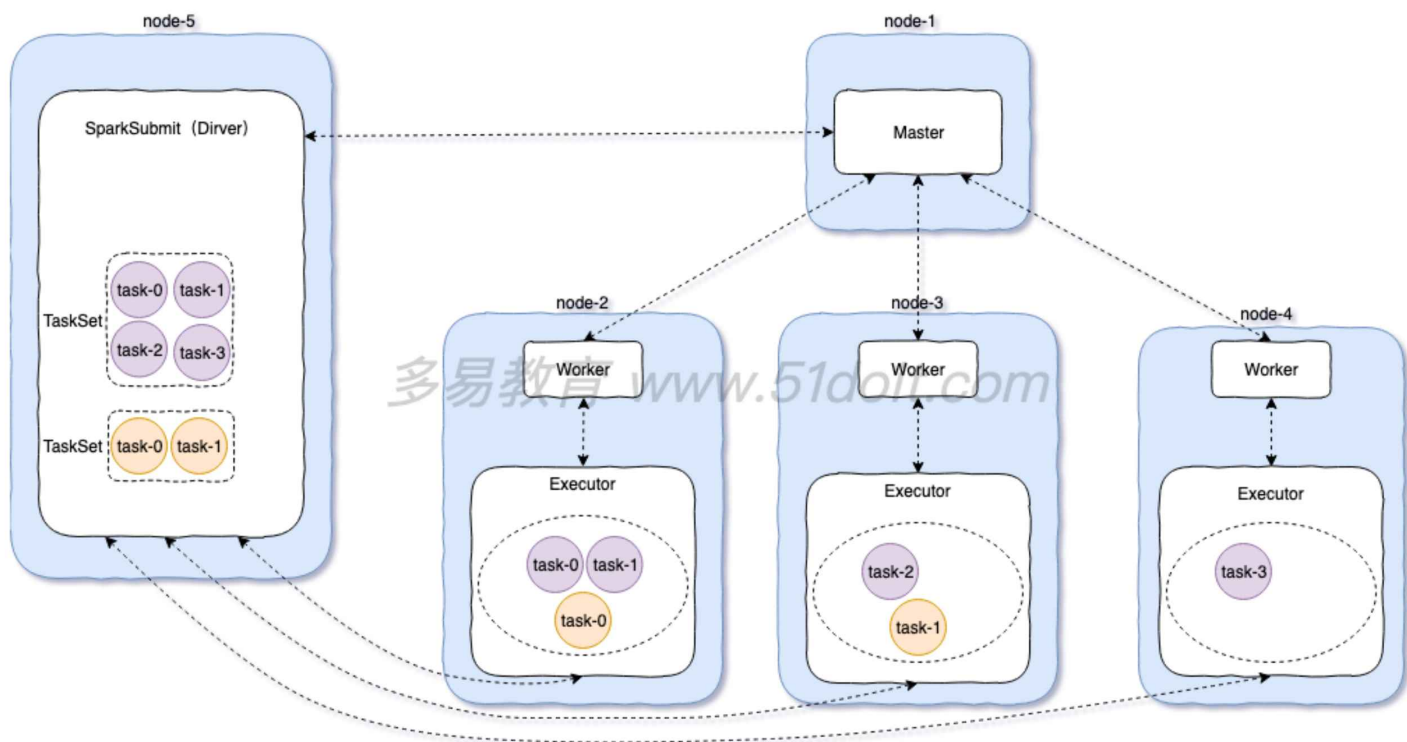
14.2 client模式

Driver运行在用来提交任务的SparkSubmit进程中，在Spark的stand alone集群中，提交spark任务时，可以使用cluster模式即`--deploy-mode cluster`（默认的）

```

--master MASTER_URL      spark://host:port, mesos://host:port, yarn,
                           k8s://https://host:port, or local (Default: local[*])
--deploy-mode DEPLOY_MODE Whether to launch the driver program locally ("client") or
                           on one of the worker machines inside the cluster ("cluster")
                           (Default: client).
--class CLASS_NAME        Your application's main class (for java / Scala apps)

```



注意：spark-shell只能以client模式运行，不能以cluster模式运行，因为提交任务的命令行客户端和SparkContext必须在同一个进程中。

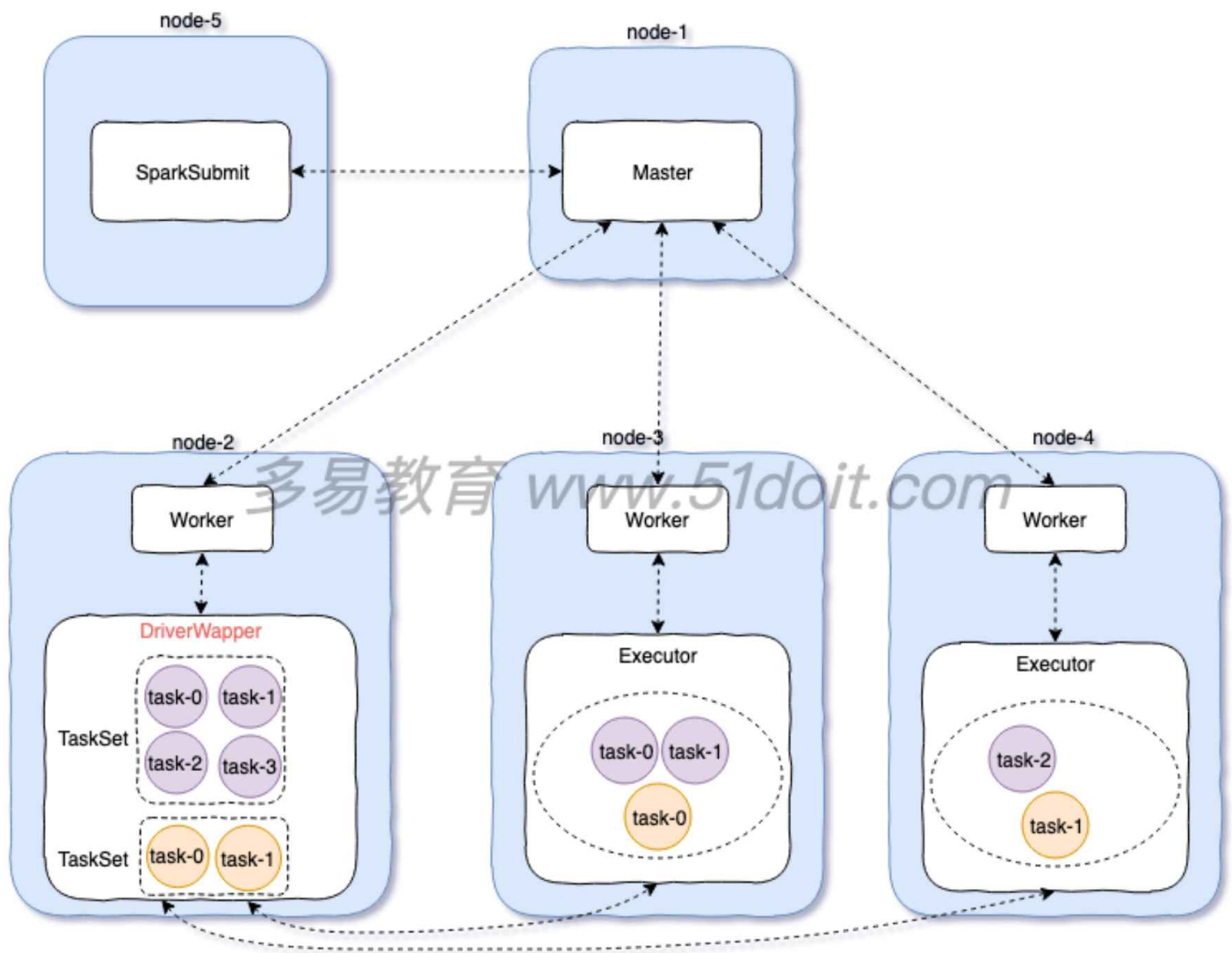
```
[root@node-1 ~]# /bigdata/spark-3.2.3-bin-hadoop3.2/bin/spark-shell --deploy-mode cluster --master spark://node-1.51doit.cn:7077 --executor-memory 1g --total-executor-cores 2
Exception in thread "main" org.apache.spark.SparkException: Cluster deploy mode is not applicable to Spark shells.
    at org.apache.spark.deploy.SparkSubmit.error(SparkSubmit.scala:972)
    at org.apache.spark.deploy.SparkSubmit.prepareSubmitEnvironment(SparkSubmit.scala:281)
    at org.apache.spark.deploy.SparkSubmit.org$apache$spark$deploy$SparkSubmit$$runMain(SparkSubmit.scala:898)
    at org.apache.spark.deploy.SparkSubmit.doRunMain$1(SparkSubmit.scala:180)
    at org.apache.spark.deploy.SparkSubmit.submit(SparkSubmit.scala:203)
    at org.apache.spark.deploy.SparkSubmit.doSubmit(SparkSubmit.scala:90)
    at org.apache.spark.deploy.SparkSubmit$$anon$2.doSubmit(SparkSubmit.scala:1043)
    at org.apache.spark.deploy.SparkSubmit$.main(SparkSubmit.scala:1052)
    at org.apache.spark.deploy.SparkSubmit.main(SparkSubmit.scala)
[root@node-1 ~]#
```

14.3 cluster模式

Driver运行在Worker启动的一个进程中，这个进程叫DriverWrapper，在Spark的stand alone集群中，提交spark任务时，可以使用cluster模式即--deploy-mode cluster

特点：Driver运行在集群中，不在SparkSubmit进程中，需要将jar包上传到hdfs中

```
1 spark-submit --master spark://node-1.51doit.cn:7077 --class
  cn._51doit.spark.day01.WordCount --deploy-mode cluster hdfs://node-
  1.51doit.cn:9000/jars/spark10-1.0-SNAPSHOT.jar hdfs://node-1.51doit.cn:9000/wc
  hdfs://node-1.51doit.cn:9000/out002
```



cluster模式的特点：可以给Driver更灵活的指定一些参数，可以给Driver指定内存大小，cores的数量。如果一些运算要在Driver进行计算，或者将数据收集到Driver端，这样就必须指定Driver的内存和cores更大一些。

- | | | |
|---|-----------------------|--|
| 1 | # 指定Driver的内存，默认是1g | |
| 2 | --driver-memory MEM | Memory for driver (e.g. 1000M, 2G) (Default: 1024M). |
| 3 | # 指定Driver的cores，默认是1 | |
| 4 | --driver-cores NUM | Number of cores used by the driver, only in cluster |

15. SparkOnYarn

15.1 Hadoop YARN回顾

15.1.1 YARN 的基本设计思想

将Hadoop 1.0中JobTracker拆分成两个独立的服务,一个全局的资源管理器ResourceManager(RM)和每个应用独有的ApplicationMaster(AM).其中RM负责整个系统的资源管理和分配,而AM负责单个的应用程序的管理

15.1.2 YARN的基本组成

- ResourceManager(RM)

全局的资源管理器,负责整个系统的资源管理和分配,由调度器(ResourceScheduler)和应用管理器(Application S Manger,ASM)组成:

1 调度器(ResourceScheduler)

调度器根据容量,队列等限制条件,将系统中的资源分配给各个正在运行的应用程序.调度器不参与任何应用程序的具体工作,仅根据应用程序的资源需求进行资源分配.调度器是个可拔插的组件,用户可根据自己的需要设计新的调度器.

2 应用程序管理器(ASM)

应用程序管理器负责整个系统中所有应用程序,包括应用程序的提交,与调度器协商资源以启动ApplicationMaster(AM),监控AM运行状态并在失败时重启它

- ApplicationMaster(AM)

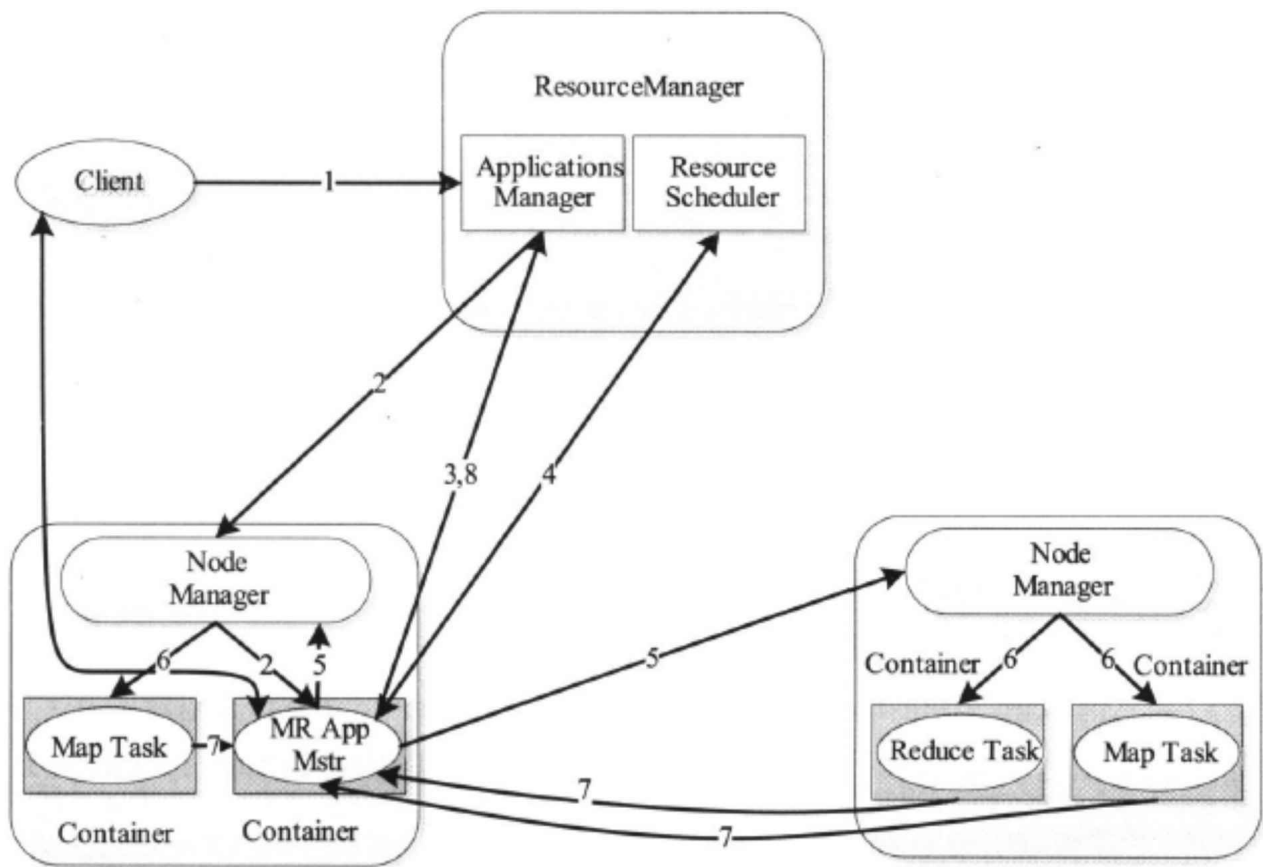
用户提交的每个应用程序均包含一个AM,主要功能包括:

- 1.与RM调度器协商以获取资源
- 2.将得到的资源进一步分配给内部的任务
- 3.与NodeManager(NM)通信,以启动\停止任务
- 4.监控所有任务运行状态,并在任务运行失败时重新为任务申请资源以重启任务

- NodeManager(NM)

NM是每个节点上的资源和任务管理器.一方面,它会定时的向RM汇报本节点上的资源使用情况和各个Container的运行状态;另一方面,它接收并处理来自AM的Container启动\停止等请求Container

15.1.3 YARN的运行流程



- ①用户向YARN提交应用程序,ResourceManager会返回一个applicationID, client将jar上传到HDFS
- ②RM(其中的调度器)为该应用程序分配第一个Container,(ASM)与对应的NM通信,要求它在这个Container中启动应用程序的AM
- ③AM首先向RM(其中的ASM)注册,这样用户可以直接通过RM查看应用程序的运行状况,然后AM会为各个任务申请资源,并监控任务的运行状态直至任务完成,运行结束.在任务未完成时,4-7步是会循环运行的
- ④AM采用轮询的方式通过RPC协议向RM(其中的调度器)申请和领取资源
- ⑤AM申请到资源后与对应NM通信,要求启动任务
- ⑥NM为任务设置好运行环境后,将任务启动命令写到一个脚本中,并通过运行该脚本启动任务.
- ⑦各个任务通过RPC协议向AM汇报自己的状态和进度,让AM随时掌握各个任务的运行状态,从而可以在任务失败时重新启动任务, 在应用程序运行过程中,用户可随时通过RPC向AM查询应用程序的当前状况
- ⑧应用程序运行完成后,AM向RM注销并关闭自己

15.2 SparkOnYarn准备工作

1. 需要在/etc/profile中配置HADOOP_CONF_DIR的目录，目的是为了让Spark找到core-site.xml、hdfs-site.xml和yarn-site.xml【让spark知道NameNode、ResourceManager】，不然会包如下错误：**Exception in thread "main" java.lang.Exception: When running with master 'yarn' either HADOOP_CONF_DIR or YARN_CONF_DIR must be set in the environment.**

修改/etc/profile

```
1 export JAVA_HOME=/usr/local/jdk1.8.0_192
2 export HADOOP_CONF_DIR=/bigdata/hadoop-3.2.1/etc/hadoop/
```

2. 关闭内存资源检测（生成环境不用关）

修改yarn-site.xml

```
1 <property>
2 <name>yarn.nodemanager.pmem-check-enabled</name>
3 <value>>false</value>
4 </property>
5
6 <property>
7 <name>yarn.nodemanager.vmem-check-enabled</name>
8 <value>>false</value>
9 </property>
```

参数说明：

yarn.nodemanager.pmem-check-enabled

是否启动一个线程检查每个任务正使用的物理内存量，如果任务超出分配值，则直接将其杀掉，默认是true。

yarn.nodemanager.vmem-check-enabled

是否启动一个线程检查每个任务正使用的虚拟内存量，如果任务超出分配值，则直接将其杀掉，默认是true

3. 配置一个yarn的container可以使用多个vcores，因为capacity schedule使用的是DefaultResourceCalculator，那么DefaultResourceCalculator它在加载Container时其实仅仅只会考虑内存而不考虑vcores，默认vcore就是1。yarn 默认情况下，只根据内存调度资源，所以spark on yarn 运行的时候，即使通过--executor-cores 指定 core 个数为 N，但是在 yarn 的资源管理页面上看到使用的 vcore 个数还是 1

修改capacity-scheduler.xml

```
1 <property>
2     <name>yarn.scheduler.capacity.resource-calculator</name>
3     <!--
4         <value>org.apache.hadoop.yarn.util.resource.DefaultResourceCalculator</value>
5     -->
6     <value>org.apache.hadoop.yarn.util.resource.DominantResourceCalculator</value>
7 </property>
```

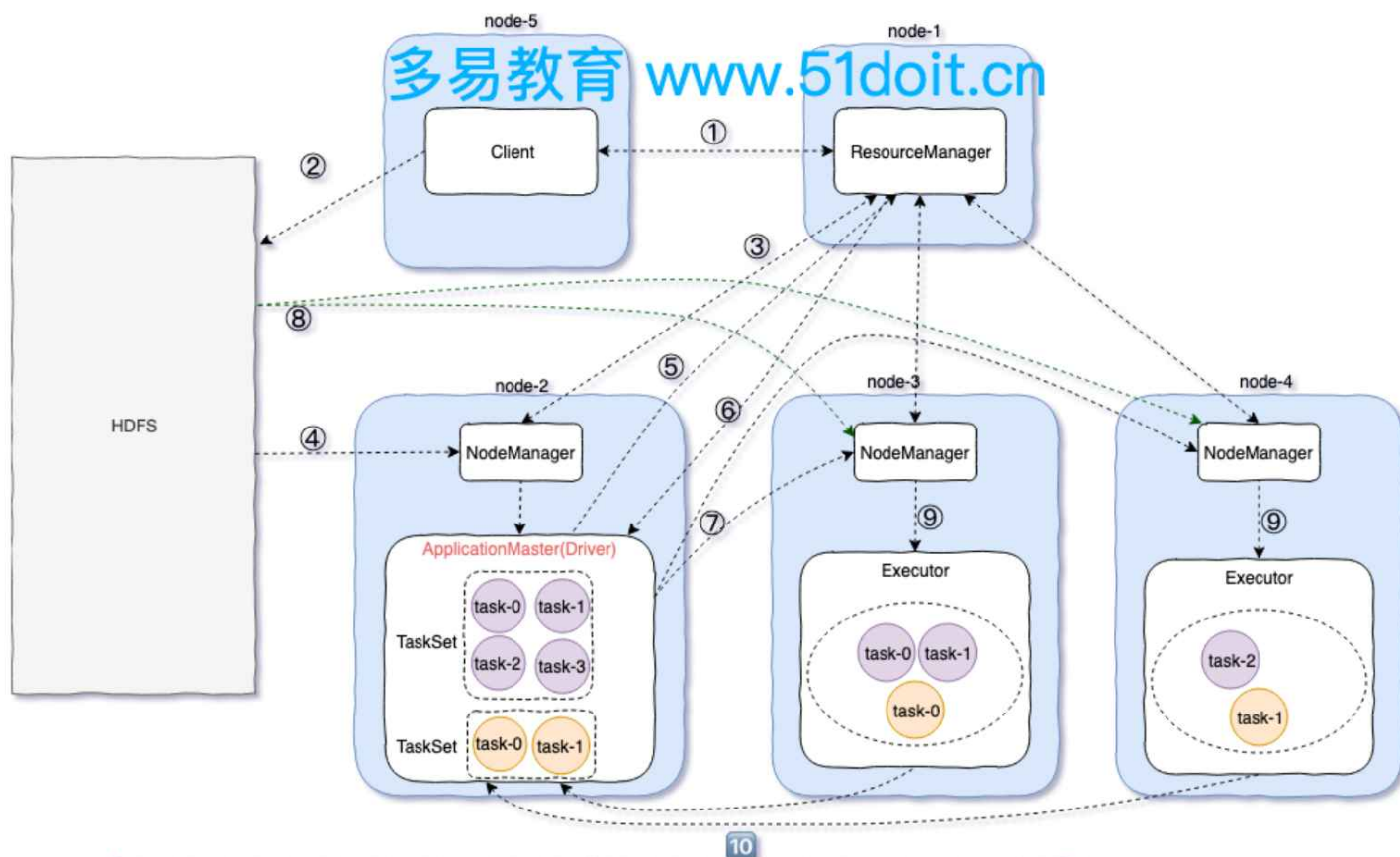
4. 重新分发到yarn中的各个节点：

```
1 for i in 2 3; do scp /etc/profile node- $\$i$ .51doit.cn:/etc/ ; done
2 for i in 2 3; do scp capacity-scheduler.xml yarn-site.xml node- $\$i$ .51doit.cn:$PWD
```

然后，启动hdfs和yarn集群。注意：要保证yarn集群的各个节点的时间是同步的。否则会报错

15.3 cluster模式

```
1 bin/spark-submit --master yarn --deploy-mode cluster --executor-memory 1g --
  executor-cores 2 --num-executors 3 --class cn._51doit.spark.day01.WordCount
  /root/spark-in-action.jar hdfs://node-1.51doit.cn:9000/wc hdfs://node-
  1.51doit.cn:9000/out01
```



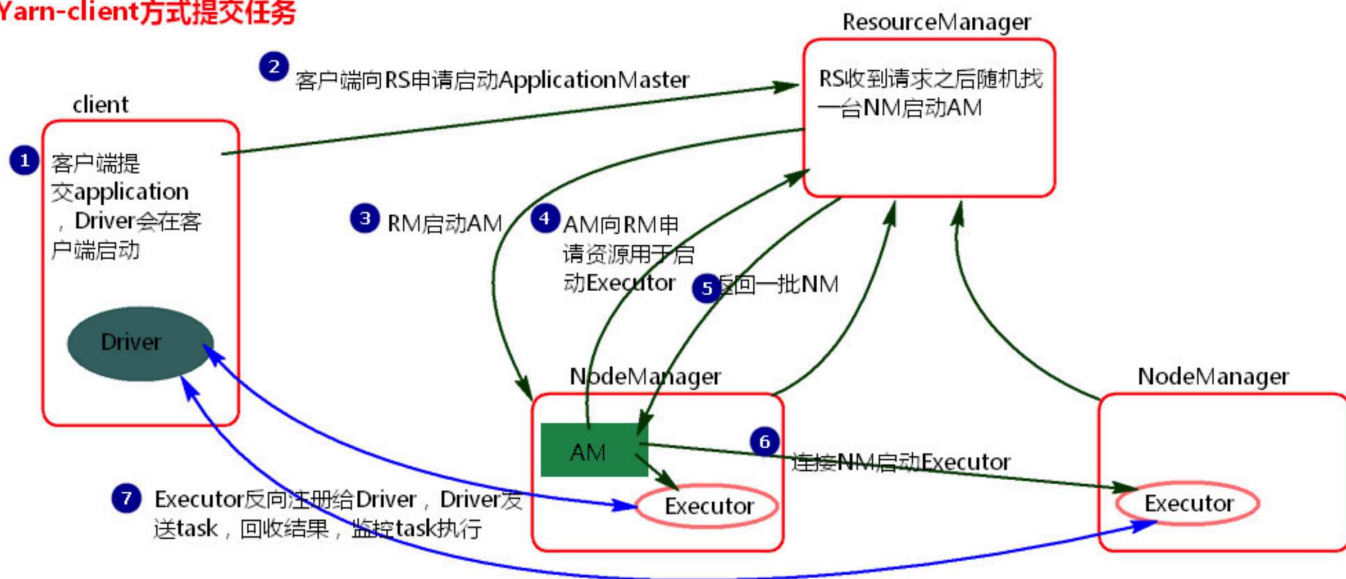
Spark On Yarn Cluster模式执行流程

- ① client向ResourceManager申请资源，返回一个application ID
- ② client上传spark jars 下面的jar包、自己写的jar和配置
- ③ ResourceManager随机找一个资源充足的NodeManager
- ④ 然后通过RPC让NodeManager从HDFS上下载jar包和配置，启动ApplicationMaster
- ⑤ ApplicationMaster向ResourceManager申请资源
- ⑥ ResourceManager 中的 ResourceScheduler找到符合条件的NodeManager，将NodeManager的信息返回给ApplicationMaster
- ⑦ ApplicationMaster跟返回的NodeManager进行通信
- ⑧ NodeManager从HDFS下载下载依赖
- ⑨ NodeManager启动Executor
- ⑩ Executor启动之后反向向ApplicationMaster 【Driver】 注册

15.4 client模式

- 1 `bin/spark-submit --master yarn --deploy-mode client --executor-memory 1g --executor-cores 2 --num-executors 3 --class cn._51doit.spark.day01.WordCount /root/spark-in-action.jar hdfs://node-1.51doit.cn:9000/wc hdfs://node-1.51doit.cn:9000/out01`

Yarn-client方式提交任务



- ①客户端提交一个Application, 在客户端启动一个Driver进程。
- ②Driver进程会向ResourceManager发送请求, 启动ApplicationMaster的资源。
- ③ResourceManager收到请求, 随机选择一台NodeManager,然后该NodeManager到HDFS下载jar包和配置, 接着启动ApplicationMaster【ExecutorLuacher】。这里的NodeManager相当于Standalone中的Worker节点。
- ④ApplicationMaster启动后, 会向ResourceManager请求一批container资源, 用于启动Executor.
- ⑤ResourceManager会找到一批符合条件NodeManager返回给ApplicationMaster,用于启动Executor。
- ⑥ApplicationMaster会向NodeManager发送请求, NodeManager到HDFS下载jar包和配置, 然后启动Executor。
- ⑦Executor启动后, 会反向注册给Driver, Driver发送task到Executor,执行情况和结果返回给Driver端

15.5 资源分配

- YARN中可以虚拟VCORE, 虚拟cpu核数, 以后部署在物理机上, 建议配置YARN的VCORES的使用等于物理机的逻辑核数, 即物理机的核数和VCORES是一一对应的, 在YARN中为spark分配任务, spark的cores跟逻辑核数一一对应, 另一个逻辑和对应一个VCORE, 一个VCORES对应一个spark cores【官方建议spark的cores是逻辑核的2到3倍】
- yarn中的资源分配, 针对的是容器, 容器默认最少的资源是1024mb, 容器接受的资源, 必须是最小资源的整数倍。

- spark中分配的资源由两部分组成，**参数 + overhead**，例如 `--executor-memory 1g`，overhead 为

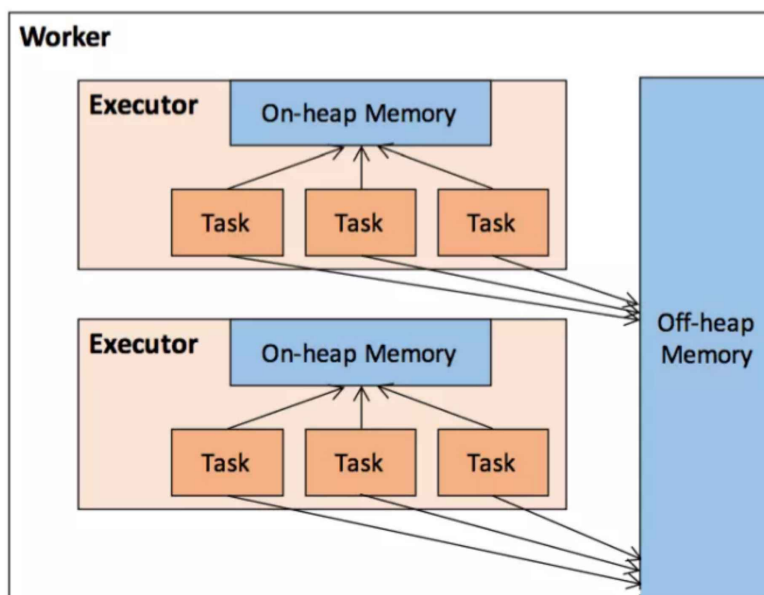
$\max(1024 * 0.1, 384)$ ，executor真正占用的资源应该是：1g + 384mb = 1408Mb

- 在yarn中，分配的资源最终都是分配给容器的1408 **向上取整**，例如`--executor-memory 2g`那么最终的内存为：**2048 + Max(2048*0.1, 384)**

16. Spark内存管理机制

16.1 概述

在执行Spark的应用程序时，Spark集群会启动Driver和Executor两种JVM进程，前者为主控进程，负责创建Spark上下文，提交Spark作业（Job），并将作业转化为计算任务（Task），在各个Executor进程间协调任务的调度，后者负责在工作节点上执行具体的计算任务，并将结果返回给Driver，同时为需要持久化的RDD提供存储功能。由于Driver的内存管理相对来说较为简单，本章节要对Executor的内存管理进行分析，下文中的Spark内存均特指Executor的内存。



16.2 堆内内存和堆外内存

作为一个JVM进程，Executor的内存管理建立在JVM的内存管理之上，Spark对JVM的堆内（On-heap）空间进行了更为详细的分配，以充分利用内存。同时，Spark引入了堆外（Off-heap）内存，使之可以直接在工作节点的系统内存中开辟空间，进一步优化了内存的使用。

16.3 堆内内存

堆内内存的大小，由Spark应用程序启动时的`-executor-memory`或`spark.executor.memory`参数配置。Executor内运行的并发任务共享JVM堆内内存，这些任务在缓存RDD和广播（Broadcast）数据时占用的内存被规划为存储（Storage）内存，而这些任务在执行Shuffle时占用的内存被规划为执行（Execution）内存，剩余的部分不做特殊规划，那些Spark内部的对象实例，或者用户定义的Spark应用程序中的对象实例，均占用剩余的空间。不同的管理模式，这三部分占用的空间大小各不相同（下面第2小节介绍）。

16.3.1 堆内内存的申请与释放

Spark对堆内内存的管理是一种逻辑上的“规划式”的管理，因为对象实例占用内存的申请和释放都由JVM完成，Spark只能在申请后和释放前记录这些内存：

- 申请内存

Spark在代码中new一个对象实例

JVM从堆内内存分配空间，创建对象并返回对象引用

Spark保存该对象的引用，记录该对象占用的内存

- 释放内存

Spark记录该对象释放的内存，删除该对象的引用

等待JVM的垃圾回收机制释放该对象占用的堆内内存

- 堆内内存优缺点分析

堆内内存采用JVM来进行管理。而JVM的对象可以以序列化的方式存储，序列化的过程是将对象转换为二进制字节流，本质上可以理解为将非连续空间的链式存储转化为连续空间或块存储，在访问时则需要进行序列化的逆过程——反序列化，将字节流转化为对象，序列化的方式可以节省存储空间，但增加了存储和读取时候的计算开销。

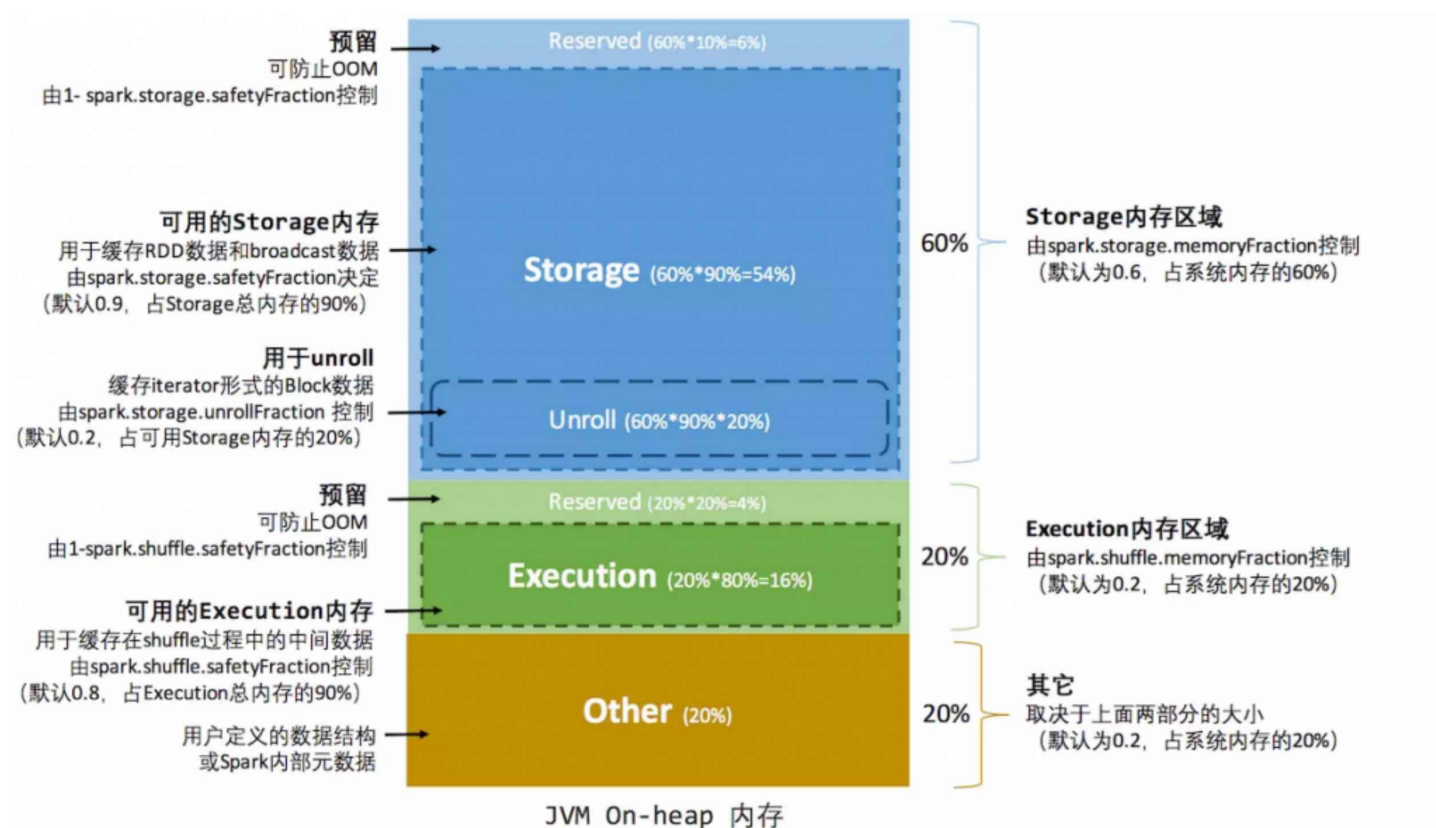
对于Spark中序列化的对象，由于是字节流的形式，其占用的内存大小可直接计算。

对于Spark中非序列化的对象，其占用的内存是通过周期性地采样近似估算而得，即并不是每次新增的数据项都会计算一次占用的内存大小。这种方法：

- 降低了时间开销但是有可能误差较大，导致某一时刻的实际内存有可能远远超出预期；此外，在被Spark标记为释放的对象实例，很有可能在实际上并没有被JVM回收，导致实际可用的内存小于Spark记录的可用内存。所以Spark并不能准确记录实际可用的堆内内存，从而也就无法完全避免内存溢出（OOM, Out of Memory）的异常。
- 虽然不能精准控制堆内内存的申请和释放，但Spark通过对存储内存和执行内存各自独立的规划管理，可以决定是否要在存储内存里缓存新的RDD，以及是否为新的任务分配执行内存，在一定程度上可以提升内存的利用率，减少异常的出现。

16.3.2 堆内内存分区(静态方式,弃)

在静态内存管理机制下，存储内存、执行内存和其他内存三部分的大小在Spark应用程序运行期间是固定的，但用户可以在应用程序启动前进行配置，堆内内存的分配如图所示：



可以看到，可用的堆内内存的大小需要按照下面的方式计算：

可用的存储内存 = $\text{systemMaxMemory} * \text{spark.storage.memoryFraction} * \text{spark.storage.safetyFraction}$

可用的执行内存 = $\text{systemMaxMemory} * \text{spark.shuffle.memoryFraction} * \text{spark.shuffle.safetyFraction}$

其中`systemMaxMemory`取决于当前JVM堆内内存的大小，最后可用的执行内存或者存储内存要在此基础上与各自的`memoryFraction`参数和`safetyFraction`参数相乘得出。上述计算公式中的两个

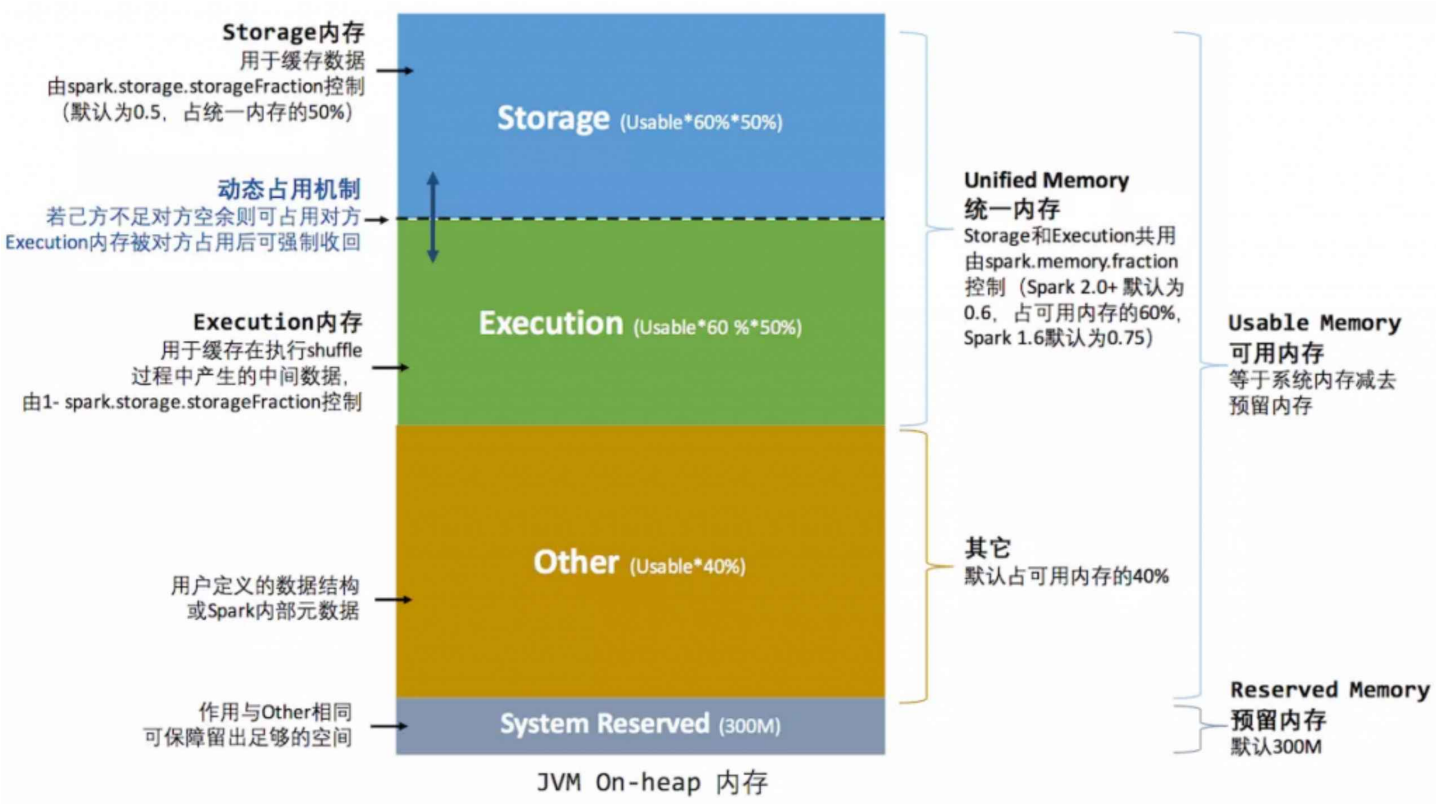
safetyFraction参数，其意义在于在逻辑上预留出1-safetyFraction这么一块保险区域，降低因实际内存超出当前预设范围而导致OOM的风险（上文提到，对于非序列化对象的内存采样估算会产生误差）。值得注意的是，这个预留的保险区域仅仅是一种逻辑上的规划，在具体使用时Spark并没有区别对待，和“其它内存”一样交给了JVM去管理。

16.3.3 堆内内存分区(统一方式,现)

默认情况下，Spark 仅仅使用了堆内内存。Executor 端的堆内内存区域大致可以分为以下四大块：

	A	B
1	分区	说明
2	Execution 内存	主要用于存放 Shuffle、Join、Sort、Aggregation 等计算过程中数据
3	Storage 内存	主要用于存储spark 的 cache 数据，例如RDD的缓存、unrolled RDD等
4	用户内存（User Memory）	主要用于存储RDD 转换操作所需要的数据，例如 RDD 依赖等值
5	预留内存（Reserved Memory）	系统预留内存，会用来存储Spark内部对象

整个 Executor 端堆内内存如果用图来表示的话，可以概括如下：



对上图进行以下说明：

- `systemMemory = Runtime.getRuntime.maxMemory`，其实就是通过参数 `spark.executor.memory` 或 `--executor-memory` 配置的。

- reservedMemory 在 Spark 2.2.1 中是写死的，其值等于 300MB，这个值是不能修改的（如果在测试环境下，我们可以通过 spark.testing.reservedMemory 参数进行修改）；
- usableMemory = systemMemory - reservedMemory，这个就是 Spark 可用内存；

16.4 堆外内存(Off-heap Memory)

为了进一步优化内存的使用以及提高Shuffle时排序的效率，Spark引入了堆外（Off-heap）内存，使之可以直接在工作节点的系统内存中开辟空间，存储经过序列化的二进制数据。除了没有other空间，堆外内存与堆内内存的划分方式相同，所有运行中的并发任务共享存储内存和执行内存。

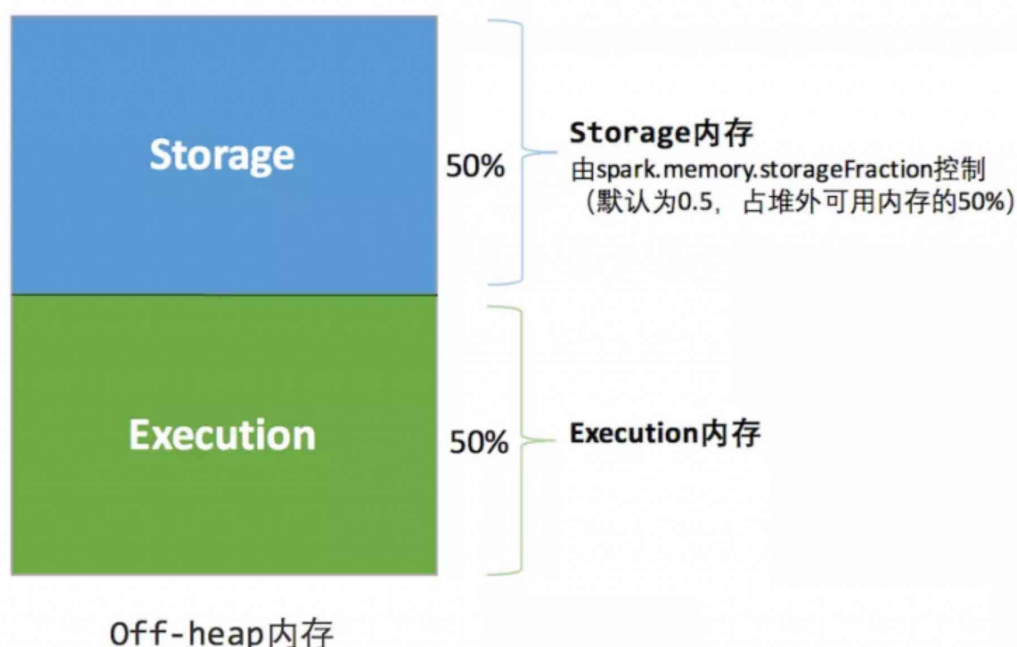
Spark 1.6 开始引入了Off-heap memory(详见SPARK-11389)。这种模式不在 JVM 内申请内存，而是调用 Java 的 unsafe 相关 API 进行诸如 C 语言里面的 malloc() 直接向操作系统申请内存。由于这种方式不经过 JVM 内存管理，所以可以避免频繁的 GC，这种内存申请的缺点是必须自己编写内存申请和释放的逻辑。

16.4.1 堆外内存的优缺点

利用JDK Unsafe API（从Spark 2.0开始，在管理堆外的存储内存时不再基于Tachyon（Alluxio），而是与堆外的执行内存一样，基于JDK Unsafe API实现[3]），Spark可以直接操作系统堆外内存，减少了不必要的内存开销，以及频繁的GC扫描和回收，提升了处理性能。堆外内存可以被精确地申请和释放，而且序列化的数据占用的空间可以被精确计算，所以相比堆内内存来说降低了管理的难度，也降低了误差。

16.4.2 堆外内存分区(静态方式,弃)

堆外的空间分配较为简单，存储内存、执行内存的大小同样是固定的

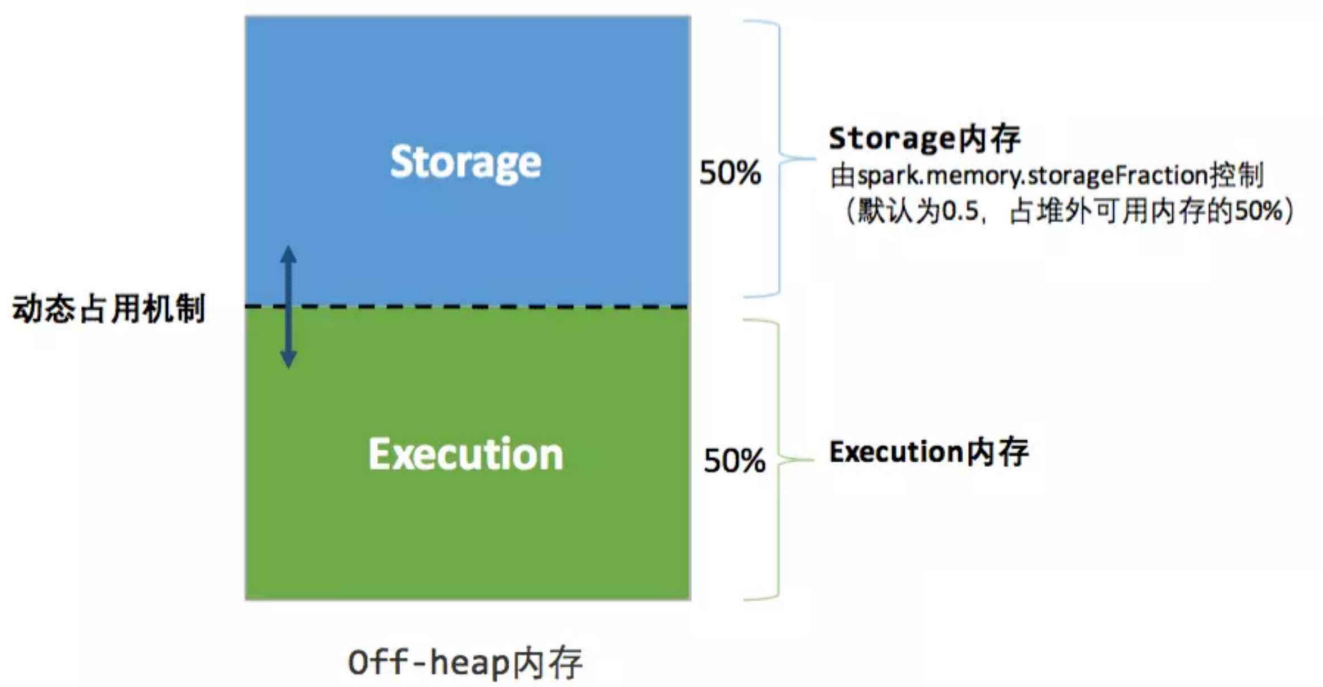


可用的执行内存和存储内存占用的空间大小直接由参数spark.memory.storageFraction决定，由于堆外内存占用的空间可以被精确计算，所以无需再设定保险区域。

静态内存管理机制实现起来较为简单，但如果用户不熟悉Spark的存储机制，或没有根据具体的数据规模和计算任务或做相应的配置，很容易造成“一半海水，一半火焰”的局面，即存储内存和执行内存中的一方剩余大量的空间，而另一方却早早被占满，不得不淘汰或移出旧的内容以存储新的内容。由于新的内存管理机制的出现，这种方式目前已经很少有开发者使用，出于兼容旧版本的应用程序的目的，Spark仍然保留了它的实现。

16.4.3 堆外内存分区(统一方式,现)

相比堆内内存，堆外内存只区分 Execution 内存和 Storage 内存，其内存分布如下图所示：



关于动态占用机制，由于统一内存管理方式中堆内堆外内存的管理均基于此机制，所以单独提出来讲解。参见文本第三节。

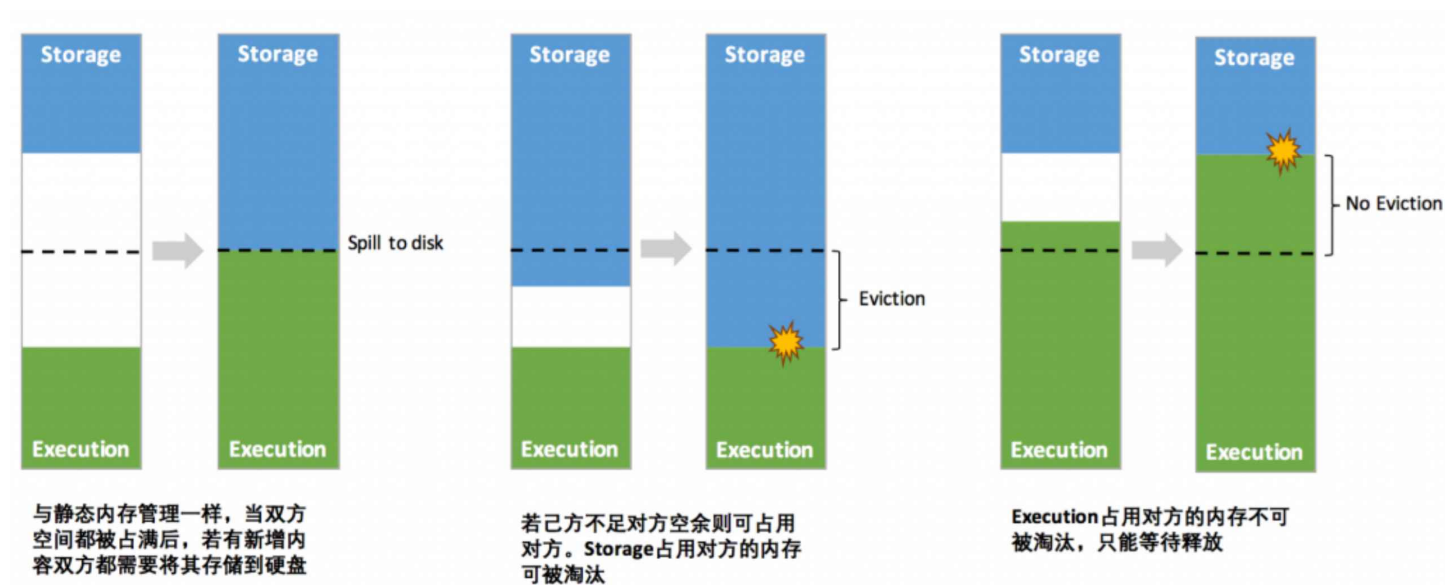
16.5 动态占用机制–Execution与Storage

上面两张图中的 Execution 内存和 Storage 内存之间存在一条虚线，这是为什么呢？

在 Spark 1.5 之前，Execution 内存和 Storage 内存分配是静态的，换句话说就是如果 Execution 内存不足，即使 Storage 内存有很大空闲程序也是无法利用到的；反之亦然。这就导致我们很难进行内存的调优工作，我们必须非常清楚地了解 Execution 和 Storage 两块区域的内存分布。

而目前 Execution 内存和 Storage 内存可以互相共享的。也就是说，如果 Execution 内存不足，而 Storage 内存有空闲，那么 Execution 可以从 Storage 中申请空间；反之亦然。所以上图中的虚线代

表 Execution 内存和 Storage 内存是可以随着运作动态调整的，这样可以有效地利用内存资源。Execution 内存和 Storage 内存之间的动态调整可以概括如下：



16.6 动态调整策略

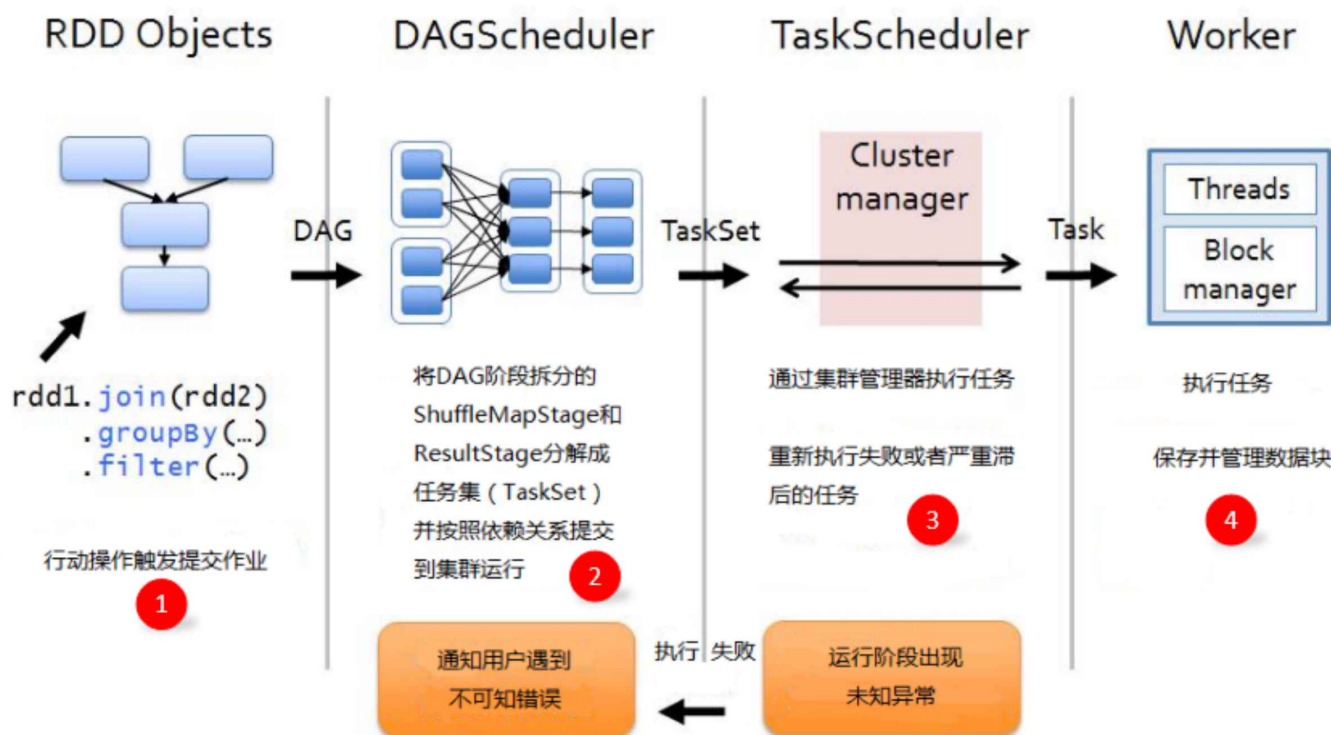
具体的实现逻辑如下：

- 程序提交的时候我们都会设定基本的 Execution 内存和 Storage 内存区域（通过 `spark.memory.storageFraction` 参数设置）；
- 在程序运行时，双方的空间都不足时，则存储到硬盘；将内存中的块存储到磁盘的策略是按照 LRU 规则(Least Recently Used)进行的。若己方空间不足而对方空余时，可借用对方的空间；（存储空间不足是指不足以放下一个完整的 Block）
- Execution 内存的空间被对方占用后，可让对方将占用的部分转存到硬盘，然后”归还”借用的空间
- Storage 内存的空间被对方占用后，目前的实现是无法让对方”归还”，因为需要考虑 Shuffle 过程中的很多因素，实现起来较为复杂；而且 Shuffle 过程产生的文件在后面一定会被使用到，而 Cache 在内存的数据不一定在后面使用。

注意，上面说的借用对方的内存需要借用方和被借用方的内存类型都一样，都是堆内内存或者都是堆外内存，不存在堆内内存不够去借用堆外内存的空间。

统一内存分配机制的优点是：提高了内存的利用率，可以更加灵活、可靠的分配和管理内存

17. Spark执行流程

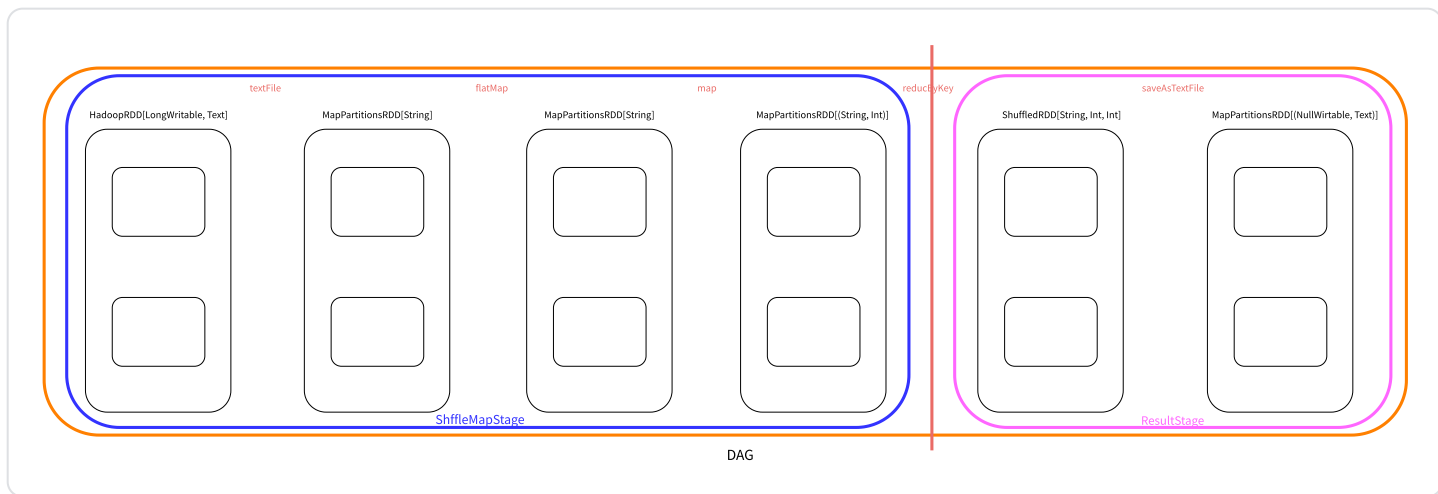


17.1 创建SparkContext

使用spark-submit脚本，会启动SparkSubmit进程，然后通过反射调用我们通过--class传入类的main方法，在main方法中，就行我们写的业务逻辑了，先创建SparkContext，向Master申请资源，然后Master跟Worker通信，启动Executor，然后所有的Executor向Driver反向注册

17.2 创建RDD并构建DAG

DAG(Directed Acyclic Graph)叫做有向无环图，是的一系列RDD转换关系的描述，原始的RDD通过一系列的转换也就形成了DAG，根据RDD之间的依赖关系的不同将DAG划分成不同的Stage，对于窄依赖，partition的转换处理在Stage中完成计算。对于宽依赖，由于有Shuffle的存在，只能在parent RDD处理完成后，才能开始接下来的计算，因此宽依赖是划分Stage的依据。



依赖关系划分为两种：窄依赖（Narrow Dependency）和宽依赖（源码中为Shuffle Dependency）。

窄依赖指的是父 RDD 中的一个分区最多只会被子RDD 中的一个分区使用，意味着父RDD的一个分区内的数据是不能被分割的，子RDD的任务可以跟父RDD在同一个Executor一起执行，不需要经过 Shuffle 阶段去重组数据。

窄依赖包括两种：一对一依赖（OneToOneDependency）和范围依赖（RangeDependency）

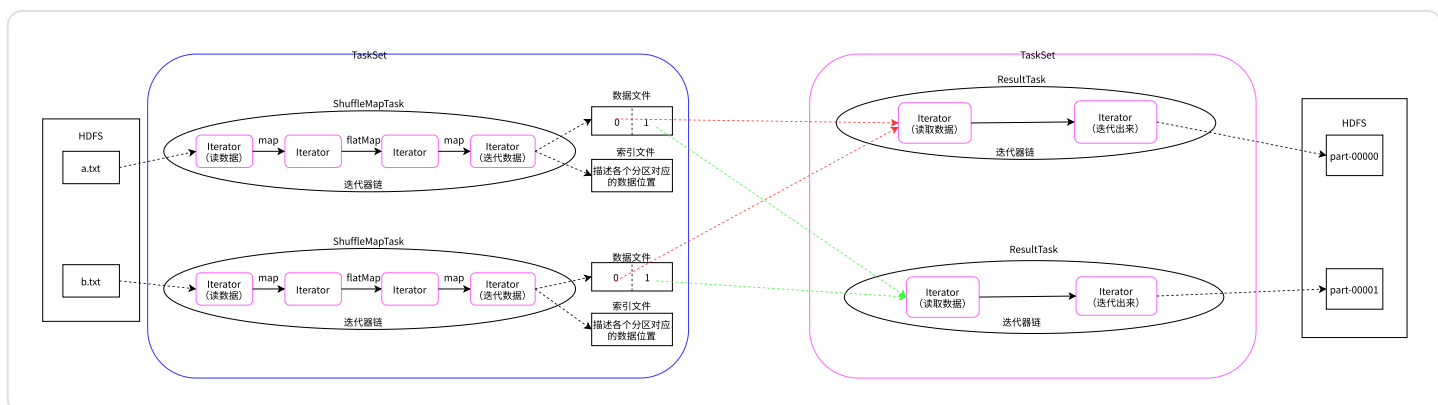
一对一依赖

宽依赖指的是父 RDD 中的分区可能会被多个子 RDD 分区使用。因为父 RDD 中一个分区内的数据会被分割，发送给子 RDD 的多个分区，因此宽依赖也意味着父 RDD 与子 RDD 之间存在着 Shuffle 过程。

宽依赖只有一种：Shuffle依赖（ShuffleDependency）

17.3 切分Stage，生成Task和TaskSet

触发Action，会根据最后一个RDD，从后往前推，如果是窄依赖（没有shuffle），继续往前推，如果是宽依赖（有shuffle），那么会递归进去，然后再根据递归进去的最后一个RDD进行向前推，如果一个RDD再也没有父RDD（递归出口），那么递归出来划分Stage（DAGScheduler完成的以上工作）

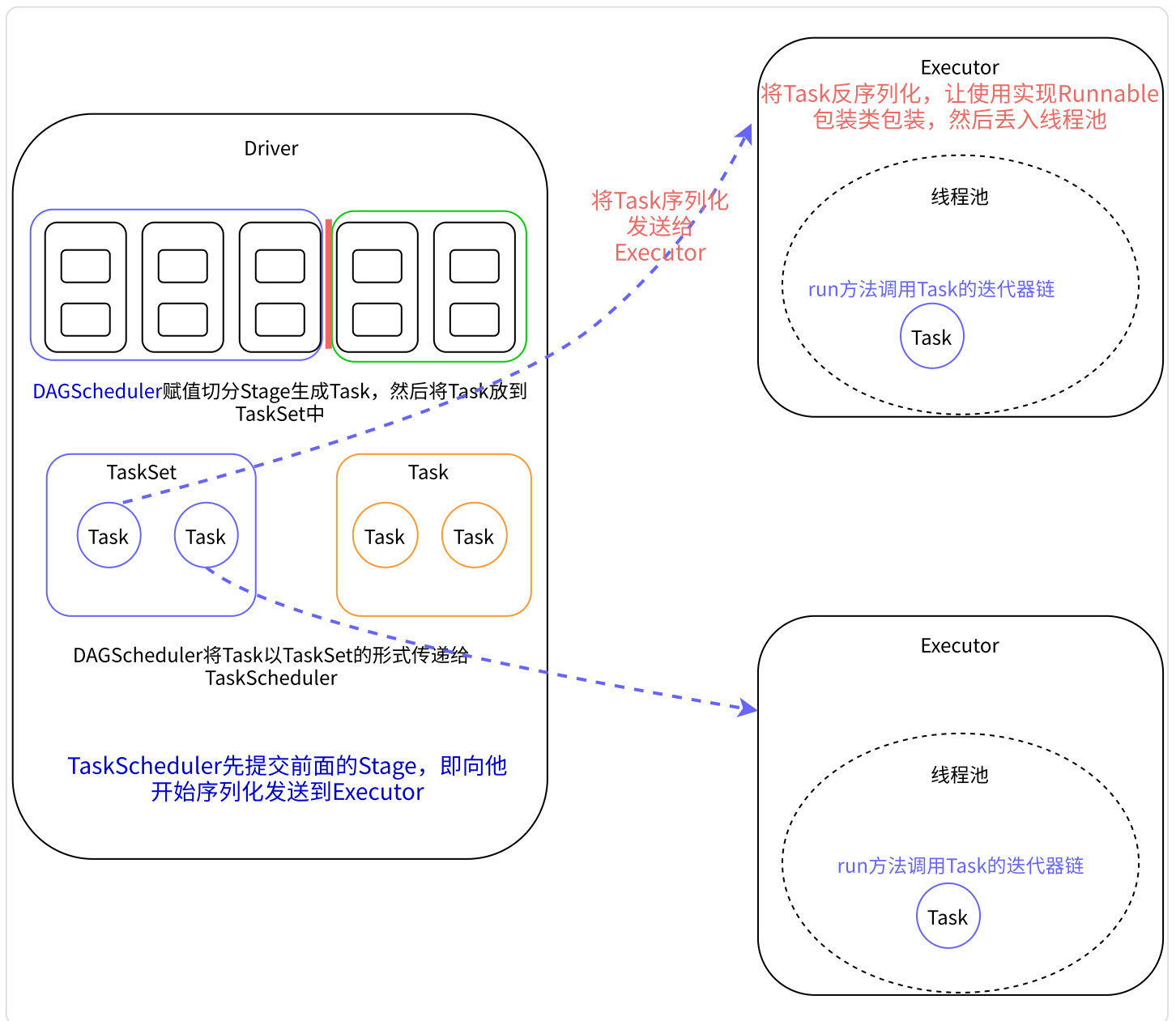


17.4 将Task调度到Executor

划分完Stage后，DAGScheduler将根据Stage的类型，生成Task，然后将同一个Stage的多个计算逻辑相同的Task放入到同一个TaskSet中，然后向DAGScheduler将TaskSet传递给TaskScheduler，TaskScheduler会根据Executor的资源情况，然后将Task序列化发送给Executor

17.5 在Executor中执行Task

Executor接收到TaskScheduler发送过来的Task后，将其反序列化，然后使用一个实现了Runnable接口的包装类进行包装，最后将包装的Task丢入到线程池，一旦丢入到线程池，run方法会执行，run方法会调用Task对应的迭代器链进行迭代数据



- Job: RDD每一个行动操作都会生成一个或者多个调度阶段 调度阶段 (Stage): 每个Job都会根据依赖关系, 以Shuffle过程作为划分, 分为Shuffle Map Stage和Result Stage。每个Stage对应一个TaskSet, 一个Task中包含多Task, TaskSet的数量与该阶段最后一个RDD的分区数相同。
- Task: 分发到Executor上的工作任务, 是Spark的最小执行单元
- DAGScheduler: DAGScheduler是将DAG根据宽依赖将切分Stage, 负责划分调度阶段并Stage转成TaskSet提交给TaskScheduler
- TaskScheduler: TaskScheduler是将Task序列化然后发送到Worker下的Exexcutor进程, 在Executor中, 将Task反序列化, 然后使用实现Runnable接口的包装类包装, 最后丢入到Executor的线程池的中进行执行

18. shuffle 过程详解

18.1 spark shuffle 演进的历史

Spark 0.8及以前 Hash Based Shuffle

Spark 0.8.1 为Hash Based Shuffle引入File Consolidation机制

Spark 0.9 引入ExternalAppendOnlyMap

Spark 1.1 引入Sort Based Shuffle，但默认仍为Hash Based Shuffle

Spark 1.2 默认的Shuffle方式改为Sort Based Shuffle

Spark 1.4 引入Tungsten-Sort Based Shuffle

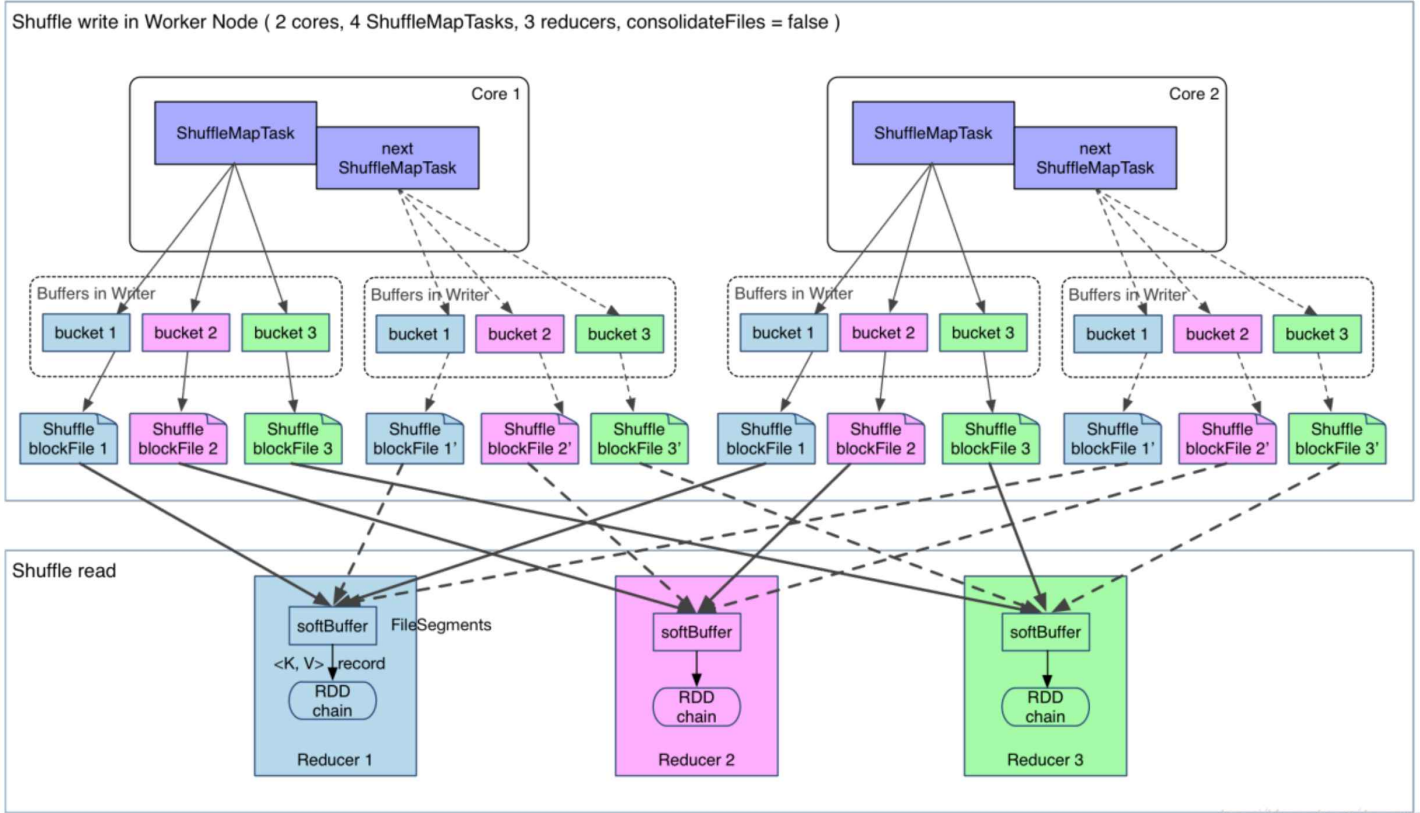
Spark 1.6 Tungsten-sort并入Sort Based Shuffle

Spark 2.0 Hash Based Shuffle退出历史舞台

18.2 HashShuffleManager（已不再使用）

- 优化前的

在shuffle write前，应用分区器，根据对应的分区规则，计算出数据partition编号，然后将数据写入bucket内存中，当数据达到一定大小或数据全部处理完后，将数据溢写持平持久化。之所以要持久化，一方面是要减少内存存储空间压力，另一方面也是为了容错降低数据恢复的代价。



上图有2个Executor，每个Executor有1个core，total-executor-cores为数为2，每个task的执行结果会被溢写到本地磁盘上。每个task包含R个缓冲区，R = reducer 个数（也就是下一个stage中task的个数），缓冲区被称为bucket，其大小为spark.shuffle.file.buffer.kb，默认是32KB。

其实bucket表示缓冲区，即ShuffleMapTask 调用分区器的后数据要存放的地方。

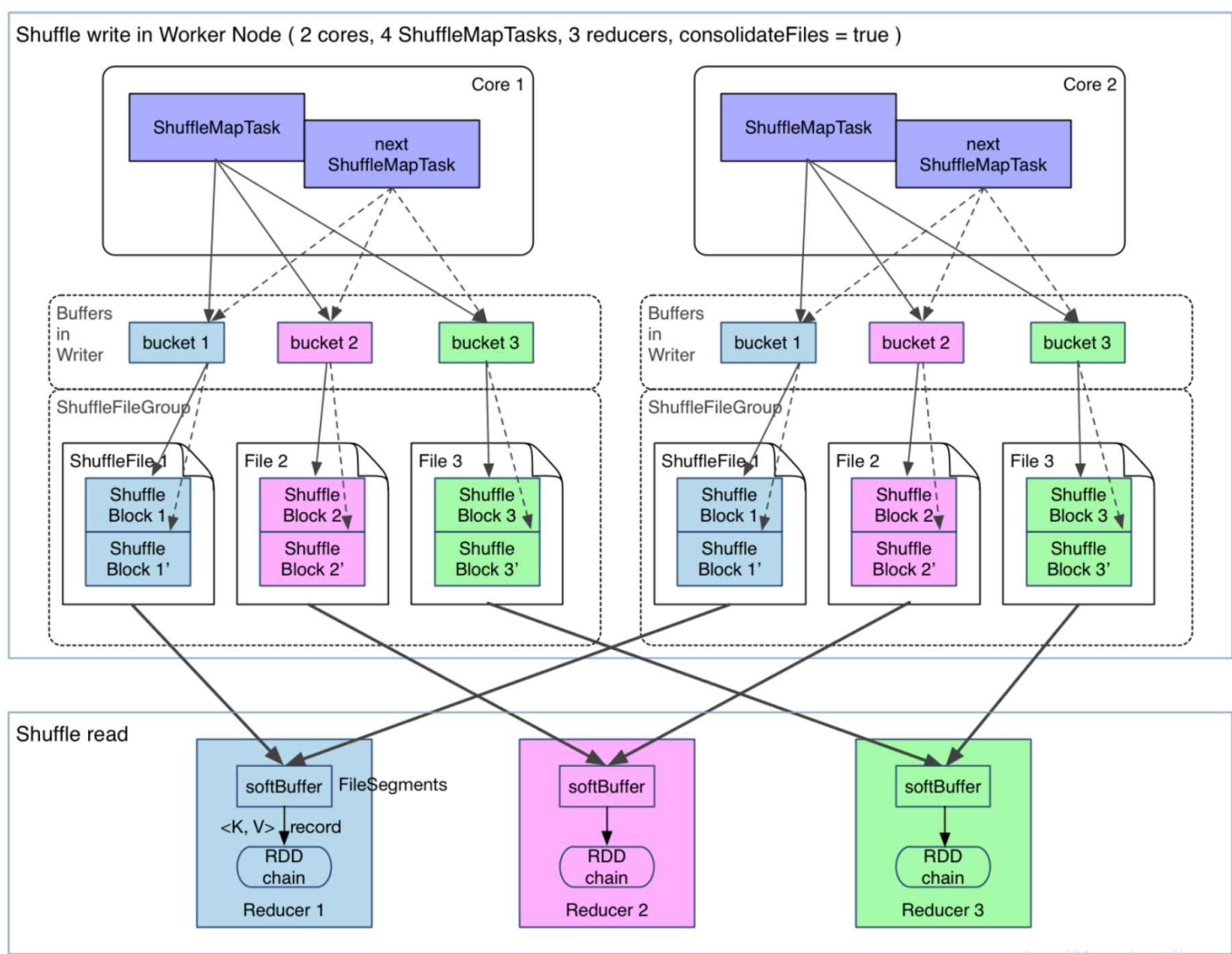
ShuffleMapTask 的执行过程：先根据 pipeline 的计算逻辑对数据进行运算，然后根据分区器计算出每一个record的分区编号。每得到一个 record 就将其送到对应的 bucket 里，具体是哪个 bucket 由 `partitioner.getPartition(record.getKey())` 决定。每个 bucket 里面的数据会满足溢写的条件会被溢写到本地磁盘上，形成一个 ShuffleBlockFile，或者简称 FileSegment。之后的下游的task会根据分区会去 fetch 属于自己的 FileSegment，进入 shuffle read 阶段。

老版本的HashShuffleManager存在的问题：

- 1.产成的 FileSegment 过多。每个 ShuffleMapTask 产生 R（下游Task的数量）个 FileSegment，M 个 ShuffleMapTask 就会产生 $M * R$ 个文件。一般 Spark job 的 M 和 R 都很大，因此磁盘上会存在大量的数据文件。
- 2.缓冲区占用内存空间大。每个 ShuffleMapTask 需要开 R 个 bucket，M 个 ShuffleMapTask 就会产生 $M * R$ 个 bucket。虽然一个 ShuffleMapTask 结束后，对应的缓冲区可以被回收，但一个 worker

node 上同时存在的 bucket 个数可以达到 $\text{cores} \times R$ 个，占用的内存空间也就达到了 $\text{cores} \times R \times 32 \text{ KB}$ 。对于 8 核 1000 个 reducer 来说，占用内存就是 256MB。

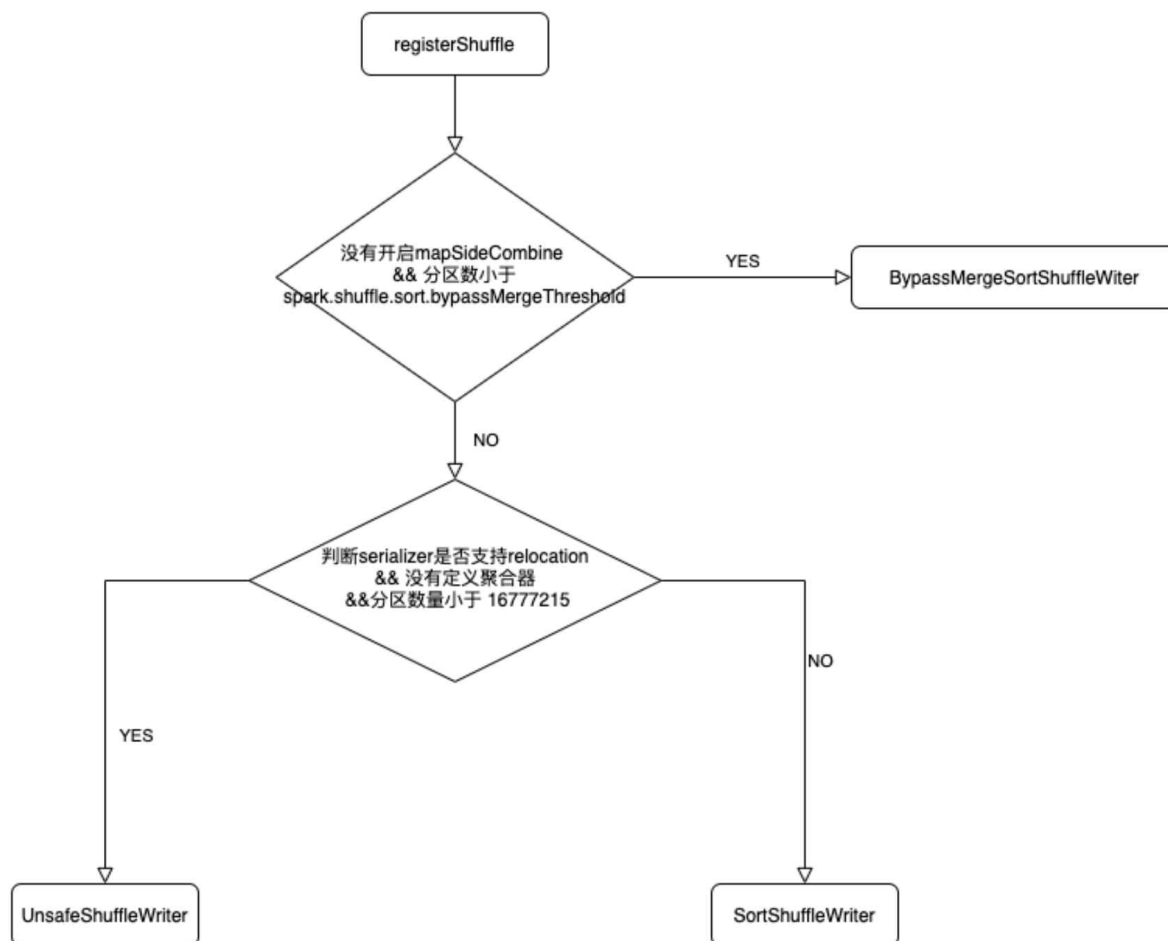
- 优化后的：



可以明显看出，在一个core上连续执行的ShuffleMapTasks可以共用一个输出文件 ShuffleFile。先执行完的 ShuffleMapTask 形成ShuffleBlock i，后执行的 ShuffleMapTask可以将输出数据直接追加到 ShuffleBlock i后面，形成ShuffleBlock'，每个ShuffleBlock被称为FileSegment。下一个stage的 reducer只需要fetch整个 ShuffleFile就行了。这样每个Executor持有的文件数降为 $\text{cores} \times R$ 。

FileConsolidation 功能可以通过`spark.shuffle consolidateFiles=true`来开启。

18.3 SortShuffleManager



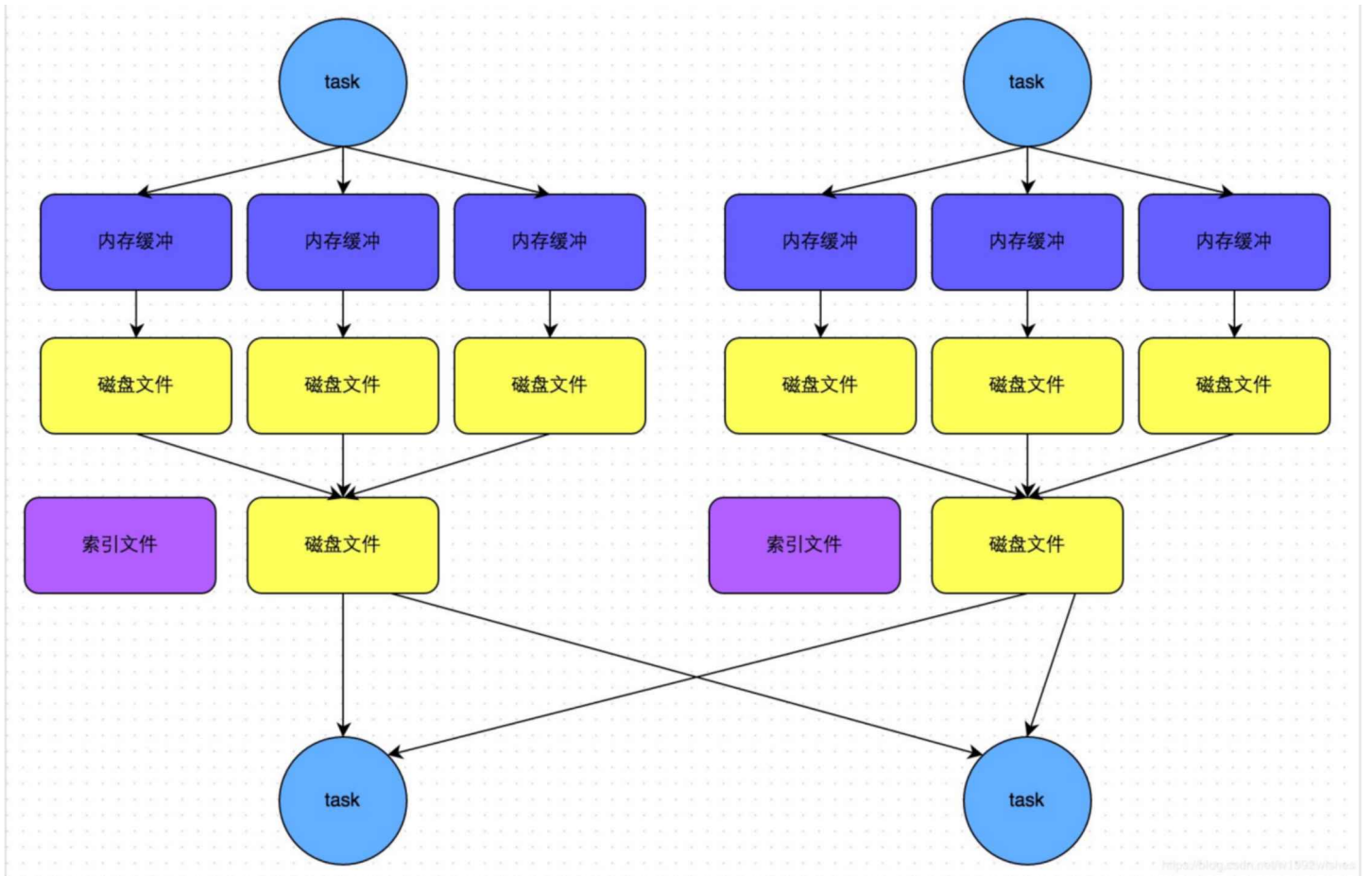
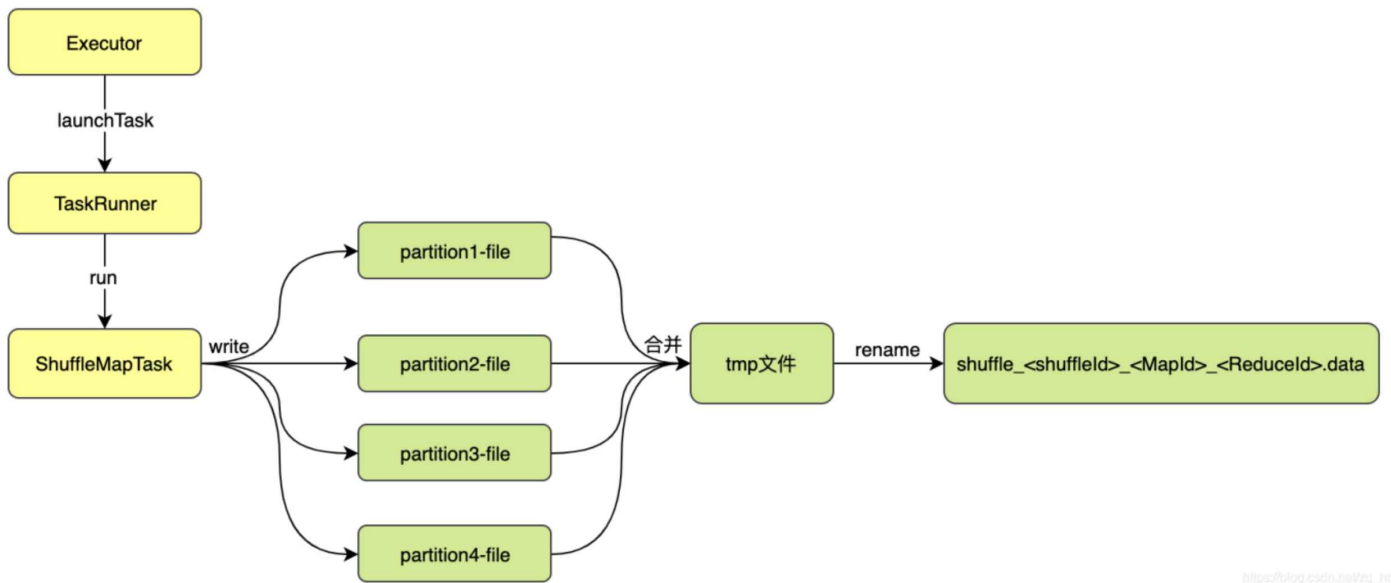
- `BypassMergeSortShuffleWriter`

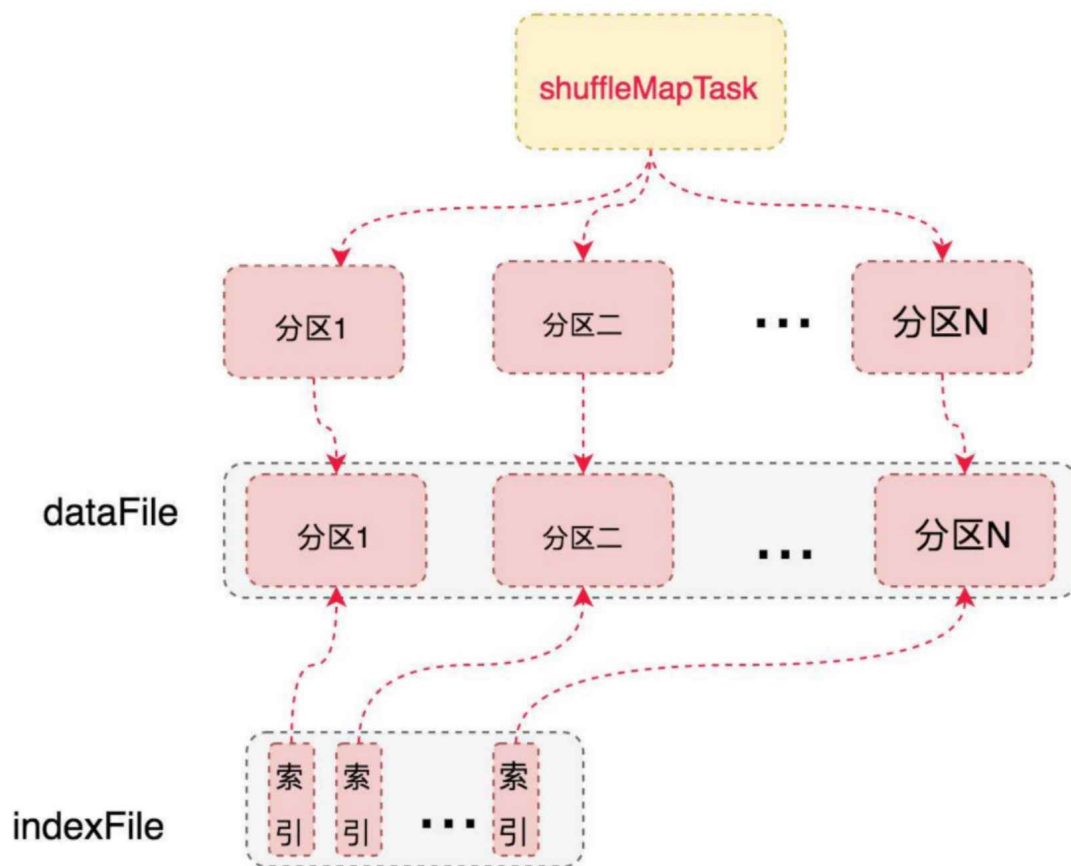
使用这种ShuffleWriter的条件是：

- (1) 没有map端的聚合操作
- (2) 分区数小于参数：`spark.shuffle.sort.bypassMergeThreshold`，默认是200

`BypassMergeSortShuffleWriter` 算法适用于没有聚合，数据量不大的场景。给每个分区分配一个临时文件，对每个 record 的 key 使用分区器（模式是hash，如果用户自定义就使用自定义的分区器）找到对应分区的输出文件并写入文件对应的文件。

因为写入磁盘文件是通过 Java 的 `BufferedOutputStream` 实现的，`BufferedOutputStream` 是 Java 的缓冲输出流，首先会将数据缓冲在内存中，当内存缓冲满溢之后再一次写入磁盘文件中，这样可以减少磁盘 IO 次数，提升性能。所以图中会有内存缓冲的概念。





<https://blog.csdn.net/w1532716161>

- `UnsafeShuffleWriter`

使用这种`ShuffleWriter`的条件是：

- `Serializer` 支持 relocation。`Serializer` 支持 relocation 是指，`Serializer` 可以对已经序列化的对象进行排序，这种排序起到的效果和先对数据排序再序列化一致。支持 relocation 的 `Serializer` 是 `KryoSerializer`，Spark 默认使用 `JavaSerializer`，通过参数 `spark.serializer` 设置；
- 没有指定 aggregation 或者 key 排序，因为 key 没有编码到排序指针中，所以只有 partition 级别的排序。
- partition 数量不能大于指定的阈值(2^{24})，因为 partition number 使用 24bit 表示的。即不能大于 `PackedRecordPointer.MAXIMUM_PARTITION_ID + 1`

`UnsafeShuffleWriter` 将 record 序列化后插入sorter，然后对已经序列化的 record 进行排序，并在排序完成后写入磁盘文件作为 spill file，再将多个 spill file 合并成一个输出文件。在合并时会基于 spill file 的数量和 IO compression codec 选择最合适的合并策

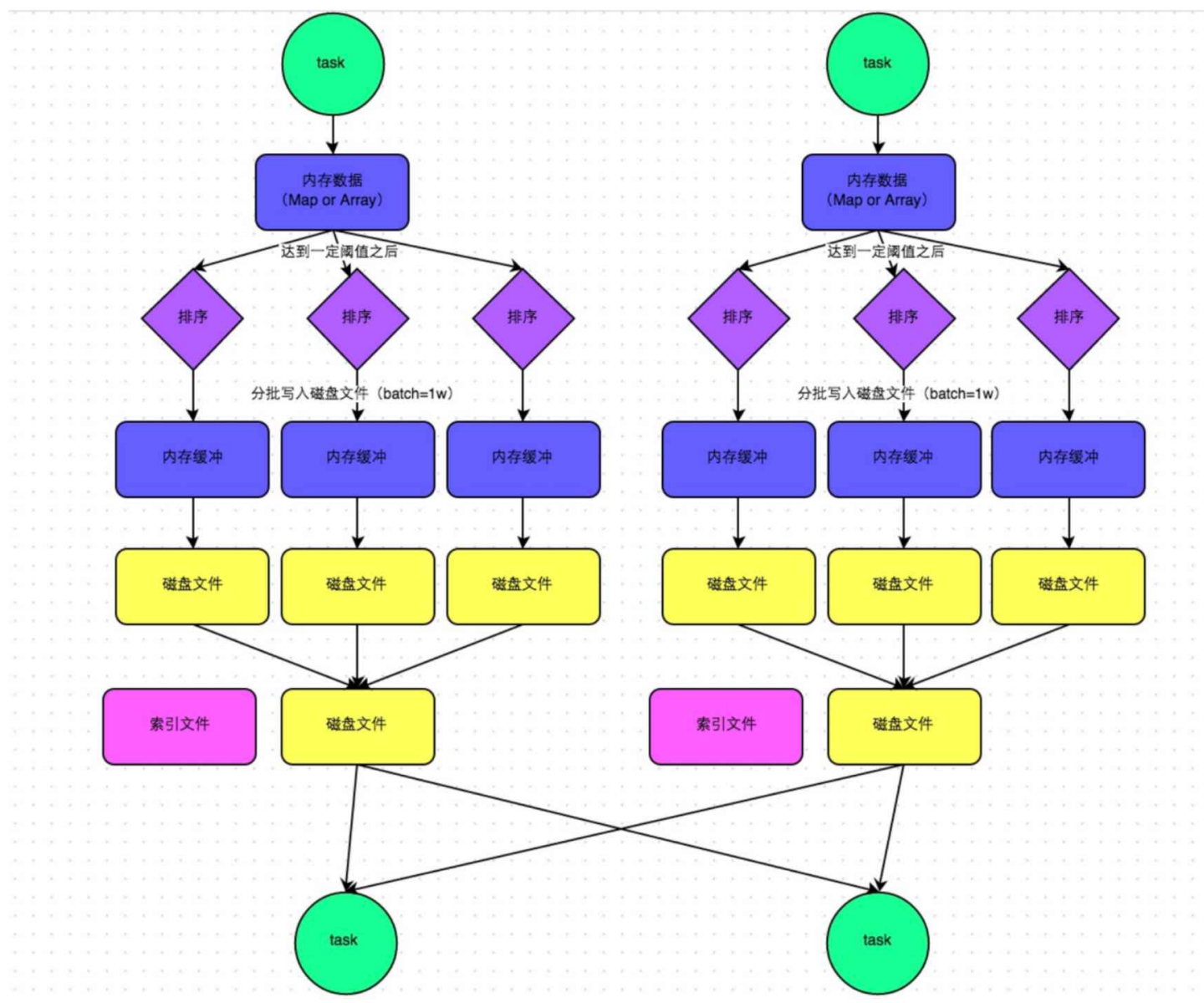
- `SortShuffleWriter`

若以上两种ShuffleWriter都不能选择，则使用SortShuffleWriter类，SortShuffleWriter也是相对比较常用的一种ShuffleWriter。

1.SortShuffleWriter会先把数据先写入到内存中，并会尝试扩展内存大小，若内存不足，则把数据持久化到磁盘上。

2.SortShuffleWriter在把数据写入磁盘时，会按分区ID进行合并，并对key进行排序，然后写入到该分区的临时文件中。

3.SortShuffleWriter最后会把前面写的分区临时文件进行合并，合并成一个文件，也就是说，会在map操作结束时把各个分区文件合并成一个文件。这样做可以有效的减少文件个数，和为了维护这些文件而产生的资源消耗。



在进行 shuffle 之前，map 端会先将数据进行排序。排序的规则，根据不同的场景，会分为两种。首先会根据 Key 将元素分成不同的 partition。第一种只需要保证元素的 partitionId 排序，但不会保证同一个 partitionId 的内部排序。第二种是既保证元素的 partitionId 排序，也会保证同一个 partitionId 的内部排序。

接着，往内存写入数据，每隔一段时间，当向 MemoryManager 申请不到足够的内存时，或者数据量超过 spark.shuffle.spill.numElementsForceSpillThreshold 这个阈值时（默认是 Long 的最大值，不起作用），就会进行 Spill 内存数据到文件，然后清空内存数据结构。假设可以源源不断的申请到内存，那么 Write 阶段的所有数据将一直保存在内存中，由此可见，PartitionedAppendOnlyMap 或者 PartitionedPairBuffer 是比较吃内存的。

在溢写到磁盘文件之前，会先根据 key 对内存数据结构中已有的数据进行排序。排序过后，会分批将数据写入磁盘文件。默认的 batch 数量是 10000 条，也就是说，排序好的数据，会以每批 1 万条数据的形式分批写入磁盘文件。写入磁盘文件也是通过 Java 的 BufferedOutputStream 实现的。

一个 task 将所有数据写入内存数据结构的过程中，会发生多次磁盘溢写操作，也就会产生多个临时文件。在将最终排序结果写入到数据文件之前，需要将内存中的 PartitionedAppendOnlyMap 或者 PartitionedPairBuffer 和已经 spill 到磁盘的 SpillFiles 进行合并。

此外，由于一个 task 就只对应一个磁盘文件，也就意味着该 task 为下游 stage 的 task 准备的数据都在这一个文件中，因此还会单独写一份索引文件，其中标识了下游各个 task 的数据在文件中的 start offset 与 end offset。

19. Spark思考题

- 1.SparkContext哪一端生成的？

Driver端即SparkContext（Driver是一个统称，里面还DAGSchedule、TaskScheduler等）

- 2.DAG是在哪一端被构建的？

Drvier端

- 3.RDD是在哪一端创建的？

Driver端，RDD不装真正要计算的数据，而是记录了数据的描述信息（以后从哪里读数据，怎么计算）

- 6.调用RDD的算子（Transformation和Action）是在哪一端调用的

Driver端

- 7.RDD在调用Transformation和Action时需要传入一个函数，函数是在哪一端声明【定义】和传入的？

Driver端

- 6.RDD在调用Transformation和Action时需要传入函数，请问传入的函数是在哪一端执行了函数的业务逻辑？

Executor中的Task指定的

- 9.Task是在哪一端生成的呢？

Driver端,Task分为ShuffleMapTask和ResultTask

- 10.DAG是在哪一端构建好的并被切分成一到多个Stage的

Driver

- 11.DAG是哪个类完成的切分Stage的功能？

DAGScheduler

- 12.DAGScheduler将切分好的Task以什么样的形式给TaskScheduler

TaskSet

- 13.分区器这个类是在哪一端实例化的？

Driver端

- 14.分区器中的getPartition方法在哪一端调用的呢？

Executror中的Task

- 15.广播变量是在哪一端调用的方法进行广播的？

Driver端

- 16.要广播的数据应该在哪一端先创建好再广播呢？

Driver端

- 17.广播变量以后能修改吗？

不能修改

- 18.广播变量广播到Executor后，一个Executor进程中有几份广播变量的数据

一份全部广播的数据